
A Greedy PDE Router for Blending Neural Operators and Classical Methods

Anonymous Author(s)

Affiliation

Address

email

1 **Revision note.** All text, tables and figures added in this revision are typeset in red (new figures
2 carry a red frame); everything in black is unchanged from the submitted version. The iteration-based
3 experiments of the submitted version on 31^2 and 63^2 grids with the LSTM router, and their appendix
4 analyses, have been removed: they are superseded by the wall-clock study of Sections E and 7,
5 which covers the same solvers and equations (plus anisotropic diffusion) on 128^2 – 512^2 grids and
6 also reports the iteration-based metrics. Only the Dirichlet experiment of the submitted version is
7 kept (Section D).

Abstract

8 When solving PDEs, classical numerical solvers are often computationally ex-
9 pensive, while machine learning methods can suffer from spectral bias, failing
10 to capture high-frequency components. Designing an optimal hybrid iterative
11 solver—where, at each iteration, a solver is selected from an ensemble of solvers to
12 leverage their complementary strengths—poses a challenging combinatorial prob-
13 lem. While greedy selection is desirable for its constant-factor approximation
14 guarantee to the optimal solution under Lipschitz and shared-eigenbasis assump-
15 tions, it requires knowledge of the true error at each step, which is unavailable in
16 practice. We address this by proposing an approximate greedy router that efficiently
17 mimics a greedy approach to solver selection. Empirical results on the Poisson,
18 convection–diffusion and anisotropic diffusion equations show that our method con-
19 sistently reduces final error and area-under-the-curve (AUC) of the error trajectory
20 relative to single-solver baselines and existing hybrid approaches such as HINTS,
21 reaching comparable error levels in substantially fewer iterations with more stable
22 error decay. With a cost-aware instantiation of the greedy rule, a lightweight router,
23 and a band-limited scale-equivariant corrector, these gains carry over to end-to-end
24 wall-clock time on 128^2 to 512^2 grids: in every one of the 15 solver–equation pair-
25 ings at 128^2 (five classical solvers; Poisson, convection–diffusion and anisotropic
26 diffusion) the learned router reaches truncation-level accuracy $238\times$ – $1983\times$ faster
27 than the classical solver alone (which never reaches it on the anisotropic equation)
28 and $2.68\times$ – $11.8\times$ faster than HINTS, it beats HINTS on every grid even when
29 the latter’s period is tuned per setting, and it is faster than geometric multigrid and
30 multigrid-preconditioned Krylov methods on the isotropic equations; routing over
31 ensembles of classical solvers is never worse than the best pair and strictly better
32 where the error passes through regimes that favour different solvers.

1 Introduction

34 Natural phenomena and engineered systems are often governed by ordinary and partial differential
35 equations (PDEs). Solving these equations enables a variety of tasks - predicting the evolution of the
36 system through simulation (e.g., forecasting weather using the Navier-Stokes equations) [Kalnay,
37 2003, Bauer et al., 2015, Staniforth and Côté, 1991], addressing control problems (e.g., optimizing

38 heat shield design for spacecrafts using heat transfer equations) [Anderson, 1989, Tröltzsch, 2010],
 39 and tackling inverse problems based on external measurements (e.g., reconstructing brain activity
 40 from EEG data using electrophysiological models) [Baillet et al., 2001, Grech et al., 2008].

41 Traditionally, PDEs are solved using finite difference methods [Smith, 1985, LeVeque, 2007], which
 42 discretize the spatial and/or temporal domain and approximate the solution by solving a system of
 43 equations at the discrete grid points. Similarly, finite element methods [Hughes, 2003, Bathe, 2006,
 44 Brenner, 2008] construct a system of equations based on local basis functions, and then rely on
 45 numerical linear algebra techniques—such as the Jacobi and Gauss-Seidel method [Saad, 2003]—to
 46 compute the solution. These iterative methods, however, **must be re-run from scratch for every new**
 47 initial condition, boundary condition, or forcing function, as even minor changes to any of these
 48 parameters require re-solving the entire system of equations from scratch.

49 These challenges have motivated the development of neural operators [Kovachki et al., 2023]—a class
 50 of machine learning models that aim to learn the solution operator directly, enabling fast inference
 51 across a range of parameters. Neural operators lift the classical linear layer in neural networks to an
 52 operator—typically a kernel integral operator—between function spaces. Despite strong approximation
 53 guarantees [Chen and Chen, 1995], neural operators are prone to spectral bias [Liu and Cai, 2021, Liu
 54 et al., 2024, You et al., 2024, Khodakarami et al., 2025, Xu et al., 2025], similar to traditional neural
 55 networks [Rahaman et al., 2019, Xu et al., 2019, Luo et al., 2019]. As a result, they favor learning
 56 low-frequency components of the solution function and often struggle to capture high-frequency
 57 features that iterative solvers excel at.

58 Recognizing this complementarity, Zhang et al. [2024] proposed HINTS, a hybrid solver that inter-
 59 leaves a neural operator pass every τ^{th} iteration of a classical solver, with τ fixed a priori. While
 60 HINTS reports significant empirical gains in convergence and accuracy over the standalone neural
 61 operator and Jacobi solver, this fixed schedule can be detrimental: a poorly timed neural correction
 62 may increase the error and undo recent progress.

63 Although state-dependent solver schedules are desirable, this naturally suggest a reinforcement
 64 learning formulation for learning routing policies. However, in our setting, the structure of iterative
 65 PDE solvers allows for a simpler approach: each solver induces a known update, and its effect can be
 66 evaluated through error-based signals. Leveraging this, we adopt a greedy rule that selects, at each
 67 iteration, the solver with the largest immediate error reduction. Such a rule can be near-optimal when
 68 the final error satisfies (weak) supermodularity—so that applying a beneficial solver earlier is at least
 69 as valuable as applying it later. To support this approach, we make the following contributions:

- 70 • We present a general hybrid PDE solver in which a routing rule chooses, at each iteration,
 71 from a set of classical solvers and neural operators. With oracle access to the true error at
 72 every step, we show that a greedy routing rule achieves a constant-factor approximation to
 73 the optimal strategy for linear PDEs, provided the updates are error-reducing (Lipschitz with
 74 constant < 1) and zero-preserving—conditions under which weak supermodularity holds—
 75 **and share an orthogonal eigenbasis, which gives the postfix monotonicity the guarantee also**
 76 **needs.**
- 77 • Given that the true error is not observed at test time, a convex surrogate loss is introduced,
 78 which, when minimized, enables the learned model to imitate the greedy router without
 79 access to true error information. This alignment is guaranteed through Bayes consistency.
- 80 • We empirically demonstrate that our approximate greedy solver achieves faster convergence
 81 than HINTS and individual solvers on the Poisson, convection-diffusion and anisotropic
 82 diffusion equations, **in iterations and in end-to-end wall-clock time on 128^2 to 512^2 grids,**
 83 **where it is also faster than geometric multigrid and multigrid-preconditioned Krylov methods**
 84 **and extends to ensembles of classical solvers.**

85 2 Background

86 Consider the following linear PDE:

$$\mathcal{L}_x^a(u) = f, x \in D; \quad \mathcal{B}_x(u) = b, x \in \partial D \quad (1)$$

87 where $\mathcal{L}_x^a$ is a differential operator with respect to the spatial variable $x \in D$ parameterized by the
 88 coefficient function a , f is a forcing function, u is the PDE solution, $\mathcal{B}_x$ is the boundary operator, and

89 b is the boundary term. Such PDEs can be solved numerically using various discretization strategies,
 90 such as finite differences [Smith, 1985, LeVeque, 2007], finite element, and spectral methods [Boyd,
 91 2001, Canuto et al., 2006]. Here, we focus on finite differences, which replace derivatives with
 92 difference quotients (e.x., $\partial_x u(x) \approx (u(x+h) - u(x))/h$) on a uniform grid.

93 Formally, let h be the grid size and $G_h(D)$ be a uniform grid with spacing h over D . For a function
 94 $g : D \rightarrow \mathbb{R}$, we denote its restriction to $G_h(D)$ by $g_h \in \mathbb{R}^N$, where $N = |G_h(D)|$. If $\mathcal{L}_h^a \in \mathbb{R}^{N \times N}$
 95 is the discretized operator, the PDE reduces to a linear system of equations

$$\mathcal{L}_h^a u_h = f_h, \quad (2)$$

96 with the boundary conditions incorporated in $\mathcal{L}_h^a$. Unlike much of the neural operator literature, which
 97 retains function space formulations for discretization invariance, we follow the conventions of classical
 98 numerical analysis and represent functions and operators as vectors and matrices. Discretization
 99 invariance is not essential here, since we focus on fixed-grid problems.

100 2.1 Iterative PDE Solvers

101 Direct methods like Gaussian elimination and Thomas Algorithm can be computationally expensive in
 102 high-dimensional domains when solving systems like Equation (2). In contrast, iterative methods like
 103 Jacobi and Gauss-Seidel offer computational speedups by iteratively updating the solution, gradually
 104 converging to the true solution for the PDE. The general iterative update is

$$u^{(t+1)} = u^{(t)} + C \left(f_h - \mathcal{L}_h^a u^{(t)} \right), \quad (3)$$

105 where $u^{(t)}$ is the t^{th} iterate of the solution and C is a preconditioning matrix. Jacobi uses $C = D^{-1}$,
 106 where D is the diagonal of $\mathcal{L}_h^a$, and Gauss-Seidel uses $C = (D + L)^{-1}$, where L denotes the strict
 107 lower triangular part. However, these methods tend to damp low frequency parts of the error slowly.

108 2.2 Neural Operators

109 Suppose we observe samples $\{(a_i, f_i, u_i)\}_{i=1}^N$ such that $(a_i, f_i) \sim \mathcal{P}$ are i.i.d. samples, and $u_i =$
 110 $\mathcal{G}^*(a_i, f_i)$ be generated by a deterministic solution operator $\mathcal{G}^*$. A neural operator (NO) $\mathcal{G}_\theta$ seeks to
 111 approximate $\mathcal{G}^*$ by minimizing the expected squared $L^2(D)$ error:

$$\mathcal{R}_{\text{NO}}(\mathcal{G}_\theta) = \mathbb{E}_{(a,f) \sim \mathcal{P}} \left[\int_D \|u_i(x) - \mathcal{G}_\theta(a_i, f_i)(x)\|_2^2 dx \right]$$

112 Neural operators use discretization-invariant layers. Prominent instantiations include Fourier Neural
 113 Operators (FNO), which use spectral transforms [Li et al., 2020a], Graph Neural Operators, which
 114 perform graph-based aggregation over sampled points [Li et al., 2020b], and DeepONets, which
 115 employ a trunk-branch decomposition to map input functions to output functions [Lu et al., 2021].

116 2.3 Greedy Optimization

117 Consider maximizing $g(S)$ over $S \subseteq \Omega$ with $|S| \leq T$ where $g : 2^\Omega \rightarrow \mathbb{R}$ is a set function defined
 118 on subsets of a set Ω . An exhaustive search is infeasible, so an efficient alternative is the greedy
 119 algorithm. It builds the solution subset iteratively by adding ω^* that yields the largest marginal
 120 gain, i.e. $\omega^* = \arg \max_{\omega \in \Omega \setminus S} g(S \cup \omega)$. When g is non-negative, monotone (adding elements
 121 never decreases the value), and submodular (diminishing returns), the greedy algorithm achieves a
 122 $(1 - e^{-1})$ approximation to the optimal solution [Nemhauser et al., 1978]. Formally, a function g
 123 is submodular if $g(A \cup \{\omega\}) - g(A) \geq g(B \cup \{\omega\}) - g(B)$ for all $A \subseteq B \subseteq \Omega$ and $\omega \in \Omega \setminus B$;
 124 intuitively, delaying an addition cannot increase its benefit. For set function minimization problems,
 125 supermodularity, where the inequality flips, plays an analogous role, and the greedy rule achieves
 126 constant-factor approximation guarantees [Liberty and Sviridenko, 2017].

127 The suboptimality of the greedy algorithm has been extensively studied for both set-function mini-
 128 mization [Bounia and Koriche, 2023] and maximization [Das and Kempe, 2018, Feige et al., 2011,
 129 Bian et al., 2017, Harshaw et al., 2019] when standard assumptions—non-negativity, monotonicity, and
 130 sub/supermodularity—are weakened. In contrast, results on greedy sequence maximization [Streeter
 131 and Golovin, 2008, Alaei et al., 2021, Zhang et al., 2015, Bernardini et al., 2020, Van Over et al.,
 132 2024, Tschitschek et al., 2017]—where the ordering of the elements affects the function—have led
 133 to sequential analogues of submodularity and monotonicity. We leverage these tools to analyze the
 134 suboptimality of the greedy solution to minimizing the final error.

3 General framework for Hybrid Solvers

To solve Equation (2), consider the following hybrid iterative update:

$$u_h^{(t+1)} = u_h^{(t)} + C_{S_t} \left(f_h - \mathcal{L}_h^a u_h^{(t)} \right) \quad (4)$$

where $\mathcal{C} = \{C_j\}_{j=1}^K$ is a set of preconditioning functions and $S_t \in [K] = \{1, \dots, K\}$ indexes the function chosen at step t . Here, we use “preconditioning function” broadly to include classical linear updates ($C_j(x) = C_j x$ where C_j is a matrix), and learned models, such as neural operators. This update generalizes the classical iterative update in Equation (3), by allowing C_j to be non-linear and enabling the preconditioning function to be adaptively chosen at every step. $\mathcal{C}$ can also accommodate parameterized solver families (e.g., different Jacobi relaxation weights), enabling adaptive selection of solver parameters. As the number of solvers K grows, the chance of selecting a more effective update increases. Furthermore, HINTS [Zhang et al., 2024] is a special case of this hybrid solver with $K = 2$, where C_1 is a neural operator and C_2 is a classical preconditioner. Its routing rule is given by $S_t = \mathbf{1}_{t \bmod \tau > 0} + 1$, which selects C_1 every τ steps and C_2 otherwise.

Let $e_h^{(t)} = u_h - u_h^{(t)}$ denote the error at step t . Then,

$$e_h^{(t+1)} = u_h - u_h^{(t)} - C_{S_t} \left(\mathcal{L}_h^a u_h - \mathcal{L}_h^a u_h^{(t)} \right) = (I - C_{S_t} \circ \mathcal{L}_h^a) (e_h^{(t)})$$

where I is the identity map and $I - C_j \circ \mathcal{L}_h^a$ is the error propagation function for solver j . Here, “ $\circ$ ” denotes function composition (matrix multiplication for linear updates). The objective is to select a sequence $S = (S_1, \dots, S_T)$ that minimizes the error norm after T steps or $\|e_h^{(T)}\|_2^2$:

$$\min_{S_t \in [K], |S| \leq T} h(S) := \left\| (I - C_{S_{|S|}} \circ \mathcal{L}_h^a) \circ \dots \circ (I - C_{S_1} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \quad (5)$$

with compositions applied from right to left, so that the S_1 update acts on the initial error.

4 Greedy Algorithm

Equation (5) defines a combinatorial optimization problem with a search space exponential in T , making exact optimization intractable for large T or K . As a starting point, we consider an “omniscient” greedy algorithm that assumes access to the true initial error $e_h^{(0)}$. This is unrealistic since knowledge of $e_h^{(0)}$ could directly recover the solution via $u_h = u_h^{(0)} + e_h^{(0)}$, but it provides a useful benchmark. In Section 5, we relax this assumption using a practical learning strategy that is Bayes consistent with this omniscient rule, thereby recovering the guarantees shown below.

As discussed in Section 2.3, when supermodularity and monotonicity hold, the greedy rule—such as the one described in Algorithm 1—enjoys constant-factor approximation guarantees. However, classical results focus on *set* functions, with only recent extensions made to sequences. Building on this line, we introduce a sequence-based notion of weak supermodularity.

Algorithm 1 Greedy Algorithm for a Hybrid PDE solver

Require: $\{C_j\}_{j=1}^K, T, \mathcal{L}_h^a, e_h^{(0)}$
 $S^0 \leftarrow \emptyset$
for $t < T$ **do**
 $S^{t+1} \leftarrow S^t \oplus \arg \min_{j \in [K]} \|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$
 $e_h^{(t+1)} \leftarrow (I - C_{S^{t+1}} \circ \mathcal{L}_h^a)(e_h^{(t)})$
end for
return S^T

Let Ω^* denote the space of sequences with elements in Ω . For $S, S' \in \Omega^*$, we denote their concatenation as $S \oplus S'$. S is a prefix of S' or $S \preceq S'$ if $S' = S \oplus L$ for some $L \in \Omega^*$. A function $g : \Omega^* \rightarrow \mathbb{R}$ is prefix monotonically non-increasing if $g(S \oplus S') \leq g(S)$ for all $S, S' \in \Omega^*$, and postfix monotonically non-increasing if $g(S \oplus S') \leq g(S')$ for all $S, S' \in \Omega^*$. A prefix non-increasing function g is sequence supermodular if, for all $S', S \in \Omega^* : S \preceq S'$, it holds that

$$g(S) - g(S \oplus \omega) \geq g(S') - g(S' \oplus \omega), \quad \forall \omega \in \Omega \quad (6)$$

168 However, $h(S)$ (defined in Equation (5)) may not satisfy this property in general. Therefore, we
 169 introduce weak sequence supermodularity. A prefix non-increasing function g is weakly supermodular
 170 with respect to $S' \in \Omega^*$ if, for any $S \in \Omega^*$, there exists $\alpha(S') \geq 1$ such that

$$g(S) - g(S \oplus S') \leq \alpha(S') \sum_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \quad (7)$$

171 The parameter $\alpha(S')$ or the supermodularity ratio quantifies deviation from exact sequence super-
 172 modularity. Expanding the $g(S) - g(S \oplus S')$ as a telescoping sum $\sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) -$
 173 $g(S \oplus (S'_1, \dots, S'_i))$ shows that the marginal decrease from appending S'_i after its predecessors is
 174 controlled by the effect of appending S'_i directly to S . Thus, postponing the inclusion of S'_i cannot
 175 yield a significantly larger benefit than adding it earlier. The supermodularity ratio $\alpha(S)$ quantifies
 176 the extent to which future gains from delays may exceed immediate gains.

177 Having introduced these notions, we now characterize the suboptimality of greedy solutions of weakly
 178 supermodular and postfix monotonic sequence functions in Theorem 4.1.

179 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal*
 180 *solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity ratio of $\alpha(O)$ and postfix monotonicity. Let*
 181 *the greedy solution of length T be S^T . If $\phi_T(\alpha) = (1 - \frac{1}{\alpha T})^T$, then*

$$g(\emptyset) - g(S^T) \geq (1 - \phi_T(\alpha(O))) (g(\emptyset) - g(O))$$

182 The proof of Theorem 4.1 appears in Section A.2. As $T \rightarrow \infty$, the factor $1 - \phi_T(\alpha(O))$ decreases to
 183 $1 - e^{-1/\alpha(O)}$, so the worst-case performance of the greedy rule saturates rather than degrades with
 184 horizon. Additionally, larger $\alpha(O)$, which indicates higher reward for delayed inclusions, loosens the
 185 suboptimality bound. Theorem 4.1 requires that the sequence objective h be weakly supermodular
 186 with respect to the optimal solution and postfix monotone—properties established in Proposition 4.2.
 187 We defer the proof to Section A.3.

188 **Proposition 4.2.** *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is*
 189 *ρ_j -Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is*
 190 *weakly supermodular with respect to the optimal solution O , with*

$$\alpha(O) = \max \left\{ \frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

191 *Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by an*
 192 *orthogonal matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also*
 193 *postfix non-increasing; invertibility alone does not suffice (Section A.3).*

194 The conditions of Proposition 4.2 are natural. For classical solvers, the Lipschitz constant is $\|I_N -$
 195 $C_j \mathcal{L}_h^a\|$, which is typically less than 1 for well-posed linear elliptic PDEs (i.e., the update is damping
 196 errors). For neural networks, Lipschitz continuity can be enforced via weight regularization [Gouk
 197 et al., 2021] and a sufficiently trained model should approximate $(\mathcal{L}_h^a)^{-1}$ well enough to make the
 198 Lipschitz constant small. The requirement $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$ is both natural and desirable: it
 199 precludes spurious updates when the residual $f_h - \mathcal{L}_h^a u_h^{(t)}$ is 0. This holds by design for classical
 200 schemes, and it can be enforced for any learned model by excluding bias terms.

201 The form of $\alpha(O)$ in Proposition 4.2 highlights that the suboptimality factor in Theorem 4.1 is
 202 governed by the collective contraction factors of the solvers chosen by the optimal solution: as
 203 $\sum_{i=1}^T \rho_{O_i}^2 \rightarrow T$, $\alpha(O)$ grows, resulting in a weaker bound. **Postfix monotonicity is the restrictive con-**
 204 **dition: it holds when the error propagation maps share an orthogonal eigenbasis—constant-coefficient**
 205 **periodic operators with Jacobi-type smoothers and band-limited spectral correctors (Section E.8)—but**
 206 **it can fail for non-commuting contractions even when all of them are invertible, and a Gauss–Seidel**
 207 **sweep or a learned corrector need not be invertible at all. Our experiments show that greedy routers**
 208 **remain effective beyond this setting.**

209 Theorem 4.1 thus indicates that the approximation guarantee is strongest when the sequence is
 210 nearly supermodular ($\alpha(O) \approx 1$). Generally, if all error propagation maps share an eigenbasis,
 211 supermodularity is established. **This occurs, for example, for linear, constant-coefficient PDEs with**
 212 **periodic boundary conditions when the solver ensemble consists of (damped) Jacobi iterations and**

band-limited spectral correctors such as a single-layer linear Fourier neural operator, all of which are diagonal in the Fourier basis; lexicographic Gauss–Seidel and SOR are not, because their sweep order breaks translation invariance. The proof of Theorem 4.3 is deferred to Section A.5.

Proposition 4.3. *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common orthogonal matrix P . Then, h is supermodular.*

Cost-aware greedy routing. Algorithm 1 treats every update as equally expensive, whereas in practice a neural-operator evaluation and a classical sweep have very different costs, and the quantity of interest is the error reached per unit of wall-clock time. We make the greedy rule cost-aware without changing the analysis: let c_j denote the (measured) cost of one application of C_j , take the cost u of one neural-operator call as the unit, and replace each preconditioner by the *macro-action* $C_j^{(m_j)}$ that applies C_j for $m_j = \max\{1, \text{round}(u/c_j)\}$ consecutive iterations, so that all actions cheaper than the unit cost approximately u ; an action dearer than the unit (a multigrid cycle, say) is applied once and its error is compared per unit of cost, $\|e\|^{u/(m_j c_j)}$, which reduces to the plain comparison when the costs match. For a linear update, m consecutive sweeps are again a stationary iteration, $(I - C_j \circ \mathcal{L}_h^a)^m = I - C_j^{(m)} \mathcal{L}_h^a$ with $C_j^{(m)} = \sum_{i=0}^{m-1} (I - C_j \mathcal{L}_h^a)^i C_j$, and for a nonlinear C_j the macro-action is the m -fold composition of its error propagation map. Hence $\{C_j^{(m_j)}\}_{j=1}^K$ is again a set of preconditioning functions in the sense of Section 4: if $I - C_j \circ \mathcal{L}_h^a$ is ρ_j -Lipschitz and zero-preserving, then so is its m_j -fold composition (with constant $\rho_j^{m_j}$), a shared eigenbasis is inherited by powers, and Theorem 4.1, Propositions 4.2 and 4.3, and the consistency result of Section 5 apply verbatim to an ensemble of macro-actions that are equalised to the unit cost. Running Algorithm 1 on such macro-actions selects, at every decision epoch, the action with the largest error reduction per unit cost, and a horizon of T macro-actions corresponds to a wall-clock budget of $\approx Tu$. Two caveats: the equalisation is up to rounding (within 8% for the stationary solvers of Section E), and for an action dearer than the unit the per-unit-cost comparison is a geometric-rate heuristic rather than an equal-cost step, so the multigrid pairings of Section E are covered empirically, not by the equal-cost analysis. Section E uses this cost-aware instantiation throughout.

5 Approximate Greedy Router

The results in Section 4 indicate that the error reduction from the greedy solution closely matches that of the optimal sequence, but this is predicated on having access to the initial error, $e_h^{(0)}$. Inaccurate error estimates can result in poor solver decisions, causing errors to amplify. To remedy this, we learn a router r that selects solvers myopically, as in Algorithm 1, without access to true errors.

We adopt the following learning setup. Let $\mathcal{A}, \mathcal{F}, \mathcal{U} \subseteq \mathbb{R}^N$ denote the spaces of coefficient, forcing, and solution functions on the grid $G_h(D)$. We assume an application-specific data distribution $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$ over $\mathcal{A} \times \mathcal{F}$ that reflects test time conditions. During training, (a_h, f_h) is drawn from $\mathcal{P}_{\mathcal{A} \times \mathcal{F}}$, and a high-accuracy reference solution u_h is computed, providing the true per-step error. The router r is learned offline on this data; at test time, it operates without access to true errors.

At iteration t , router r selects a solver using the coefficient $a_h \in \mathcal{A}$, forcing $f_h \in \mathcal{F}$, and the current iterate $u_h^{(t)} \in \mathcal{U}$. If $r(a_h, f_h, u_h^{(t)}) = j$, the next iterate is computed with solver j , resulting in an error of $\|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$. Learning such a router requires minimizing the following loss:

$$l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h) = \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)})=j} \quad (8)$$

with risk $\mathcal{R}_{\text{route}}(r) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} [l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h)]$. For analysis, we fix the iterate generation via teacher-forcing [Williams and Zipser, 1989, Lamb et al., 2016]: during training, the iterates fed to the router are produced by an oracle greedy rollout, not by the router’s own past choices. Formally, with $u_h^{(0)} = 0_N$, for $t \geq 1$,

$$j_t^* \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a) \left(e_h^{(t-1)} \right) \right\|_2^2, \quad u_h^{(t)} = u_h^{(t-1)} + C_{j_t^*} \left(f_h - \mathcal{L}_h^a u_h^{(t-1)} \right)$$

Thus, $\{u_h^{(t)}\}$ is a deterministic function of (a_h, f_h) . At each t , l_{route} is evaluated on $r(a_h, f_h, u_h^{(t)})$ which takes in the teacher-forced iterate. Finally, the greedy choice j_{t+1}^* is used to advance $u_h^{(t+1)}$.

Thus the input distribution seen by r depends on (a_h, f_h) , not on the router's predictions. However, full teacher forcing induces a distributional mismatch at test time (exposure bias). In experiments, we mitigate this with scheduled sampling [Bengio et al., 2015]; see Section C for details.

Minimizing $\mathcal{R}_{\text{route}}(r)$ yields the following Bayes-Optimal Router:

$$r^*(a_h, f_h, u_h^{(t)}) \in \operatorname{argmin}_{j \in [K]} \left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 \quad (9)$$

which aligns with the update in Algorithm 1. Hence, l_{route} is consistent with learning the greedy rule.

5.1 Surrogate Loss

Since Equation (8) is discontinuous and non-convex, direct minimization is intractable in practice. We instead introduce a surrogate loss that (1) is convex, enabling efficient optimization; (2) upper bounds the original loss; and (3) is Bayes consistent, ensuring the Bayes optimal decision of the original loss is preserved upon minimization. Formally, a surrogate ϕ is considered to be Bayes consistent with respect to the loss l if

$$\lim_{n \rightarrow \infty} \mathcal{R}_\phi(f_n) - \mathcal{R}_\phi^* \implies \lim_{n \rightarrow \infty} \mathcal{R}_l(f_n) - \mathcal{R}_l^*$$

where $\mathcal{R}_l(f) = \mathbb{E}[l(f(X), Y)]$ and $\mathcal{R}_l^* = \inf_f \mathcal{R}_l(f)$. In other words, in the limit of infinite data, if the risk of a sequence of learned hypotheses $\{f_n\}$ converges to the optimal risk under ϕ , it also converges to the optimal risk with respect to the original loss l .

To define a surrogate loss for the routing problem, consider a set of scoring functions $\mathbf{g} = \{g_j\}_{j=1}^K$ with $g_j : \mathcal{A} \times \mathcal{F} \times \mathcal{U} \rightarrow \mathbb{R}$ and we define the router as $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$. For example, $\mathbf{g}$ can be a neural network with K outputs. Then, minimizing the following surrogate loss yields the same decision as minimizing Equation (8):

$$\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h) = - \sum_{j=1}^K \sum_{k=1}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h) \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{m=1}^K \exp(g_m(a_h, f_h, u_h^{(t)}))} \right) \quad (10)$$

where $\tilde{c}_j(a_h, u_h^{(t)}, u_h) = \|(I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)})\|_2^2$. The Ψ -risk is denoted by $\mathcal{R}_\Psi(\mathbf{g}) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}}[\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)]$. The convexity of Ψ with respect to $\mathbf{g}$ follows from the convexity of log-softmax function in its inputs. Moreover, Ψ upper bounds l_{route} up to a constant factor, and we refer the reader to Section B.2 for the proof. Finally, Theorem 5.1 shows that Ψ achieves Bayes consistency with respect to l_{route} ; the proof can be found in Section B.3.

Theorem 5.1. *Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{\min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route} .*

Theorem 5.1 is a cost-sensitive analogue of the classical Bayes-consistency of multiclass cross-entropy for 0-1 loss: in the infinite-sample limit, minimizing cross-entropy recovers the true conditional class probabilities, so the induced decision is Bayes optimal. Our result extends this to cross-entropy with instance-dependent weights $\sum_{k \neq j}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h)$. The uniform upper bound holds when all preconditioning functions are error-damping. It is also reasonable to assume at least one solver cannot annihilate the error in one step, yielding the lower bound. Under these conditions, minimizing Ψ recovers the Bayes-optimal router of Equation (9), i.e., the greedy solution of Equation (5). Following the work of Mao et al. [2024], the proof uses standard conditional-risk calibration: we relate the excess risks of l_{route} and Ψ and take the infinite-sample limit. **This is a per-decision statement on the state distribution the router is trained on: it says that minimising Ψ recovers the greedy decision at each visited state, not that the induced sequence, its time to tolerance, or its distance to the optimal sequence are controlled; the latter would require a sequential argument about the router's own state distribution, which we do not provide and mitigate empirically with DAGger (Section E).**

6 Related Works

Hybrid PDE Solvers: Early data-driven solvers sought convergence guarantees by predicting parameters—e.g., preconditioning matrices, multi-grid smoothers or restriction matrices—within itera-

tive schemes [Taghibakhshi et al., 2021, Caldana et al., 2024, Kopaničáková and Karniadakis, 2025, Huang et al., 2022, Katrutsa et al., 2020]. These works, however, do not leverage the neural surrogates’ ability to generalize across varying coefficients and forcings highlighted in Section 2.2 and thus, offer only modest speedups over classical solvers. HINTS [Zhang et al., 2024] introduced hybrid solvers that interleave a classical method with a pre-trained DeepONet on a fixed schedule (e.g., 24 Jacobi steps, then one DeepONet correction), achieving faster convergence than the standalone numerical solver. Subsequent works extend this idea to new geometries [Kahana et al., 2023] and analyze error mode damping, replacing DeepONet with alternatives such as MIONet [Jin et al., 2022] [Hu and Jin, 2025] or FNO [Li et al., 2020a, Cui et al., 2022]. However, these hybrid solvers are limited in two ways: they, firstly, only combine the trained surrogate with a *single* numerical solver, and secondly rely on a fixed, heuristic schedule. Our proposed method addresses these shortcomings, resulting in significant empirical improvements (Section 7).

Model Routing: Routing [Shnitzer et al., 2023, Hu et al., 2024, Ding et al., 2024, Huang et al., 2025] selects, for each input, a model from a fixed set to optimize a task metric under cost or latency constraints. Simple heuristics—e.g., thresholding a cheap model’s uncertainty estimates [Chuang et al., 2024, 2025]—often poorly balance the cost-accuracy tradeoff, motivating many systems to instead learn a router that maps inputs to the best model [Hari and Thomson, 2023, Mohammadshahi et al., 2024, Šakota et al., 2024]. We similarly learn a router, but over numerical and neural solvers for PDEs at the iteration level. Our work is the first to apply routing to the problem of learning hybrid PDE solvers, in contrast to prior approaches that use fixed schedules. Additionally, by exploiting the algebraic structure of PDE solvers, we derive theoretical guarantees for our routing strategy (Section 4), unlike typical model-routing settings.

Mixture of Experts: Classical mixture-of-experts (MoE) [Jacobs et al., 1991, Jordan and Jacobs, 1994] uses a gating network to combine the outputs of multiple experts, trained jointly with the gate. In modern LLMs, MoE instead performs sparse, token-level routing to a subset of in-layer experts, enabling large capacity without proportional compute, typically with load-balancing mechanisms [Shazeer et al., 2017, Lepikhin et al., 2020, Fedus et al., 2022, Jiang et al., 2024]. Our method can be regarded as gating of a different kind: “experts” (solvers) are not jointly trained with the router, and a single router is reused across iterations, unlike MoE which commonly employs layer-specific gates.

7 Experiments

We evaluate the approximate greedy router on three PDEs posed on the unit domain $D = [0, 1]^2$ with periodic boundary conditions:

$$\text{Poisson: } -\Delta u = f, \quad \text{ConvDiff: } -\Delta u + 20 \partial_{x_1} u + 20 \partial_{x_2} u = f, \quad \text{AnisoDiff: } -\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f, \quad \epsilon = 10^{-2},$$

where f is centred to satisfy the compatibility condition $\int_D f(\mathbf{x}) d\mathbf{x} = 0$ and is drawn from a zero-mean hierarchical Gaussian random field with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$ and sample-dependent (α, β, γ) (Section C). Anisotropic diffusion is the standard problem on which point relaxation fails to smooth: an error mode oscillatory in x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep. The domain is discretised on 128^2 , 256^2 and 512^2 grids; results with Dirichlet boundary conditions, obtained with an earlier version of the pipeline, are deferred to Section D.1. Because iteration counts favour methods whose iterations are expensive, our primary metric is the end-to-end *wall-clock time to a target relative error* $\varepsilon = \|e_h^{(t)}\|/\|u_h\|$ on 64, 32 and 16 test instances per grid, with $\varepsilon = h^2$ —a grid-dependent tolerance that lies below the measured discretisation error of every test instance, so that further algebraic refinement does not improve the computed solution (Section E.1)—highlighted; every comparison is paired over instances and tested with a one-sided Wilcoxon signed-rank test. We also report the iteration-based metrics of the original submission, the final relative error and the area under the error curve, $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\|/\|u_h\|$ over $T = 300$ iterations. Section E gives the full protocol and collects every result per equation.

Setup. The classical solvers are Jacobi, damped Jacobi ($\omega = 0.67$), Gauss–Seidel (GS), symmetric GS (SymGS), SOR ($\omega = 1.5$) and geometric multigrid (V(2,2)-cycles with GS smoothing), all implemented as $O(N^2)$ stencil or cached sparse-triangular sweeps that match the dense implementations to machine precision, so the classical baselines are not artificially slowed. The neural operator is

a DeepONet corrector that acts on the residual through a 64×64 sensor grid with a Fourier trunk basis and is applied scale-equivariantly: its deployed form is a linear, Fourier-structured map (a least-squares fit of the band-limited inverse of the discrete operator, which it reproduces to $\approx 10^{-6}$), so one call solves the smooth band of the spectrum at the cost of about 12 Jacobi or 3 GS sweeps at 128^2 (and only 2.5 Jacobi sweeps at 512^2 , since its cost is nearly grid-independent) and never injects error outside its band. The router is made *cost-aware* through the cost-equalised macro-actions of Section 4 and selects from $6+K$ scalar features (log residual, its recent decay rate, the previous action and call counts) with a two-layer MLP whose decision costs $\approx 5 \mu\text{s}$ independently of N ; it is trained with the surrogate of Section 5.1 to imitate the cost-aware greedy oracle on oracle rollouts followed by DAgger rounds. We compare against the classical solver alone, HINTS with the published period $\tau=25$ and with the best period $\tau \in \{5, 10, 25, 50\}$ chosen *a posteriori* on the test set for each cell, the cost-aware greedy oracle (Algorithm 1 with the true error; not deployable), and the classical methods a practitioner would use on these problems without a corrector, multigrid alone and multigrid-preconditioned CG or BiCGSTAB.

Paired solvers. Table 1 reports, for every pairing $\{\text{NO}, \text{solver}\}$, equation and grid, the median wall-clock time to $\varepsilon = h^2$ of HINTS, of HINTS with the best period, and of the learned router. On the 128^2 grid the router is $238\times$ – $1983\times$ faster than the classical solver alone on the two isotropic equations (on the anisotropic equation no point smoother reaches h^2 at all within 60,000 iterations), $2.68\times$ – $11.8\times$ faster than HINTS ($\tau=25$) and $1.21\times$ – $3.22\times$ faster than the best period in every one of the 15 solver–equation pairings; on 256^2 it is $1.55\times$ – $10.9\times$ and $1.11\times$ – $2.33\times$ faster in all 14 pairings; and on 512^2 still $1.05\times$ – $3.10\times$ and $1.08\times$ – $1.24\times$ faster in all 10 stationary pairings, with the $\{\text{NO}, \text{multigrid}\}$ router the fastest iterative method we measured at that size (all $p < 0.01$; the margins shrink at 512^2 for the Jacobi and GS pairings because the corrector’s fixed band leaves those smoothers a slowly damped band that every schedule has to sweep through). Against the classical methods without corrector, the router with its best stationary pairing is $3.62\times$ – $3.71\times$ faster than geometric multigrid alone and $3.85\times$ – $5.31\times$ faster than the best multigrid-preconditioned Krylov method at 128^2 , and routing over $\{\text{NO}, \text{multigrid}\}$ is itself $1.93\times$ – $2.03\times$ faster than multigrid alone. The router tracks the cost-aware oracle within $1.06\times$ in median time and its decisions are identical across five independent training seeds. Figure 1 shows where the gain comes from: the router calls the corrector once at the very start, where the error is smooth and one call removes four orders of magnitude, spends a solver-dependent number of sweeps on the high-frequency band, and calls it again only if the smooth error re-emerges—between one and three calls in total—while HINTS spends $\tau-1$ sweeps before its first call and keeps calling the corrector when it no longer helps. The iteration-based metrics tell the same story: at 128^2 the router’s AUC over $T = 300$ iterations is orders of magnitude below HINTS’ and the solver-only baseline’s for every pairing and equation (paired t -tests, $p < 10^{-3}$) and its final error tracks the oracle’s (Tables 9, 15 and 21).

Solver ensembles. The same construction routes over ensembles $\text{NO} \cup \mathcal{W}$ of several classical solvers. Since the greedy rule is a per-decision comparison, a larger ensemble can only help where the error passes through regimes that favour *different* members; an exhaustive oracle-level screening of every subset of up to three members (Section E.5) shows that this never happens at $\varepsilon = h^2$ —one corrector call and one macro-action of the cheapest smoother already reach truncation level—but does at tolerances far below h^2 , where the smoother must damp the whole band outside the corrector’s reach: undamped Jacobi is cheapest on the modes near the band edge but cannot damp the near-Nyquist modes, which damped Jacobi removes. We therefore evaluate the learned router over the nested sets $\{\text{Jacobi}\}$, $\{\text{Jacobi} (0.67)\}$, $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ and $\{\text{Jacobi}, \text{Jacobi} (0.67), \text{GS}\}$ on every equation at 128^2 and 256^2 , with the single-member routers evaluated in the same session on the same instances. Two properties emerge. At $\varepsilon = 10^{-8}$ the ensemble router is strictly faster than every single-member router in 7 of the 12 ensemble cells (six settings, two nested sets) ($1.08\times$ – $1.42\times$ over the best single solver, $p < 0.01$, largest on the 256^2 grids), reproducing its oracle: it learns a *nonstationary* damping schedule—undamped Jacobi while the band-edge modes dominate, damped Jacobi once the near-Nyquist modes do—that no single member can reproduce. And enlarging the set is nearly free: adding GS changes nothing, because the oracle ignores a member that never wins a cost-aware comparison and the router imitates it, and at h^2 every ensemble router is within a few percent of the best single-member router; the one exception (Poisson at 128^2 , 10^{-8} , where the ensemble router is 8% slower) is inherited from the greedy oracle itself, which is not monotone in its action set (Section E.5). Finally, we verify numerically that the assumptions of Sections 4 and 5 hold in all these settings, in the energy norm for the non-normal sweeps (Section E.8).

Table 1: Median wall-clock time to truncation-level relative error $\varepsilon = h^2$ for every solver pairing $\{\text{NO, solver}\}$, equation and grid ($64, 32$ and 16 test instances at $128^2, 256^2$ and 512^2): HINTS with the published period $\tau=25$, HINTS with the best period chosen per cell on the test set itself (subscript), and the learned router (paired median speedup over HINTS-25 / over the best period in parentheses; bold: faster than both). The classical solver alone is $10^2\text{--}10^3\times$ slower than the router on the isotropic equations and never reaches h^2 on the anisotropic one (Section E). The last rows of each block give geometric multigrid alone and the best multigrid-preconditioned Krylov method, without corrector, on the same instances. Anisotropic diffusion is not run at 512^2 : its corrector keeps full resolution along x_1 , and the resulting $65,536 \times 65,536$ branch matrix does not fit in memory (Section E); multigrid is not run on it either (Section E.1). $\dagger n$: n instances did not reach h^2 within the iteration cap.

Equation	Pairing	128×128			256×256			512×512		
		HINTS-25	best τ	router (ours)	HINTS-25	best τ	router (ours)	HINTS-25	best τ	router (ours)
Poisson	Jacobi	2.65 ms	1.10 ms _{$\tau=5$}	987 μs ($2.68\times/1.21\times$)	5.95 ms	2.47 ms _{$\tau=5$}	2.38 ms ($2.52\times/1.56\times$)	87.0 ms	87.0 ms _{$\tau=25$}	78.8 ms ^{†2} ($1.11\times/1.11\times$)
	Jacobi (0.67)	2.71 ms	1.13 ms _{$\tau=5$}	912 μs ($2.90\times/1.28\times$)	6.41 ms	3.43 ms _{$\tau=5$}	2.83 ms ($2.26\times/1.73\times$)	130.8 ms	126.8 ms _{$\tau=25$}	115.2 ms ($1.16\times/1.10\times$)
	GS	6.65 ms	1.99 ms _{$\tau=5$}	1.01 ms ($6.75\times/1.94\times$)	22.0 ms	6.07 ms _{$\tau=5$}	3.92 ms ($5.77\times/1.70\times$)	142.8 ms	142.8 ms _{$\tau=25$}	131.5 ms ($1.11\times/1.11\times$)
	SymGS	10.3 ms	2.60 ms _{$\tau=5$}	1.13 ms ($9.21\times/2.34\times$)	37.3 ms	8.71 ms _{$\tau=5$}	4.22 ms ($9.96\times/2.27\times$)	216.0 ms	128.3 ms _{$\tau=5$}	115.1 ms ($1.83\times/1.10\times$)
	SOR (1.5)	12.2 ms	3.37 ms _{$\tau=5$}	1.02 ms ($10.3\times/3.22\times$)	40.7 ms	9.06 ms _{$\tau=5$}	3.80 ms ($10.9\times/2.33\times$)	182.7 ms	74.4 ms _{$\tau=5$}	60.7 ms ($3.10\times/1.24\times$)
	Multigrid	3.83 ms	3.81 ms _{$\tau=5$}	2.05 ms ($1.96\times/1.88\times$)	14.7 ms	14.6 ms _{$\tau=5$}	5.97 ms ($2.46\times/2.39\times$)	92.0 ms	90.5 ms _{$\tau=10$}	38.7 ms ($2.08\times/2.10\times$)
	Multigrid alone (no corrector)			3.99 ms			14.1 ms			81.9 ms
ConvDiff	Jacobi	3.44 ms	1.28 ms _{$\tau=5$}	1.14 ms ($2.95\times/1.34\times$)	9.37 ms	4.95 ms _{$\tau=10$}	4.76 ms ($1.92\times/1.30\times$)	158.0 ms	147.4 ms _{$\tau=50$}	144.3 ms ($1.17\times/1.14\times$)
	Jacobi (0.67)	3.40 ms	1.31 ms _{$\tau=5$}	1.10 ms ($3.14\times/1.32\times$)	9.31 ms	5.68 ms _{$\tau=10$}	5.76 ms ($1.80\times/1.25\times$)	203.4 ms	203.4 ms _{$\tau=50$}	222.2 ms ($1.08\times/1.08\times$)
	GS	8.27 ms	3.60 ms _{$\tau=5$}	1.35 ms ($6.76\times/2.60\times$)	31.0 ms	10.5 ms _{$\tau=5$}	7.05 ms ($5.52\times/1.53\times$)	214.1 ms	192.7 ms _{$\tau=10$}	179.9 ms ($1.05\times/1.09\times$)
	SymGS	11.1 ms	2.96 ms _{$\tau=5$}	1.28 ms ($8.70\times/2.40\times$)	42.3 ms	10.9 ms _{$\tau=5$}	6.96 ms ($6.26\times/1.69\times$)	282.8 ms	187.9 ms _{$\tau=5$}	172.6 ms ($1.55\times/1.15\times$)
	SOR (1.5)	13.6 ms	3.64 ms _{$\tau=5$}	1.59 ms ($8.89\times/2.40\times$)	46.1 ms	10.6 ms _{$\tau=5$}	5.80 ms ($7.99\times/1.96\times$)	206.6 ms	89.3 ms _{$\tau=5$}	83.6 ms ($2.48\times/1.18\times$)
	Multigrid	4.68 ms	4.66 ms _{$\tau=5$}	2.32 ms ($2.07\times/2.01\times$)	16.7 ms	16.5 ms _{$\tau=5$}	7.26 ms ($2.19\times/2.20\times$)	97.6 ms	92.5 ms _{$\tau=10$}	53.7 ms ($1.83\times/1.75\times$)
	Multigrid alone (no corrector)			4.52 ms			18.8 ms			99.7 ms
AnisoDiff	Jacobi	2.90 ms	1.15 ms _{$\tau=5$}	911 μs ($3.16\times/1.25\times$)	14.9 ms	10.3 ms _{$\tau=5$}	9.55 ms ($1.58\times/1.11\times$)	—	—	—
	Jacobi (0.67)	2.85 ms	1.18 ms _{$\tau=5$}	850 μs ($3.31\times/1.34\times$)	14.9 ms	10.2 ms _{$\tau=5$}	9.38 ms ($1.55\times/1.11\times$)	—	—	—
	GS	6.48 ms	1.74 ms _{$\tau=5$}	852 μs ($7.61\times/1.99\times$)	29.7 ms	12.8 ms _{$\tau=5$}	9.30 ms ($3.18\times/1.37\times$)	—	—	—
	SymGS	10.3 ms	2.40 ms _{$\tau=5$}	871 μs ($11.8\times/2.72\times$)	—	—	—	—	—	—
	SOR (1.5)	6.62 ms	1.80 ms _{$\tau=5$}	869 μs ($7.36\times/2.04\times$)	29.7 ms	12.8 ms _{$\tau=5$}	9.93 ms ($2.98\times/1.38\times$)	—	—	—

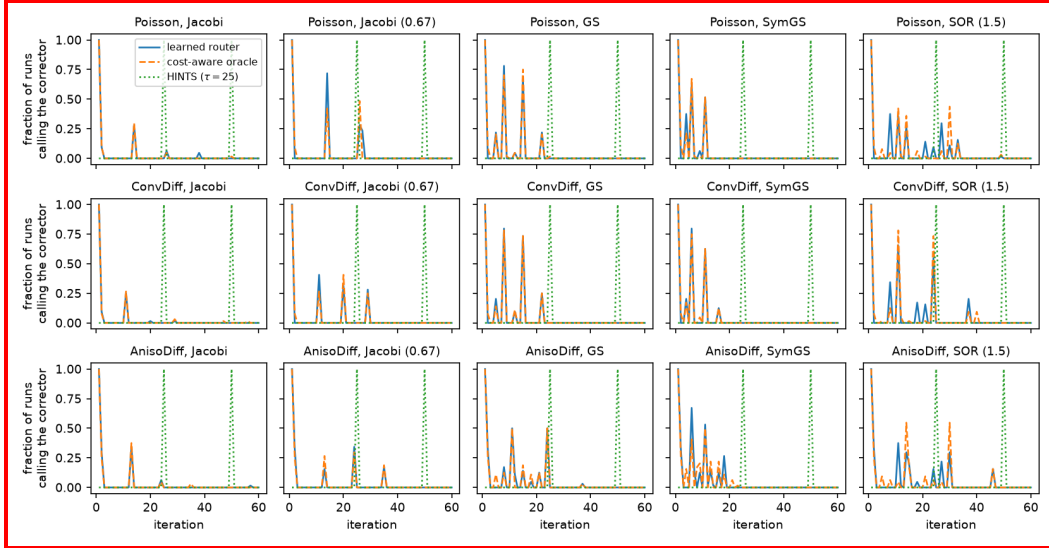


Figure 1: Corrector usage on the 128×128 grid: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=25$), for every solver pairing (columns) and all three equations (rows); the same figure for the 256^2 and 512^2 grids is in Section E.

Every result per equation and grid, the ensemble study, the accounting of overheads and training cost, and the verification of the theoretical assumptions are collected in Section E; Section D reports the Dirichlet experiment of the original submission.

8 Discussion

We have introduced an adaptive method for selecting solvers that efficiently minimizes the final error of a PDE in iterative methods, opening many directions for future work. Our current DeepONet is

trained without accounting for its downstream role in correction term prediction; jointly training the ML solver and router could yield larger gains. Another avenue is to use reinforcement learning to learn a cost-aware routing strategy that optimizes error under compute budgets; **Sections 4 and E show that a myopic cost-aware variant of the greedy rule—Algorithm 1 on cost-equalised macro-actions—already delivers uniform wall-clock gains, and RL could extend it to non-myopic budgeted routing.** RL would additionally enable extending the ensemble routing to *continuous* parameterization of solvers. Finally, framing routing as an online learning problem could enable adaptation to unknown deployment conditions.

References

- Eugenia Kalnay. *Atmospheric modeling, data assimilation and predictability*. Cambridge university press, 2003.
- Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.
- Andrew Staniforth and Jean Côté. Semi-lagrangian integration schemes for atmospheric models—a review. *Monthly weather review*, 119(9):2206–2223, 1991.
- John David Anderson. *Hypersonic and high temperature gas dynamics*. Aiaa, 1989.
- Fredi Tröltzsch. *Optimal control of partial differential equations: theory, methods, and applications*, volume 112. American Mathematical Soc., 2010.
- Sylvain Baillet, John C Mosher, and Richard M Leahy. Electromagnetic brain mapping. *IEEE Signal processing magazine*, 18(6):14–30, 2001.
- Roberta Grech, Tracey Cassar, Joseph Muscat, Kenneth P Camilleri, Simon G Fabri, Michalis Zervakis, Petros Xanthopoulos, Vangelis Sakkalis, and Bart Vanrumste. Review on solving the inverse problem in eeg source analysis. *Journal of neuroengineering and rehabilitation*, 5:1–33, 2008.
- Gordon D Smith. *Numerical solution of partial differential equations: finite difference methods*. Oxford university press, 1985.
- Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems*. SIAM, 2007.
- Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*. Courier Corporation, 2003.
- Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.
- Susanne C Brenner. *The mathematical theory of finite element methods*. Springer, 2008.
- Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.
- Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4):911–917, 1995.
- Lizuo Liu and Wei Cai. Multiscale deepnet for nonlinear operators in oscillatory function spaces for building seismic wave responses. *arXiv preprint arXiv:2111.04860*, 2021.
- Xinliang Liu, Bo Xu, Shuhao Cao, and Lei Zhang. Mitigating spectral bias for the multiscale operator learning. *Journal of Computational Physics*, 506:112944, 2024.
- Zhilin You, Zhenli Xu, and Wei Cai. Mscalefno: Multi-scale fourier neural operator learning for oscillatory function spaces. *arXiv preprint arXiv:2412.20183*, 2024.

458 Siavash Khodakarami, Vivek Oommen, Aniruddha Bora, and George Em Karniadakis. Mitigating
459 spectral bias in neural operators via high-frequency scaling for physical systems. *arXiv preprint*
460 *arXiv:2503.13695*, 2025.

461 Zhi-Qin John Xu, Lulu Zhang, and Wei Cai. On understanding and overcoming spectral biases of
462 deep neural network learning methods for solving pdes. *arXiv preprint arXiv:2501.09987*, 2025.

463 Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua
464 Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference*
465 *on machine learning*, pages 5301–5310. PMLR, 2019.

466 Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle:
467 Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.

468 Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for
469 general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019.

470 Enrui Zhang, Adar Kahana, Alena Kopaničáková, Eli Turkel, Rishikesh Ranade, Jay Pathak, and
471 George Em Karniadakis. Blending neural operators and relaxation methods in pde numerical
472 solvers. *Nature Machine Intelligence*, pages 1–11, 2024.

473 John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.

474 Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods:*
475 *Fundamentals in Single Domains*. Springer, 2006. ISBN 9783540307264.

476 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew
477 Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations.
478 *arXiv preprint arXiv:2010.08895*, 2020a.

479 Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik
480 Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial
481 differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020b.

482 Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning
483 nonlinear operators via deepnet based on the universal approximation theorem of operators.
484 *Nature machine intelligence*, 3(3):218–229, 2021.

485 George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for
486 maximizing submodular set functions—i. *Mathematical programming*, 14(1):265–294, 1978.

487 Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions.
488 In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques*
489 *(APPROX/RANDOM 2017)*, pages 19–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

490 Louenas Bounia and Frederic Koriche. Approximating probabilistic explanations via supermodular
491 minimization. In *Uncertainty in Artificial Intelligence*, pages 216–225. PMLR, 2023.

492 Abhimanyu Das and David Kempe. Approximate submodularity and its applications: Subset selection,
493 sparse approximation and dictionary selection. *Journal of Machine Learning Research*, 19(3):
494 1–34, 2018.

495 Uriel Feige, Vahab S Mirrokni, and Jan Vondrák. Maximizing non-monotone submodular functions.
496 *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

497 Andrew An Bian, Joachim M Buhmann, Andreas Krause, and Sebastian Tschachtschek. Guarantees for
498 greedy maximization of non-submodular functions with applications. In *International conference*
499 *on machine learning*, pages 498–507. PMLR, 2017.

500 Chris Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. Submodular maximization beyond
501 non-negativity: Guarantees, fast algorithms, and applications. In *International Conference on*
502 *Machine Learning*, pages 2634–2643. PMLR, 2019.

503 Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions.
504 *Advances in Neural Information Processing Systems*, 21, 2008.

505 Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. Maximizing sequence-submodular functions
506 and its application to online advertising. *Management Science*, 67(10):6030–6054, 2021.

507 Zhenliang Zhang, Edwin KP Chong, Ali Pezeshki, and William Moran. String submodular functions
508 with curvature constraints. *IEEE Transactions on Automatic Control*, 61(3):601–616, 2015.

509 Sara Bernardini, Fabio Fagnani, and Chiara Piacentini. Through the lens of sequence submodularity.
510 In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30,
511 pages 38–47, 2020.

512 Brandon Van Over, Bowen Li, Edwin KP Chong, and Ali Pezeshki. A performance bound for
513 the greedy algorithm in a generalized class of string optimization problems. *arXiv preprint*
514 *arXiv:2409.05020*, 2024.

515 Sebastian Tschischek, Adish Singla, and Andreas Krause. Selecting sequences of items via
516 submodular maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
517 volume 31, 2017.

518 Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J Cree. Regularisation of neural networks
519 by enforcing lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021.

520 Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent
521 neural networks. *Neural computation*, 1(2):270–280, 1989.

522 Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C
523 Courville, and Yoshua Bengio. Professor forcing: A new algorithm for training recurrent networks.
524 *Advances in neural information processing systems*, 29, 2016.

525 Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence
526 prediction with recurrent neural networks. *Advances in neural information processing systems*, 28,
527 2015.

528 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Regression with multi-expert deferral. In *International*
529 *Conference on Machine Learning*, pages 34738–34759. PMLR, 2024.

530 Ali Taghibakhshi, Scott MacLachlan, Luke Olson, and Matthew West. Optimization-based algebraic
531 multigrid coarsening using reinforcement learning. *Advances in neural information processing*
532 *systems*, 34:12129–12140, 2021.

533 Matteo Caldana, Paola F Antonietti, et al. A deep learning algorithm to accelerate algebraic multigrid
534 methods in finite element solvers of 3d elliptic pdes. *Computers & Mathematics with Applications*,
535 167:217–231, 2024.

536 Alena Kopaničáková and George Em Karniadakis. Deepnet based preconditioning strategies for
537 solving parametric linear systems of equations. *SIAM Journal on Scientific Computing*, 47(1):
538 C151–C181, 2025.

539 Ru Huang, Ruipeng Li, and Yuanzhe Xi. Learning optimal multigrid smoothers via neural networks.
540 *SIAM Journal on Scientific Computing*, 45(3):S199–S225, 2022.

541 Alexandr Katrutsa, Talgat Daulbaev, and Ivan Oseledets. Black-box learning of multigrid parameters.
542 *Journal of Computational and Applied Mathematics*, 368:112524, 2020.

543 Adar Kahana, Enrui Zhang, Somdatta Goswami, George Karniadakis, Rishikesh Ranade, and Jay
544 Pathak. On the geometry transferability of the hybrid iterative numerical solver for differential
545 equations. *Computational Mechanics*, 72(3):471–484, 2023.

546 Pengzhan Jin, Shuai Meng, and Lu Lu. Mionet: Learning multiple-input operators via tensor product.
547 *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.

548 Jun Hu and Pengzhan Jin. A hybrid iterative method based on mionet for pdes: Theory and numerical
549 examples. *Mathematics of Computation*, 2025.

550 Chen Cui, Kai Jiang, Yun Liu, and Shi Shu. Fourier neural solver for large sparse linear algebraic
551 systems. *Mathematics*, 10(21):4014, 2022.

552 Tal Shnitzer, Anthony Ou, Mirian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson,
553 and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint*
554 *arXiv:2309.15789*, 2023.

555 Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt
556 Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing
557 system. *arXiv preprint arXiv:2403.12031*, 2024.

558 Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS
559 Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query
560 routing. *arXiv preprint arXiv:2404.14618*, 2024.

561 Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and
562 Liang Lin. Routereval: A comprehensive benchmark for routing llms to explore model-level
563 scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.

564 Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki,
565 Xia Hu, and Helen Zhou. Learning to route llms with confidence tokens. *arXiv preprint*
566 *arXiv:2410.13284*, 2024.

567 Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanning Cai, Yang Sui,
568 Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based
569 on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*,
570 2025.

571 Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to
572 large language models. *arXiv preprint arXiv:2308.11601*, 2023.

573 Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to
574 large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.

575 Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language
576 model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on*
577 *Web Search and Data Mining*, pages 606–615, 2024.

578 Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of
579 local experts. *Neural computation*, 3(1):79–87, 1991.

580 Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural*
581 *computation*, 6(2):181–214, 1994.

582 Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and
583 Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv*
584 *preprint arXiv:1701.06538*, 2017.

585 Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang,
586 Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional
587 computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

588 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter
589 models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39,
590 2022.

591 Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris
592 Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al.
593 Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

594 Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and
595 applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.

596 Michael C Mozer. A focused backpropagation algorithm for temporal pattern recognition. In
597 *Backpropagation*, pages 137–169. Psychology Press, 2013.

598 Anthony J Robinson and Frank Fallside. *The utility driven dynamic error propagation network*,
 599 volume 11. University of Cambridge Department of Engineering Cambridge, 1987.

600 Paul J Werbos. Generalization of backpropagation with application to a recurrent gas market model.
 601 *Neural networks*, 1(4):339–356, 1988.

602 Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and
 603 George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with prac-
 604 tical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*,
 605 393:114778, 2022.

606 Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured
 607 prediction to no-regret online learning. In *Proceedings of the Fourteenth International Confer-*
 608 *ence on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference
 609 Proceedings, 2011.

610 A Proofs for Section 4

611 A.1 Proof of Proposition A.1

612 **Proposition A.1.** *Any prefix monotonically non-increasing sequence supermodular function g is*
 613 *weakly supermodular with respect to all sequences $S \in \Omega^*$ with $\alpha(S) = 1$*

614 *Proof.* This proof is adapted from Liberty and Sviridenko [2017].

615 If g is sequence supermodular then,

$$\begin{aligned} & g(S) - g(S \oplus S') \\ &= \sum_{i=1}^{|S'|} g(S \oplus (S'_1, \dots, S'_{i-1})) - g(S \oplus (S'_1, \dots, S'_i)) \\ &\stackrel{(a)}{\leq} \sum_{i=1}^{|S'|} g(S) - g(S \oplus S'_i) \\ &\leq |S'| \max_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \end{aligned}$$

616 (a) by supermodularity

617 □

618 A.2 Proof of Theorem 4.1

619 **Theorem 4.1.** *Let $g : \Omega^* \rightarrow \mathbb{R}$ be a weakly supermodular function with respect to the optimal*
 620 *solution $O = \arg \min_{S \in \Omega^T} g(S)$ with a supermodularity ratio of $\alpha(O)$ and postfix monotonicity. Let*
 621 *the greedy solution of length T be S^T . If $\phi_T(\alpha) = \left(1 - \frac{1}{\alpha T}\right)^T$, then*

$$g(S^T) \leq (1 - \phi_T(\alpha(O))) g(O) + \phi_T(\alpha(O)) g(\emptyset)$$

622 *Proof.* This proof strategy is inspired by Streeter and Golovin [2008] and Liberty and Sviridenko
 623 [2017]

$$\begin{aligned}
g(S^t) - g(O) &\stackrel{(a)}{\leq} g(S^t) - g(S^t \oplus O) \\
&= \sum_{i=1}^{|O|} g(S^t \oplus (o_1, \dots, o_{i-1})) - g(S^t \oplus (o_1, \dots, o_i)) \\
&\stackrel{(b)}{\leq} \alpha(O) \sum_{i=1}^{|O|} g(S^t) - g(S^t \oplus o_i) \\
&\leq \alpha(O)|O| \max_{i \in [|O|]} g(S^t) - g(S^t \oplus o_i) \\
&\leq \alpha(O)Tg(S^t) - \alpha(O)T \min_{\omega \in \Omega} g(S^t \oplus \omega) \\
&= \alpha(O)Tg(S^t) - \alpha(O)Tg(S^{t+1})
\end{aligned}$$

624 (a) by postfix monotonicity, (b) by weak supermodularity with respect to O (applied to the prefix S^t).

625 After rearranging the inequality, we get:

$$\begin{aligned}
g(S^{t+1}) &\leq \frac{1}{\alpha(O)T} (g(O) - (\alpha(O)T - 1)g(S^t)) \\
&= \frac{1}{\alpha(O)T}g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)g(S^t)
\end{aligned}$$

626 When recursively applying this inequality, we get:

$$\begin{aligned}
g(S^T) &\leq \frac{1}{\alpha(O)T}g(O) \sum_{i=0}^{T-1} \left(1 - \frac{1}{\alpha(O)T}\right)^i + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \\
&= \left(1 - \left(1 - \frac{1}{\alpha(O)T}\right)^T\right)g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset)
\end{aligned}$$

627

□

628 A.3 Proof of Proposition 4.2

629 **Proposition 4.2.** Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is
630 ρ_j -Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is
631 weakly supermodular with respect to the optimal solution O , with

$$\alpha(O) = \max \left\{ \frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\}$$

632 *Furthermore, if the error propagation maps are linear and simultaneously diagonalizable by an*
633 *orthogonal matrix with eigenvalues of modulus at most one (the setting of Theorem 4.3), h is also*
634 *postfix non-increasing; invertibility alone does not suffice (Section A.3).*

635 *Proof.* For brevity, we use the notation $(g_1 \circ \dots \circ g_T)(x) = \circ_{t=1}^T g_t(x)$, where composition is
636 applied from right to left so that g_T acts first. In this proof, we use a few properties of Lipschitz
637 continuous functions:

- 638 • **Property 1:** If g is ρ -Lipschitz continuous and $g(0) = 0$, then $\|g(x)\|_2 = \|g(x) - 0\|_2 =$
639 $\|g(x) - g(0)\|_2 \leq \rho\|x - 0\|_2 = \rho\|x\|_2$
- 640 • **Property 2:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
641 and ρ_2 respectively, the Lipschitz constant of $g_1 + g_2$ and $g_1 - g_2$ is $\rho_1 + \rho_2$.

642 • **Property 3:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1
 643 and ρ_2 respectively, the Lipschitz constant of $g_1 \circ g_2$ is $\rho_1 \rho_2$.

644 In order to prove weakly- α -supermodularity, we must first prove prefix monotonicity.

645 **Prefix monotonicity:** Let $S \preceq S'$ where $S' = S \oplus N$.

$$\begin{aligned} h(S') &= \left\| \circ_{t=|S'|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &= \left\| \circ_{t=|S'|}^{|S|+1} (I - C_{S'_t} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\stackrel{(a)}{\leq} \left(\prod_{t=|S|+1}^{|S'|} \rho_{S'_t}^2 \right) \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\leq h(S) \end{aligned}$$

646 (a) by Property 1

647 **Weak supermodularity:** We will upper bound $\alpha(O)$ by providing an upper bound for $h(S) -$
 648 $h(S \oplus O)$ and a lower bound for $\sum_{i=1}^T h(S) - h(S \oplus O_i)$.

$$\begin{aligned} h(S) - h(S \oplus O) &= \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \left\| \circ_{t=|O|}^1 (I - C_{O_t} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\stackrel{(a)}{\leq} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) - \circ_{t=|O|}^1 (I - C_{O_t} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &= \left\| \left(I - \circ_{t=|O|}^1 (I - C_{O_t} \circ \mathcal{L}_h^a) \right) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\stackrel{(b)}{\leq} \left(1 + \prod_{t=1}^{|O|} \rho_{O_t} \right)^2 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\stackrel{(c)}{\leq} 4 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \end{aligned}$$

649 (a) by reverse triangle property and $h(S) - h(S \oplus S') > 0$ by prefix monotonicity, (b) since the
 650 Lipschitz constant of $I - \circ_{t=|S'|}^1 (I - C_{S'_t} \circ \mathcal{L}_h^a)$ is $1 + \prod_{t=1}^{|S'|} \rho_{S'_t}$ by Property 2 and 3, (c) since
 651 $\rho_j < 1$

652 To lower bound $\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i)$

$$\begin{aligned} \sum_{i=1}^{|O|} h(S) - h(S \oplus O_i) &= \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \left\| (I - C_{O_i} \circ \mathcal{L}_h^a) \circ \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &\geq \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 - \rho_{O_i}^2 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \\ &= \left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right) \end{aligned}$$

653 Finally,

$$\begin{aligned} \frac{h(S) - h(S \oplus O)}{\sum_{i=1}^T (h(S) - h(S \oplus O_i))} &\leq \max \left\{ \frac{4 \left\| \circ_{t=|S|}^1 (I - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2}{\left\| \circ_{t=|S|}^1 (I_N - C_{S_t} \circ \mathcal{L}_h^a) \left(e_h^{(0)} \right) \right\|_2^2 \left(T - \sum_{i=1}^T \rho_{O_i}^2 \right)}, 1 \right\} \\ &= \max \left\{ \frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1 \right\} \end{aligned}$$

654 **Postfix monotonicity.** Let $S' = S \oplus N$ and suppose that $I - C_j \mathcal{L}_h^a = P \Lambda_j P^\top$ for all j with a
 655 common orthogonal P and $|\lambda_{ji}| \leq 1$. By Theorem A.2,

$$h(S') = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \leq \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(N)} = h(N),$$

656 since every factor $\lambda_{ji}^{2m_j(S)}$ lies in $[0, 1]$. The shared eigenbasis cannot be replaced by invertibility of
 657 the individual maps: for

$$A = \begin{bmatrix} 0.0674240 & -0.1864331 \\ -0.1473150 & -0.8707252 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7575738 & 0.4816284 \\ -0.1369844 & 0.3257283 \end{bmatrix}, \quad x = \begin{bmatrix} 0.2812107 \\ -0.5538228 \end{bmatrix},$$

658 both matrices are invertible with spectral norm 0.9, yet $\|BAx\|_2 = 0.330 > 0.225 = \|Bx\|_2$, i.e.,
 659 $h((A) \oplus (B)) > h((B))$. (An earlier version of this proof bounded $\|G^{-1}\|$ by the reciprocal of the
 660 Lipschitz constant of G , which is not valid.)

661

□

662 A.4 Proof of Theorem A.2

663 **Lemma A.2.** Let $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ where P is an orthogonal matrix and $\Lambda_j =$
 664 $\text{diag}(\lambda_{j1}, \dots, \lambda_{jN})$. If $P^{-1}e_h^{(0)} = z$, then the following equality holds:

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \quad (11)$$

665 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$

Proof.

$$\begin{aligned} h(S) &= \left\| \prod_{t=|S|}^1 (I_N - C_{S_t} \mathcal{L}_h^a) e_h^{(0)} \right\|^2 \\ &= \left\| \prod_{t=|S|}^1 (P \Lambda_{S_t} P^{-1}) e_h^{(0)} \right\|^2 \\ &= \left\| P \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\ &= \left\| \prod_{t=|S|}^1 \Lambda_{S_t} P^{-1} e_h^{(0)} \right\|^2 \\ &= \sum_{i=1}^N z_i^2 \prod_{t=|S|}^1 \lambda_{S_t i}^2 \\ &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \end{aligned}$$

666

□

667 A.5 Proof of Theorem 4.3

668 **Proposition 4.3.** Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$ with a common
 669 orthogonal matrix P . Then, h is supermodular:

670 *Proof.* Let $S \preceq S'$ where $S' = S \oplus B$. By Theorem A.2,

$$h(S) = \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)}$$

671 where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$ be the number of times a sequence S calls the solver j . Recall that h is
 672 considered sequence supermodular if $\forall S', S \in \Omega^*$ such that $S \preceq S'$, it holds that

$$h(S) - h(S \oplus \omega) \geq h(S') - h(S' \oplus \omega)$$

673

$$\begin{aligned} h(S) - h(S \oplus \omega) &= \sum_{i=1}^N z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} - \sum_{i=1}^N z_i^2 \lambda_{\omega i}^2 \prod_{k=1}^K \lambda_{ki}^{2m_j(S)} \\ &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \end{aligned}$$

674 Similarly,

$$\begin{aligned} h(S') - h(S' \oplus \omega) &= \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S')} \\ &\stackrel{(a)}{=} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2(m_j(S) + m_j(B))} \\ &\stackrel{(b)}{\leq} \sum_{i=1}^N (1 - \lambda_{\omega i}^2) z_i^2 \prod_{j=1}^K \lambda_{ji}^{2m_j(S)} \\ &= h(S) - h(S \oplus \omega) \end{aligned}$$

675 (a) Since $m_j(S') = \sum_{t=1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S'_t=j} + \sum_{t=|S|+1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=k} + \sum_{t=1}^{|B|} \mathbf{1}_{B_t=j} =$
 676 $m_j(S) + m_j(B)$, (b) since $|\lambda_{ji}| \leq 1$ (as $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ and P is orthogonal) and $m_j(B) \geq 0$
 677 for all $j \in [K]$

678 □

679 B Proofs for Section 5

680 B.1 Proof of Theorem B.1

681 **Lemma B.1.** For any set of preconditioning functions $\mathcal{C}$, any discrete operator $\mathcal{L}_h^a$, any router r , any
 682 $a_h, f_h, u_h^{(t)}, u_h \in \mathcal{A} \times \mathcal{F} \times \mathcal{U} \times \mathcal{U}$, the following equality holds true:

$$\begin{aligned} l_{route}(r, a_h, f_h, u_h^{(t)}, u_h) &= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ &\quad - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) (u_h - u_h^{(t)}) \right\|_2^2 \end{aligned}$$

683 *Proof.* Note that $\sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} = K - 1$

$$\begin{aligned}
l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) = j} \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad + (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad + \sum_{j=1}^K \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\quad - (K - 1) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

684

$$\begin{aligned}
&= \sum_{j=1}^K \left(\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \right) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - (K - 2) \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}$$

685

□

686 **B.2 Proof of Theorem B.2**

687 **Proposition B.2.** For any router r defined by $r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$, any $a_h \in \mathcal{A}$,
688 $f_h \in \mathcal{F}$, and $u_h^{(t)}, u_h \in \mathcal{U}$, the routing loss l_{route} satisfies:

$$\log(2) l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) \leq \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)$$

689 *Proof.* By Theorem B.1, we know that

$$\begin{aligned}
\log(2)l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, u_h\right) &= \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - \log(2) (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\leq \log(2) \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\stackrel{(a)}{\leq} - \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \\
&= \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)
\end{aligned}$$

690 (a) if $r(a_h, f_h, u_h^{(t)}) \neq j$, $\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} < 0.5$ which implies that

$$691 -\log \left(\frac{\exp(g_j(a, f, u^{(t)}))}{\sum_{k=1}^K \exp(g_k(a, f, u^{(t)}))} \right) \geq \log(2) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}$$

692 $\square$

693 B.3 Proof of Theorem 5.1

694 **Theorem 5.1.** Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that
695 $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{\min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete
696 operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route} .

697 *Proof.* For a given a_h, f_h , let u_h be $\mathcal{G}_h(a_h, f_h)$ where $\mathcal{G}_h$ denotes the solution operator acting on the
698 grid G_h . Furthermore, let's consider routers of the form

$$r(a, f, u^{(t)}) = \operatorname{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$$

699 For a given $a_h, f_h, u_h^{(t)} \in \mathcal{A} \times \mathcal{F} \times \mathcal{U}$, let the optimal loss under l_{route} be
700 $l_{\text{route}}^*(a_h, f_h, u_h^{(t)}) = \inf_{\tilde{r}} l_{\text{route}}(\tilde{r}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h))$. Similarly, let the optimal loss un-
701 der Ψ be $\Psi^*(a_h, f_h, u_h^{(t)}) = \inf_{\tilde{\mathbf{g}}} \Psi(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h))$. Let $B_j(a_h, f_h, u_h^{(t)}) =$
702 $\sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(\mathcal{G}_h(a_h, f_h) - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j}$.

$$\begin{aligned}
&l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)\right) - l_{\text{route}}^*\left(a_h, f_h, u_h^{(t)}\right) \\
&\stackrel{(a)}{=} \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\quad - \inf_{\tilde{r}} \sum_{j=1}^K \sum_{k=1}^K \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} + (K-2) \sum_{j=1}^K \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)}) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)}) \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \\
&= \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right)
\end{aligned}$$

703 (a) by Theorem B.1

Let $\mathcal{X} = \mathcal{A} \times \mathcal{F} \times \mathcal{U}$ and $\mathcal{Y} = [K]$. Let $\mathcal{P}_{\mathcal{X}}$ denote the degenerate distribution supported at the point $(a_h, f_h, u_h^{(t)})$. We define the conditional distribution - $P(Y = j \mid X = (a_h, f_h, u_h^{(t)})) = \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}$ for $j \in [K]$. The risk and optimal risk of 0 - 1 loss under this distribution can be written as:

$$\begin{aligned}\mathcal{R}_{0-1}(r) &= \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\ \mathcal{R}_{0-1}^* &= \inf_r \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}\end{aligned}$$

If $r(a_h, f_h, u_h^{(t)}) = \arg \max_{j \in [K]} g_j(a_h, f_h, u_h^{(t)})$ for all $x \in \mathcal{X}$, then the he risk and optimal risk of cross entropy loss ($l_{ce}(\mathbf{g}, x, y) - \log \left(\frac{\exp(g_y(x))}{\sum_{k=1}^K \exp(g_k(x))} \right)$) under this distribution can be written as:

$$\begin{aligned}\mathcal{R}_{ce}(\mathbf{g}) &= - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \\ \mathcal{R}_{ce}^* &= \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right)\end{aligned}$$

From Theorem 3.1 of [Mao et al., 2023], $\mathcal{R}_{0-1}(r) - \mathcal{R}_{0-1}^* \leq \Gamma^{-1}(\mathcal{R}_{ce}(\mathbf{g}) - \mathcal{R}_{ce}^*)$ if $r(a_h, f_h, u_h^{(t)}) = \arg \max_{j \in [K]} g_j(a_h, f_h, u_h^{(t)})$ where $\Gamma(z) = \frac{1+z}{2} \log(1+z) + \frac{1-z}{2} \log(1-z)$. Then,

$$\begin{aligned}
& l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \\
&= \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right) \\
&\leq \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&\stackrel{(a)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&\stackrel{(b)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right. \\
&\quad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \log \left(\frac{\exp(g_j(a_h, f_h, u_h^{(t)}))}{\sum_{k=1}^K \exp(g_k(a_h, f_h, u_h^{(t)}))} \right) \right) \\
&= \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \right)
\end{aligned}$$

713 (a) since $\left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 < \bar{E}$ for all $j \in [K]$, (b) since Γ^{-1} is non-decreasing and

714 $\exists j \in [K]$ such that $\left\| (I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)}) \right\|_2^2 > E_{\min}$

715 Finally,

$$\begin{aligned}
& \lim_{n \rightarrow \infty} \mathcal{R}_{\text{route}}(r_n) - \mathcal{R}_{\text{route}}^* \\
&\stackrel{(a)}{=} \lim_{n \rightarrow \infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[l_{\text{route}} \left(r_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h) \right) - l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \right] \\
&\leq \lim_{n \rightarrow \infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[\bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)})}{(K-1)E_{\min}} \right) \right] \\
&\stackrel{(b)}{\leq} \lim_{n \rightarrow \infty} \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[\Psi(\mathbf{g}_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h(a_h, f_h)) - \Psi^*(a_h, f_h, u_h^{(t)}) \right]}{(K-1)E_{\min}} \right)
\end{aligned}$$

$$\begin{aligned}
&= \lim_{n \rightarrow \infty} \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\mathcal{R}_\Psi(\mathbf{g}_n) - \mathcal{R}_\Psi^*}{(K-1)E_{min}} \right) \\
&\stackrel{(c)}{=} \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\lim_{n \rightarrow \infty} \mathcal{R}_\Psi(\mathbf{g}_n) - \mathcal{R}_\Psi^*}{(K-1)E_{min}} \right) \\
&= \bar{E} K (K-1) \Gamma^{-1}(0) \\
&\stackrel{(d)}{=} 0
\end{aligned}$$

717 (a) $\mathcal{R}_{\text{route}}^* = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A} \times \mathcal{F}}} \left[l_{\text{route}}^* \left(a_h, f_h, u_h^{(t)} \right) \right]$ since the infimum is taken over all measurable
718 functions, (b) by Jensen's inequality since Γ^{-1} is concave, (c) by continuity of Γ^{-1} at 0, (d) $\Gamma^{-1}(0) =$
719 0

720

□

721 C Training Details

722 The periodic experiments of Section 7 use the corrector and the lightweight router described in
723 Section E.1; the data distribution below is shared by all experiments, and the DeepONet and LSTM
724 router described afterwards are those of the Dirichlet experiment in Section D.1.

725 Data for both DeepONet and the routers is sampled from a zero-mean Hierarchical Gaussian Random
726 Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ are sampled
727 as:

$$\begin{aligned}
\alpha &\sim \text{Log-Uniform}(0.01, 100) \\
\beta &\sim \text{Log-Uniform}(0.1, 1000) \\
\gamma &\sim \text{Uniform}(\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\})
\end{aligned}$$

728 Given (α, β, γ) , samples are generated in the Fourier space. For each non-zero frequency mode
729 k , we draw an independent complex coefficient from a Gaussian distribution with mean 0 and
730 variance $\alpha(4\pi^2\|k\|_2^2 + \beta)^{-\gamma}$. We enforce a Hermitian symmetry to obtain a real-valued field, set the
731 zero-frequency (DC) mode to 0 to obtain a zero mean field, and apply the inverse Discrete Fourier
732 Transform to obtain the field in physical space.

733 For each sample, we compute reference solutions with a least squares solver and treat them as ground
734 truth.

735 This data is used to trained our DeepONet models and LSTM routers. All the models were imple-
736 mented using PyTorch and all the models were trained on one Nvidia A40 GPU.

737 Table 2 contains all hyperparameter details for the DeepONet. DeepONet took 30 minutes to train.
738 We then use the model with the best validation loss.

Table 2: Hyperparameter settings for DeepONet

Hyperparameter	Value
Learning rate	1e-3
Branch Dimension	128
Hidden dimension for branch net	256
No. of hidden layers in branch net	4
Hidden dimension for trunk net	256
No. of hidden layers in trunk net	4
Gradient Clipping Norm	1.0
Weight Decay	0.005
Batch size	256
Training samples	10000
Validation samples	2000
Epochs	1000

739 The routers are LSTM models. Along with the inputs specified in Section 5, namely $a_h, f_h, u_h^{(t)}$,
740 we also supply the routers with the iteration index t , current residual $f_h - \mathcal{L}_h^a u_h^{(t)}$, and the routing
741 decisions from the previous iteration. While the iteration index and residual are deterministic
742 functions of $(a_h, f_h, u_h^{(t)})$ and align with the theory presented in Section 5, incorporating previous
743 routing decisions is a mild deviation from the idealized setting. Nevertheless, this modification is
744 motivated by the same considerations that justify the use of scheduled sampling: under deployment,
745 routing decisions are predicted autoregressively and influence future inputs, whereas training under
746 teacher forcing assumes independence from past predictions. Including previous routing decisions
747 bridges the gap induced by exposure bias by providing the model with trajectory-level information.

748 All the routers are trained with scheduled sampling. We use a warm-up of e_w epochs with teacher-
749 forcing probability $p_{tf}(e) = ss_{start}$. After the warm-up, the p_{tf} decays geometrically by a factor of
750 $\gamma_{tf} < 1$ per epoch and is floored by ss_{end} :

$$p_{tf}(e) = \begin{cases} ss_{start} & e \leq e_w \\ \max(ss_{start}\gamma_{tf}^{e-e_w}, ss_{end}) & e > e_w \end{cases}$$

751 At each time step, with probability $p_{tf}(e)$, we feed the teacher-forced greedy iterate; otherwise, we
752 feed the router’s own predicted iterate.

753 Since LSTMs on long rollouts can suffer from exploding/vanishing gradients, we use truncated
754 backpropagation through time (TBPTT) [Mozier, 2013, Robinson and Fallside, 1987, Werbos, 1988]:
755 the forward pass unrolls the entire trajectory, but gradients are propagated only through the most
756 recent $w_{bptt}(e)$ steps at epoch e . Hidden states are passed forward between segments, while earlier
757 segments are treated as stop-gradient.

758 We employ a curriculum learning approach analogous to scheduled sampling. Let T_{max} be the
759 horizon (300 iterations). With a warm-up of e_w epochs,

$$w_{bptt}(e) = \begin{cases} w_{start} & e \leq e_w \\ \min\left(T_{max}, w_{start}\gamma_{bptt}^{\lfloor \frac{e-e_w}{f_{bptt}} \rfloor}\right) & e > e_w \end{cases} \quad (12)$$

760 so the window grows geometrically by a factor of $\gamma_{bptt} > 1$ every f_{bptt} epochs and is capped at the
761 full trajectory length.

762 Table 3 contains all hyperparameter details for the LSTM routers. The routers for the Paired Solver
763 experiment took a maximum of 4 hours and 30 minutes to train while the routers for the Solver
764 Ensemble experiment took a maximum of 6 hours to train. We then use the model with the best
765 validation loss for testing. The architecture and dataset size is larger for the Solver ensemble
766 experiments to encourage the model to learn some of the nuanced differences between the classes.

Table 3: Hyperparameter settings for routers in Paired solver and Solver Ensemble Experiments

Hyperparameter	Paired Solver	Solver Ensemble
Learning rate	1e-3	1e-3
Hidden dimension	256	512
No. of hidden layers	4	3
Gradient Clipping Norm	1.0	1.0
Weight Decay	0.005	0.005
Batch size	32	32
Training samples	256	1024
Validation samples	32	128
Epochs	200	200
ss_{start}	1.0	1.0
γ_{tf}	0.95	0.95
ss_{end}	0.0	0.0
w_{start}	50	50
γ_{bptt}	1.25	1.25
e_w	10	10
f_{bptt}	1	1

D Additional Experimental Results

This appendix keeps the one experiment of the original submission that the wall-clock study of Section E does not supersede: the paired-solver comparison under Dirichlet boundary conditions, which was run with the original pipeline (a 31×31 grid, a DeepONet with an MLP branch and an LSTM router trained with scheduled sampling; Section C). The other results of the original submission—iteration-based comparisons on 31^2 and 63^2 periodic grids with the LSTM router, residual and Fourier-mode analyses, and ensemble decision traces—are superseded by the per-equation results, the usage analyses and the ensemble study of Section E and have been removed.

D.1 Results with Dirichlet Boundary Conditions

We replicated the paired-solver experiment of the original submission for solutions with zero Dirichlet boundary conditions, with the original pipeline; the metrics are the final error and AUC over 300 iterations, with paired t -tests against the learned router. The dense symmetric-GS reference of that pipeline mis-implemented the SSOR scaling (it added the factor $\omega/(2 - \omega)$ to D^{-1} instead of multiplying it), so the SymGS rows below use a convergent but non-standard symmetric sweep; the implementation has since been corrected and the wall-clock study uses the standard SSOR.

In Table 4, we notice that, similar to PDEs with periodic boundary conditions, the learned greedy router closely matches the performance of the true greedy oracle and outperforms its single solver and HINTS baselines with great statistical significance. Despite the learned router matching the performance of the true greedy oracle, we observe that the learned router doesn't seem to provide much improvement over single-solver baseline in the case of SOR (1.5)-related solvers. This may be a result of the deepnet correction always being weaker relative to the SOR (1.5) correction in terms of error reduction.

Table 4: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances with dirichlet boundaries; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t -tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p -values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value	$\ e_h^{(T)}\ \times 10^3$	$\ e_h^{(T)}\ $ p -value	AUC	AUC p -value
Jacobi-related solvers								
Jacobi Only	0.704 (2.151)	0.001	0.703 (1.929)	$< 10^{-3}$	0.003 (0.006)	0.057	0.169 (0.430)	$< 10^{-3}$
HINTS-Jacobi	2.674 (7.315)	$< 10^{-3}$	0.869 (2.396)	$< 10^{-3}$	0.474 (0.001)	$< 10^{-3}$	0.199 (0.254)	$< 10^{-3}$
Learned Greedy-Jacobi	0.243 (0.851)	-	0.236 (0.719)	-	0.003 (0.006)	-	0.076 (0.162)	-
True-Greedy-Jacobi	0.042 (0.047)	-	0.099 (0.252)	-	0.003 (0.007)	-	0.042 (0.089)	-
GS-related solvers								
GS only	0.138 (0.431)	0.001	0.408 (1.139)	$< 10^{-3}$	0.003 (0.006)	0.028	0.057 (0.146)	$< 10^{-3}$
HINTS-GS	2.668 (7.316)	$< 10^{-3}$	0.841 (2.360)	$< 10^{-3}$	0.470 (0.001)	$< 10^{-3}$	0.115 (0.137)	$< 10^{-3}$
Learned Greedy-GS	0.032 (0.158)	-	0.115 (0.433)	-	0.003 (0.006)	-	0.024 (0.048)	-
True-Greedy-GS	0.010 (0.015)	-	0.063 (0.166)	-	0.003 (0.006)	-	0.017 (0.036)	-
SymGS-related solvers								
SymGS only	2.924 (8.865)	0.01	0.926 (2.740)	0.006	1.331 (3.687)	0.004	0.413 (1.133)	0.004
HINTS-SymGS	2.748 (7.375)	0.008	0.856 (2.427)	0.008	0.917 (1.015)	0.006	0.332 (0.697)	0.003
Learned Greedy-SymGS	1.333 (5.715)	-	0.416 (1.722)	-	0.628 (1.468)	-	0.196 (0.449)	-
True-Greedy-SymGS	0.342 (0.906)	-	0.147 (0.375)	-	0.184 (0.323)	-	0.067 (0.118)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	1.265 (3.768)	$< 10^{-3}$	0.910 (2.468)	$< 10^{-3}$	0.008 (0.021)	0.003	0.253 (0.642)	$< 10^{-3}$
HINTS-Jacobi (0.67)	2.686 (7.311)	$< 10^{-3}$	0.886 (2.412)	$< 10^{-3}$	0.487 (0.002)	$< 10^{-3}$	0.220 (0.279)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.134 (0.275)	-	0.215 (0.660)	-	0.004 (0.010)	-	0.127 (0.393)	-
True-Greedy-Jacobi (0.67)	0.096 (0.220)	-	0.145 (0.408)	-	0.004 (0.013)	-	0.061 (0.131)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.004 (0.011)	0.012	0.145 (0.412)	$< 10^{-3}$	0.003 (0.006)	1.0	0.006 (0.014)	1.0
HINTS-SOR (1.5)	2.650 (7.321)	$< 10^{-3}$	0.791 (2.295)	$< 10^{-3}$	0.468 (0.001)	$< 10^{-3}$	0.014 (0.014)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.004 (0.011)	-	0.039 (0.091)	-	0.003 (0.006)	-	0.006 (0.014)	-
True-Greedy-SOR (1.5)	0.004 (0.011)	-	0.028 (0.070)	-	0.003 (0.006)	-	0.006 (0.014)	-

E Cost-Aware Routing and Wall-Clock Evaluation

The experiments of Section 7 compare methods per iteration, which favors methods whose iterations are expensive. This appendix evaluates the end-to-end wall-clock time to a target accuracy of the learned router against HINTS, the classical solver alone and the classical methods a practitioner would reach for on these problems (geometric multigrid and preconditioned Krylov methods), for all

794 five classical solvers and three PDEs, on 128^2 , 256^2 and 512^2 grids, using the cost-aware instantiation
795 of the greedy rule introduced at the end of Section 4. Besides the Poisson and convection–diffusion
796 equations of Section 7 we add *anisotropic diffusion*, $-\epsilon \partial_{x_1}^2 u - \partial_{x_2}^2 u = f$ with $\epsilon = 10^{-2}$ (diffusion
797 tensor $\text{diag}(\epsilon, 1)$), on the same periodic domain with the same forcing distribution. It is the standard
798 model problem on which point-relaxation methods fail to smooth: an error component oscillatory in
799 x_1 and smooth in x_2 is damped by only $(1 - \epsilon)/(1 + \epsilon) \approx 0.98$ per Jacobi sweep, and geometric
800 multigrid with point smoothing loses its grid-independent convergence (line relaxation or semi-
801 coarsening would be needed). Section E.1 describes the protocol; Sections E.2 to E.4 collect all
802 results per equation (every grid, every tolerance, every baseline, with significance tests and seed
803 trials); Section E.5 studies ensembles of one, two and three classical solvers; Section E.6 accounts for
804 overheads and training cost; and Section E.8 verifies numerically that the assumptions of Theorem 4.1,
805 Propositions 4.2–4.3 and Theorem 5.1 hold in these settings.

806 E.1 Protocol

807 **Fast, matched implementations.** A wall-clock comparison is meaningful only if the classical
808 baseline is not handicapped by implementation. All solvers are therefore re-implemented for the
809 periodic constant-coefficient setting as $O(N^2)$ operations: Jacobi and damped Jacobi as 5-point
810 stencil sweeps, Gauss–Seidel, SOR and symmetric GS via cached sparse triangular factorizations,
811 geometric multigrid as V(2,2)-cycles with lexicographic Gauss–Seidel smoothing, full-weighting
812 restriction, bilinear prolongation, rediscritised coarse operators and a direct solve on the 32×32
813 coarsest grid (error contraction ≈ 0.03 per cycle on the isotropic equations). Their operators and one-
814 step iterates match the dense reference implementations of the original submission to the reference’s
815 own precision (relative error below 10^{-7} , the reference assembling its matrices in single precision;
816 10^{-12} where the reference is exact); the comparison is an assertion in `validate_fast_pde.py`. In
817 the course of this check we found that the original dense symmetric-GS reference mis-implemented
818 the SSOR scaling (the factor $\omega/(2 - \omega)$ was added to D^{-1} instead of multiplying it); it has been
819 corrected, and the wall-clock study uses the standard SSOR throughout. Reference solutions are
820 computed by FFT diagonalization of the same discrete operator, i.e., they are exact solutions of
821 the discrete system, so true errors are available for evaluation at no charged cost; the FFT solve is
822 used only as this reference (it is a direct method that exists because the test problems have constant
823 coefficients and periodic boundaries, and is not a competitor of the iterative methods compared
824 here). All timings are single-threaded CPU measurements (Apple M4 Pro, numpy/scipy/PyTorch,
825 one thread) on an otherwise idle machine.

826 **Corrector.** The neural corrector is a DeepONet applied to the residual $r = f_h - \mathcal{L}_h^a u_h^{(t)}$ and
827 evaluated on a 64×64 sensor grid, i.e., on the coarser grid obtained by band-limited (FFT) restriction
828 to the modes $|k_1|, |k_2| \leq 31$ at every grid size (for the anisotropic equation the sensor grid is $N \times N/4$,
829 i.e., full resolution along x_1 , so that the band contains every mode the point smoothers cannot damp;
830 the sensor grid is the only PDE-specific hyperparameter of the pipeline); its output is prolonged back
831 to the fine grid by exact trigonometric interpolation, so the correction is band-limited by construction
832 and never injects error into the high-frequency band that the classical smoother owns (Figure 2).
833 The trunk net is the fixed orthonormal Fourier basis of that band ($p = 3969$ basis functions, as in
834 POD-DeepONet [Lu et al., 2022]), and the branch net is a linear layer on the 4096 sensor values; it
835 is fit by least squares on 32,000–64,000 exact (residual, error) pairs of the discrete operator drawn
836 from a mixture of spectral shapes (forcing-like, residual-like, white and randomly tilted spectra), and
837 reaches a validation relative error of $\approx 2 \times 10^{-6}$ on every mode of the band. The corrector is applied
838 scale-equivariantly, $C_{\text{NO}}(r) = \|Rr\| \mathcal{G}_\theta(Rr/\|Rr\|)$, which preserves $C_{\text{NO}}(0) = 0$ (Proposition 4.2).
839 Its deployed form is therefore a linear, Fourier-structured map—a least-squares approximation of the
840 exact band-limited inverse $P(\mathcal{L}_h)_{\text{band}}^{-1} R$ of the discrete operator. Replacing it by that exact inverse
841 changes nothing: on the GS pairing at 128^2 the cost-aware oracle’s median work-unit time to 10^{-8}
842 is 4.472 ms with either corrector and its decision sequence is identical on 13 of 16 instances. The
843 gains reported below are thus those of an accurate band-limited spectral correction applied at the
844 right moments; the learned corrector stands in for a spectral inverse that, on these constant-coefficient
845 periodic problems, could also be computed directly. Its cost is dominated by a 4096×4096 matrix–
846 vector product and is therefore almost independent of N (0.65 ms at 128^2 , 3.6 ms at 512^2 , Table 29),
847 i.e., about 12 Jacobi or 3 GS sweeps at 128^2 and about 2.5 Jacobi sweeps at 512^2 (Tables 8, 14
848 and 20). The same corrector is used by HINTS and by all routing policies.

Cost-equalised macro-actions. Per-iteration costs c_j (residual evaluation plus the update) are measured once per pairing and grid on the benchmark machine (minimum over seven repeated blocks of the block median, cached), the unit of cost u is one corrector call, and every operation is applied $m_j = \max\{1, \text{round}(u/c_j)\}$ times per decision; for the five stationary solvers this equalises costs to within 8%. An operation dearer than the unit (a multigrid cycle) is applied once and compared per unit of cost, $\|e_j\|^{u/(m_j c_j)}$; the surrogate weights $\tilde{c}_j$ use the same exponent. Ensembles reuse the cached pairwise costs of their members, so every member has the same macro-action size in the ensemble as in its pairwise run. The cost-aware oracle runs Algorithm 1 on these macro-actions with the true error; the *greedy oracle* rows run the cost-agnostic Algorithm 1 on single operations, as in the main text.

Lightweight router. The learned router selects macro-actions from $6 + K$ scalar features, all $O(1)$ given the residual norm that any stopping test computes: the log relative residual, its per-iteration change over the last macro-action and an exponential moving average thereof, the log epoch index, a one-hot encoding of the previous macro-action, and the log counts of corrector calls so far and of iterations since the last call. It is a two-hidden-layer MLP (64 units) evaluated in numpy, costing $\approx 5 \mu\text{s}$ per decision independently of N . It is trained with the surrogate of Section 5.1, with the per-state weights $\tilde{c}_k$ normalised by $\sum_k \tilde{c}_k$ (which does not change the Bayes decision), on states visited by oracle rollouts with ε -exploration (probability 0.1), followed by two to three DAgger rounds [Ross et al., 2011] in which the current router’s own rollouts are relabelled by the oracle—the batch analogue of the scheduled sampling used in Section C. Training uses 32–128 instances per round (fewer on the larger grids) drawn from the same GRF distribution as the test set but with a different seed; the router of every pairing is trained in seconds to a few minutes. Ensemble routers, which must additionally learn to choose among classical members, use a larger imitation budget (256 rollouts at 128^2 and 128 at 256^2 , six DAgger rounds, hidden width 128, 300 epochs): with the pairwise recipe the router over $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$ imitated its oracle imperfectly at tight tolerances (convection–diffusion, 128^2 , $\varepsilon = 10^{-8}$: 3.34 against the oracle’s 2.67 work-unit ms), which the larger budget removes (2.70). At 512^2 the routers are trained from 32 oracle rollouts of at most 400 iterations (stopping error 10^{-8}); for the convection–diffusion Jacobi pairing, where the first such router missed h^2 within the cap on 4 of 16 instances, the reported router uses 64 rollouts of at most 800 iterations—the only per-cell change to the recipe.

Baselines and metric. HINTS applies the corrector every τ -th iteration; we report $\tau = 25$ (the schedule used in the main text and in Zhang et al. [2024]) and the best of $\tau \in \{5, 10, 25, 50\}$ selected *per cell and per tolerance* on the test instances themselves (an optimistic baseline). We also report the *one-shot schedule*—the corrector once at iteration 0, then the classical solver only—which is the static rule closest to what the router learns at truncation level and therefore the natural ablation for state-dependent routing. The classical baselines without corrector run on the same stencil with the same accounting: multigrid V(2,2) alone; conjugate gradients, CG preconditioned by symmetric GS and by a symmetric V-cycle for Poisson; BiCGSTAB, multigrid-preconditioned BiCGSTAB and restarted GMRES(20) for convection–diffusion (GMRES is accounted at the granularity of its restart cycles and was not run at 512^2 , where it was two orders of magnitude slower than the router on the smaller grids). They are not run for anisotropic diffusion, where point-smoothed multigrid loses its grid-independent contraction (≈ 0.85 per cycle) and would not be informative. The Krylov methods are timed in their SciPy implementations, which keep their own residual bookkeeping instead of the benchmark’s per-iteration true-error check; their iteration counts come from the untimed pass with a per-iteration callback, so their timed runs carry less per-iteration overhead than the hybrid policies, not more. For each test instance (64 at 128^2 , 32 at 256^2 , 16 at 512^2) we record, after every iteration, the true relative error $\|e_h^{(t)}\|/\|u_h\|$ (untimed, benchmark-only) and the cumulative charged wall-clock time of a replay that executes only the chosen operations (residual, update, and—for the learned router—every feature and decision computation; oracle decisions are not charged). We report the median first time at which the error drops below a tolerance ε , paired speedups—the median over instances of the per-instance time ratio, which is the estimand behind every speedup, bold mark and test in this appendix (medians of raw times are shown for scale and can order differently from the paired median on small samples)—and one-sided Wilcoxon signed-rank tests on the paired log time ratios (alternative: the router is faster); the study performs several hundred such tests, so the speedup tables mark as significant only comparisons that survive a Holm correction at level 0.05 over all tests of the table; a run that does not reach ε within the iteration cap (60,000 at 128^2 , 40,000 at 256^2 , 8,000 at 512^2) is marked $\dagger$ with the number of censored instances, reported as a lower

906 bound when it is the majority, counted against the router when it is the router's, and entered at the
 907 time it spent up to the cap on the baseline side of the tests—a labelled lower-bound analysis rather
 908 than a survival analysis. Timed replays are repeated once in the main runs and three times in the
 909 held-out confirmation runs of Section E.7. The cached per-operation costs (minimum over blocks
 910 of the block median) are a lower envelope that is used only to set the macro-action sizes and to
 911 compute work units; all reported live times are measured directly. We highlight $\varepsilon = h^2$ as a nominal
 912 grid-dependent tolerance and verified that it lies below the discretisation error of the test instances:
 913 comparing the exact discrete solution with the exact discrete solution on a four times finer grid for the
 914 same forcing (`discretization_error.py`), the relative discretisation error has median 3.1×10^{-4} ,
 915 7.3×10^{-4} and 2.5×10^{-4} at 128^2 and 7.1×10^{-5} , 1.6×10^{-4} and 5.7×10^{-5} at 256^2 for Poisson,
 916 convection–diffusion and anisotropic diffusion ($4\text{--}12 h^2$; the roughest instances reach $70 h^2$), so
 917 driving the algebraic error below h^2 does not improve the computed solution on any instance; we
 918 also report 10^{-3} and 10^{-8} , and the iteration-based metrics of the main text (final relative error and
 919 $\text{AUC} = \sum_{t=1}^T \|e_h^{(t)}\| / \|u_h\|$ over $T = 300$ iterations) on the 128^2 grid. Every router was additionally
 920 retrained from scratch with five independent seeds (different training instances, exploration and
 921 initialisation; the corrector and the test set are fixed) and re-run on the test set; because those trials ran
 922 in separate sessions, whose live timings are not comparable with the main run's, they are compared
 923 in *work units* (measured per-iteration costs times executed operations, plus $5 \mu\text{s}$ per decision), which
 924 are determined by the decision sequence alone.

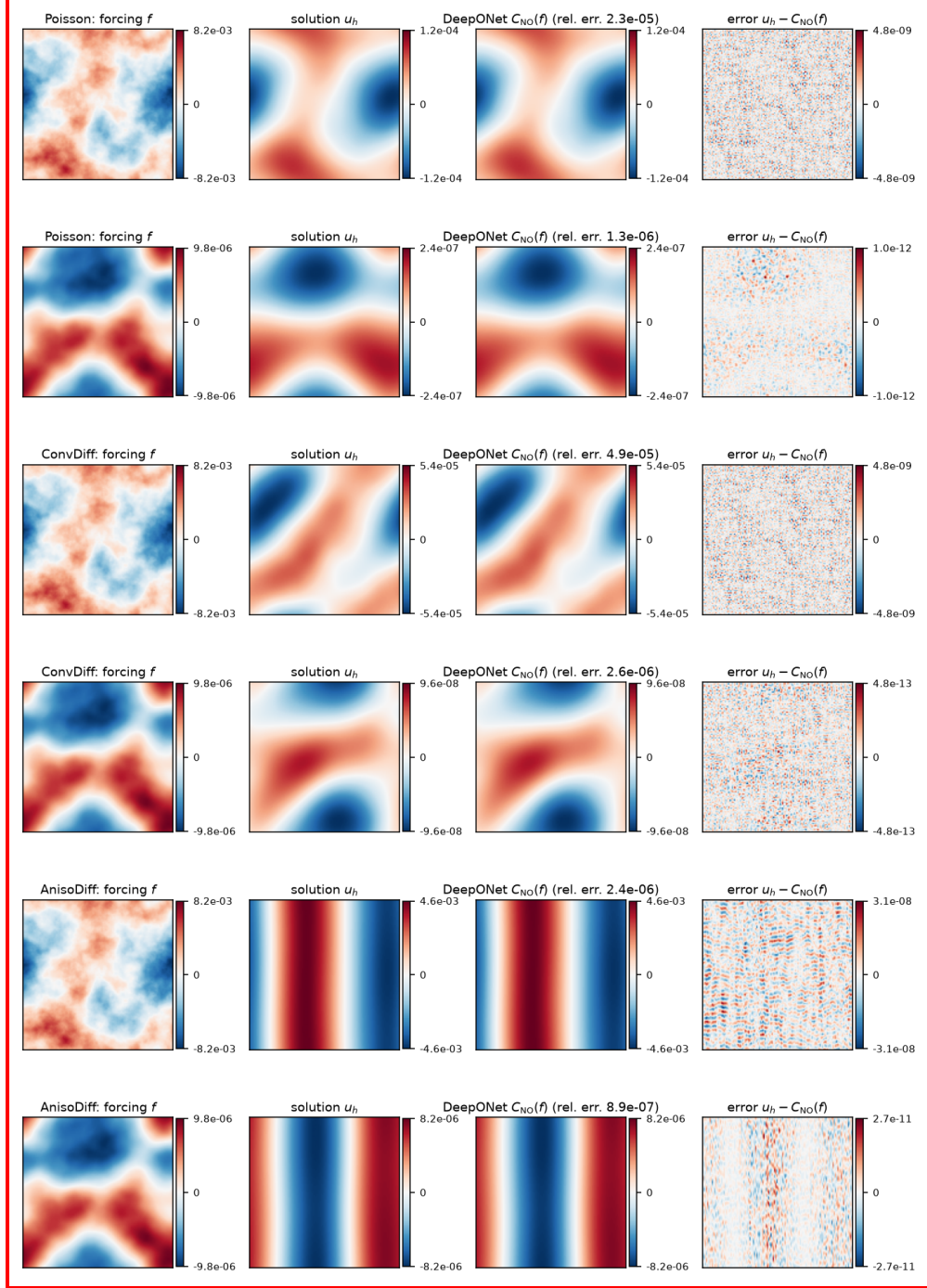


Figure 2: One-shot corrector predictions on the 128×128 grid (two test instances per equation): forcing, exact discrete solution, the DeepONet correction applied to the initial residual ($u_h^{(0)} = 0$), and the remaining error, which is confined to the modes outside the corrector's band (the relative error of the prediction is 10^{-6} – 10^{-5} on the isotropic equations and 10^{-6} on anisotropic diffusion, whose solution is dominated by modes that are smooth in x_2).

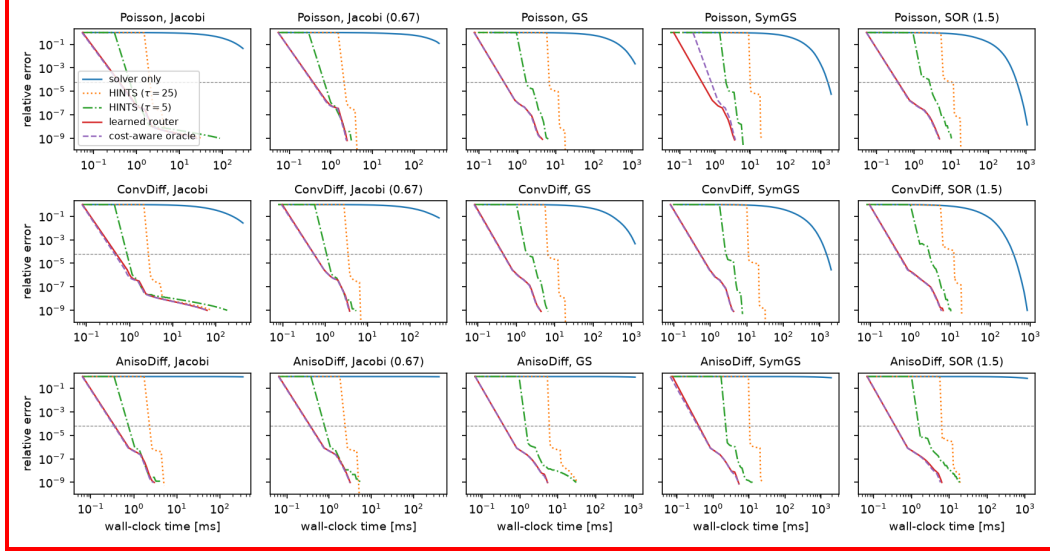


Figure 3: Relative error versus charged wall-clock time on the 128×128 grid for one representative test instance of every pairing (columns) and equation (rows); the dashed line is $\varepsilon = h^2$. The learned router and the cost-aware oracle are indistinguishable.

925 E.2 Poisson

926 Table 5 collects the median time to 10^{-3} , h^2 and 10^{-8} for every pairing, every policy and the classical
 927 baselines on all three grids; Table 6 the paired speedups of the learned router with their p -values;
 928 Figure 4 and table 7 where the iterations go; Table 8 the measured costs and macro-action sizes;
 929 Table 9 the iteration-based metrics; and Table 10 the seed trials.

930 **Uniform gains and their mechanism.** On the 128^2 grid the learned router is the fastest deployable
 931 method at $\varepsilon = h^2$ in every pairing: hundreds to thousands of times faster than the solver alone,
 932 several times faster than HINTS ($\tau=25$) and still faster than the best fixed period chosen per cell and
 933 per tolerance on the test set, with all paired $p < 0.01$ (Table 6); at 10^{-8} the margins over the best
 934 fixed period shrink but remain significant in every pairing, and the best period itself varies across
 935 pairings and tolerances ($\tau=5$ at truncation level, $\tau \in \{5, 10, 50\}$ at 10^{-8}), so no single fixed schedule
 936 is competitive everywhere. Table 7 explains the numbers: with a corrector that removes the smooth
 937 band to 10^{-6} in one call, the cost-aware oracle and the router call it first and reach h^2 after one or
 938 two further sweeps, i.e., in about two iterations, whereas HINTS reaches the same accuracy only
 939 at its first scheduled call (iteration τ), having spent $\tau-1$ sweeps on an error that is almost entirely
 940 smooth. Below h^2 the remaining error is the high-frequency band that the corrector cannot see; the
 941 router therefore stops calling it (one to three calls in total down to 10^{-8} ; Figure 4), whereas a fixed
 942 schedule keeps paying for a call every τ iterations. The cost-agnostic greedy oracle (Algorithm 1 on
 943 single operations) makes the same decisions as its cost-aware counterpart at 128^2 , because after the
 944 first call the corrector is never better than a single sweep on the remaining high-frequency error; the
 945 learned router reproduces the cost-aware oracle’s decisions almost exactly (same median iteration
 946 and call counts, median times within a few percent, the difference being its own decision cost and
 947 timer noise), and the five retrained routers reproduce the main router’s decision sequence to h^2 on
 948 essentially every test instance (Table 10).

949 **Classical baselines and larger grids.** Against the classical methods without corrector (bottom
 950 block of Table 5), the router with its best stationary pairing reaches truncation-level accuracy several
 951 times faster than multigrid alone (three V-cycles) and than multigrid-preconditioned CG at 128^2
 952 and 256^2 , and an order of magnitude faster than unpreconditioned or SymGS-preconditioned CG.
 953 The reason is the corrector’s accuracy on the smooth band: one call removes the modes $|k| \leq 31$ to
 954 10^{-6} at the price of a single 4096×4096 matrix–vector product, whereas a V(2,2)-cycle contracts
 955 the whole spectrum by ≈ 0.03 at the price of four Gauss–Seidel sweeps on the fine grid plus the

coarse-grid work. Routing over $\{\text{NO}, \text{multigrid}\}$ is itself about twice as fast as multigrid alone on every grid: the oracle and the router call the corrector once and let the V-cycle remove the rest, which is the case in which the per-unit-cost comparison of Section 4 matters (the cycle is dearer than the unit). On 256^2 the router remains the fastest iterative method in every pairing ($1.55 \times - 10.9 \times$ over HINTS, $1.11 \times - 2.33 \times$ over the best period across the three equations). At 512^2 the margins shrink but the ordering holds for the stationary pairings, and the $\{\text{NO}, \text{multigrid}\}$ router is the fastest iterative method we measured at this size. With Jacobi and Gauss–Seidel the gains at 512^2 are small because the corrector’s fixed band $|k| \leq 31$ leaves the smoother all modes $k \geq 32$, whose slowest member contracts by only 0.98 per Jacobi sweep at $N = 512$ (against 0.71 at $N = 128$): every policy then needs the same ≈ 80 sweeps after a single corrector call, the wall-clock time is dominated by sweeps, and the schedule barely matters (with Jacobi the router misses h^2 within the 8,000-iteration cap on 2 of 16 instances, marked in the table). This is the expected limit of a corrector whose band does not grow with the grid: the hybrid’s advantage comes from removing the modes the smoother cannot damp, and a corrector with a band that scales with N (at the price of a dense matrix–vector product that grows like the fourth power of the band) or a multigrid member restores the gains, as the multigrid pairing shows.

Table 5: Poisson: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids (64, 32 and 16 test instances) for every pairing $\{\text{NO}, \text{solver}\}$ and policy, and for the classical baselines without corrector. “HINTS (best τ)” is the best of $\tau \in \{5, 10, 25, 50\}$ chosen per cell on the test set itself (the chosen τ is the subscript). The greedy oracle is the cost-agnostic Algorithm 1, the cost-aware oracle is Algorithm 1 on cost-equalised macro-actions; both use the true error and are not deployable. Bold marks the router where it is the fastest deployable method of its pairing; $\dagger n$: n instances did not reach the tolerance within the iteration cap ($>$: lower bound).

Pairing	Method	128 × 128			256 × 256			512 × 512		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	806.1 ms	1.15 s	2.14 s	$>7.61 \text{ s}^{132}$	$>7.61 \text{ s}^{132}$	$>7.61 \text{ s}^{132}$	$>9.85 \text{ s}^{116}$	$>9.85 \text{ s}^{116}$	$>9.85 \text{ s}^{116}$
	HINTS ($\tau=25$)	2.61 ms	2.65 ms	226.1 ms	5.90 ms	5.95 ms	85.5 ms	33.1 ms	87.0 ms	309.2 ms ¹⁷
	HINTS (best τ)	1.06 ms _{$\tau=5$}	1.10 ms _{$\tau=5$}	182.8 ms _{$\tau=50$}	2.07 ms _{$\tau=5$}	2.47 ms _{$\tau=5$}	72.9 ms _{$\tau=50$}	9.27 ms _{$\tau=5$}	87.0 ms _{$\tau=25$}	293.9 ms ¹⁷ _{$\tau=50$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	803 μs	854 μs	155.1 ms	1.28 ms	2.30 ms	67.2 ms	4.74 ms	78.2 ms	275.5 ms ¹⁷
	Cost-aware oracle	880 μs	942 μs	154.1 ms	1.42 ms	2.32 ms	67.0 ms	4.74 ms	74.2 ms	261.5 ms ¹⁷
	Learned router (ours)	918 μs	987 μs	151.3 ms	1.47 ms	2.38 ms	73.1 ms	4.86 ms	78.8 ms¹²	282.9 ms¹⁷
{NO, Jacobi (0.67)}	Solver only	1.16 s	1.62 s	3.16 s	$>7.74 \text{ s}^{132}$	$>7.74 \text{ s}^{132}$	$>7.74 \text{ s}^{132}$	$>10.17 \text{ s}^{116}$	$>10.17 \text{ s}^{116}$	$>10.17 \text{ s}^{116}$
	HINTS ($\tau=25$)	2.71 ms	2.71 ms	5.16 ms	6.02 ms	6.41 ms	16.8 ms	35.2 ms	130.8 ms	420.9 ms
	HINTS (best τ)	1.11 ms _{$\tau=5$}	1.13 ms _{$\tau=5$}	3.03 ms _{$\tau=10$}	2.10 ms _{$\tau=5$}	3.43 ms _{$\tau=5$}	16.8 ms _{$\tau=25$}	9.95 ms _{$\tau=5$}	126.8 ms _{$\tau=50$}	405.7 ms _{$\tau=50$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	818 μs	915 μs	2.83 ms	1.30 ms	2.93 ms	16.2 ms	4.96 ms	119.4 ms	386.2 ms
	Cost-aware oracle	820 μs	922 μs	2.93 ms	1.30 ms	2.77 ms	16.1 ms	4.92 ms	115.7 ms	387.2 ms
	Learned router (ours)	839 μs	912 μs	2.93 ms	1.32 ms	2.83 ms	16.8 ms	5.04 ms	118.2 ms	382.1 ms
{NO, GS}	Solver only	1.29 s	1.83 s	3.45 s	17.80 s	28.83 s	$>31.70 \text{ s}^{132}$	$>29.96 \text{ s}^{116}$	$>29.96 \text{ s}^{116}$	$>29.96 \text{ s}^{116}$
	HINTS ($\tau=25$)	6.43 ms	6.65 ms	13.9 ms	20.4 ms	22.0 ms	43.5 ms	92.5 ms	142.8 ms	439.6 ms
	HINTS (best τ)	1.72 ms _{$\tau=5$}	1.99 ms _{$\tau=5$}	5.55 ms _{$\tau=5$}	4.49 ms _{$\tau=5$}	6.07 ms _{$\tau=5$}	30.4 ms _{$\tau=10$}	20.0 ms _{$\tau=5$}	142.8 ms _{$\tau=25$}	439.6 ms _{$\tau=25$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	835 μs	960 μs	4.34 ms	1.36 ms	3.85 ms	25.4 ms	4.65 ms	141.4 ms	447.1 ms
	Cost-aware oracle	827 μs	953 μs	4.37 ms	1.33 ms	3.78 ms	25.6 ms	4.87 ms	130.0 ms	433.0 ms
	Learned router (ours)	850 μs	1.01 ms	4.45 ms	1.36 ms	3.92 ms	26.0 ms	4.70 ms	131.5 ms	417.8 ms
{NO, SymGS}	Solver only	1.08 s	1.52 s	2.91 s	16.41 s	26.53 s	44.30 s	$>53.47 \text{ s}^{116}$	$>53.47 \text{ s}^{116}$	$>53.47 \text{ s}^{116}$
	HINTS ($\tau=25$)	10.2 ms	10.3 ms	20.2 ms	36.1 ms	37.3 ms	71.9 ms	164.5 ms	216.0 ms	473.9 ms
	HINTS (best τ)	2.35 ms _{$\tau=5$}	2.60 ms _{$\tau=5$}	6.94 ms _{$\tau=5$}	7.18 ms _{$\tau=5$}	8.71 ms _{$\tau=5$}	29.8 ms _{$\tau=5$}	32.1 ms _{$\tau=5$}	128.3 ms _{$\tau=5$}	396.8 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	829 μs	1.09 ms	3.87 ms	1.36 ms	4.18 ms	24.3 ms	4.96 ms	114.7 ms	365.0 ms
	Cost-aware oracle	829 μs	1.13 ms	3.86 ms	1.35 ms	4.19 ms	23.8 ms	4.79 ms	112.6 ms	361.8 ms
	Learned router (ours)	853 μs	1.13 ms	3.88 ms	1.34 ms	4.22 ms	24.4 ms	5.06 ms	115.1 ms	365.9 ms
{NO, SOR (1.5)}	Solver only	424.4 ms	600.9 ms	1.15 s	5.94 s	9.60 s	15.97 s	$>29.74 \text{ s}^{116}$	$>29.74 \text{ s}^{116}$	$>29.74 \text{ s}^{116}$
	HINTS ($\tau=25$)	6.37 ms	12.2 ms	18.5 ms	20.5 ms	40.7 ms	60.9 ms	93.2 ms	182.7 ms	269.4 ms
	HINTS (best τ)	1.72 ms _{$\tau=5$}	3.37 ms _{$\tau=5$}	8.56 ms _{$\tau=10$}	4.57 ms _{$\tau=5$}	9.06 ms _{$\tau=5$}	27.2 ms _{$\tau=10$}	18.7 ms _{$\tau=5$}	74.4 ms _{$\tau=5$}	214.9 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	814 μs	1.03 ms	7.48 ms	1.31 ms	3.69 ms	22.7 ms	4.56 ms	61.1 ms	186.7 ms
	Cost-aware oracle	800 μs	1.02 ms	6.81 ms	1.30 ms	3.51 ms	22.4 ms	4.62 ms	60.3 ms	185.9 ms
	Learned router (ours)	823 μs	1.02 ms	7.05 ms	1.36 ms	3.80 ms	24.2 ms	4.69 ms	60.7 ms	188.5 ms
{NO, Multigrid}	Solver only	2.62 ms	3.84 ms	7.64 ms	9.61 ms	15.1 ms	28.6 ms	47.3 ms	90.8 ms	140.9 ms
	HINTS ($\tau=25$)	2.62 ms	3.83 ms	7.61 ms	9.73 ms	14.7 ms	28.7 ms	47.2 ms	92.0 ms	140.2 ms
	HINTS (best τ)	2.59 ms _{$\tau=5$}	3.81 ms _{$\tau=5$}	5.89 ms _{$\tau=5$}	9.56 ms _{$\tau=50$}	14.6 ms _{$\tau=5$}	20.5 ms _{$\tau=5$}	47.2 ms _{$\tau=25$}	90.5 ms _{$\tau=10$}	102.2 ms _{$\tau=5$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	828 μs	2.01 ms	4.63 ms	1.39 ms	6.14 ms	15.8 ms	4.68 ms	39.5 ms	95.5 ms
	Cost-aware oracle	773 μs	2.02 ms	4.66 ms	1.31 ms	6.09 ms	16.2 ms	4.86 ms	38.6 ms	97.0 ms
	Learned router (ours)	783 μs	2.05 ms	4.72 ms	1.32 ms	5.97 ms	16.6 ms	4.76 ms	38.7 ms	99.5 ms
Classical baselines without corrector (same stencil, same accounting)										
	Multigrid V(2,2) alone	2.75 ms	3.99 ms	7.97 ms	9.27 ms	14.1 ms	27.0 ms	42.8 ms	81.9 ms	126.5 ms
	CG	9.37 ms	11.5 ms	16.3 ms	58.0 ms	73.1 ms	102.5 ms	689.5 ms	920.9 ms	1.18 s
	PCG (SymGS)	20.3 ms	30.8 ms	52.5 ms	146.5 ms	254.7 ms	365.0 ms	1.33 s	2.52 s	3.14 s
	PCG (multigrid)	2.97 ms	4.35 ms	7.36 ms	10.5 ms	15.2 ms	24.5 ms	44.2 ms	85.6 ms	113.2 ms

Table 6: Poisson: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), over the best fixed period for that cell, over multigrid alone and over multigrid-preconditioned CG, with the one-sided Wilcoxon p -value that the router is faster (or, where its median is larger, that it is slower). Bold: $\geq 1.1 \times$ with $p < 0.01$.

Pairing	ε	128 × 128						256 × 256						512 × 512					
		vs. HINTS-25	vs. best τ	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. one-shot
{NO, Jacobi}	h^2	2.68 × (0.10⁻⁹)	1.21 × (0.10⁻⁹)	—	3.43 × (0.10⁻⁹)	3.57 × (0.10⁻⁹)	2.52 × (0.10⁻⁹)	1.56 × (0.10⁻⁹)	—	6.70 × (0.10⁻⁹)	6.30 × (0.10⁻⁹)	1.11 × (0.0001)	1.11 × (0.0001)	—	1.14 × (0.121)	1.26 × (0.086)	—	—	—
	10^{-8}	1.55 × (0.10⁻⁹)	1.22 × (0.10⁻⁹)	—	0.06 × (down, 0.10⁻⁹)	0.05 × (down, 0.10⁻⁹)	1.23 × (0.10⁻⁹)	1.11 × (0.10⁻⁹)	—	0.37 × (down, 0.000)	0.35 × (down, 0.007)	0.05 × (down, 0.590)	0.05 × (down, 0.545)	—	0.45 × (down, 0.002)	0.40 × (down, 0.002)	—	—	—
{NO, Jacobi (0.67)}	h^2	2.90 × (0.10⁻⁹)	1.28 × (0.10⁻⁹)	—	3.62 × (0.10⁻⁹)	3.85 × (0.10⁻⁹)	2.36 × (0.10⁻⁹)	1.73 × (0.10⁻⁹)	—	5.62 × (0.10⁻⁹)	5.29 × (0.10⁻⁹)	1.16 × (0.10⁻⁹)	1.16 × (0.10⁻⁹)	—	0.80 × (down, 0.590)	0.88 × (down, 0.647)	—	—	—
	10^{-8}	1.78 × (0.10⁻⁹)	1.09 × (0.10⁻⁹)	—	2.65 × (0.10⁻⁹)	2.42 × (0.10⁻⁹)	1.05 × (0.10⁻⁹)	1.05 × (0.10⁻⁹)	—	1.65 × (0.10⁻⁹)	1.59 × (0.10⁻⁹)	1.05 × (0.10⁻⁹)	1.04 × (0.0001)	—	0.33 × (down, 0.10⁻⁹)	0.29 × (down, 0.10⁻⁹)	—	—	—
{NO, GS}	h^2	6.75 × (0.10⁻⁹)	1.94 × (0.10⁻⁹)	—	3.39 × (0.10⁻⁹)	3.52 × (0.10⁻⁹)	5.77 × (0.10⁻⁹)	1.70 × (0.10⁻⁹)	—	4.15 × (0.10⁻⁹)	3.85 × (0.10⁻⁹)	1.11 × (0.0001)	1.11 × (0.0001)	—	0.69 × (down, 0.530)	0.76 × (down, 0.590)	—	—	—
	10^{-8}	3.25 × (0.10⁻⁹)	1.28 × (0.10⁻⁹)	—	1.72 × (0.10⁻⁹)	1.75 × (0.10⁻⁹)	1.67 × (0.10⁻⁹)	1.30 × (0.10⁻⁹)	—	1.06 × (0.007)	0.98 × (down, 0.701)	1.07 × (0.0001)	1.07 × (0.0001)	—	0.31 × (down, 0.10⁻⁹)	0.28 × (down, 0.10⁻⁹)	—	—	—
{NO, SymGS}	h^2	9.21 × (0.10⁻⁹)	2.34 × (0.10⁻⁹)	—	3.20 × (0.10⁻⁹)	3.42 × (0.10⁻⁹)	9.96 × (0.10⁻⁹)	2.27 × (0.10⁻⁹)	—	4.00 × (0.10⁻⁹)	3.98 × (0.10⁻⁹)	1.13 × (0.0001)	1.13 × (0.0001)	—	0.78 × (down, 0.510)	0.87 × (down, 0.550)	—	—	—
	10^{-8}	5.26 × (0.10⁻⁹)	1.81 × (0.10⁻⁹)	—	1.90 × (0.10⁻⁹)	1.93 × (0.10⁻⁹)	2.97 × (0.10⁻⁹)	1.22 × (0.10⁻⁹)	—	1.06 × (0.000)	1.05 × (0.000)	1.23 × (0.0001)	1.09 × (0.10⁻⁹)	—	0.34 × (down, 0.10⁻⁹)	0.22 × (down, 0.10⁻⁹)	—	—	—
{NO, SOR (1.5)}	h^2	10.3 × (0.10⁻⁹)	3.22 × (0.10⁻⁹)	—	3.28 × (0.10⁻⁹)	3.45 × (0.10⁻⁹)	10.9 × (0.10⁻⁹)	2.33 × (0.10⁻⁹)	—	4.28 × (0.10⁻⁹)	4.13 × (0.10⁻⁹)	1.10 × (0.10⁻⁹)	1.24 × (0.10⁻⁹)	—	1.40 × (0.002)	1.64 × (0.007)	—	—	—
	10^{-8}	2.63 × (0.10⁻⁹)	1.23 × (0.10⁻⁹)	—	1.14 × (0.10⁻⁹)	1.08 × (0.001)	2.40 × (0.10⁻⁹)	1.09 × (0.10⁻⁹)	—	1.16 × (0.000)	1.04 × (0.000)	1.27 × (0.10⁻⁹)	1.09 × (0.10⁻⁹)	—	0.68 × (down, 0.005)	0.62 × (down, 0.005)	—	—	—
{NO, Multigrid}	h^2	1.96 × (0.10⁻⁹)	1.00 × (0.10⁻⁹)	—	2.62 × (0.10⁻⁹)	3.42 × (0.10⁻⁹)	2.40 × (0.10⁻⁹)	2.30 × (0.10⁻⁹)	—	2.30 × (0.10⁻⁹)	2.03 × (0.10⁻⁹)	2.40 × (0.10⁻⁹)	2.40 × (0.10⁻⁹)	—	1.06 × (0.10⁻⁹)	2.12 × (0.10⁻⁹)	—	—	—
	10^{-8}	1.66 × (0.10⁻⁹)	1.27 × (0.10⁻⁹)	—	1.75 × (0.10⁻⁹)	1.60 × (0.10⁻⁹)	1.71 × (0.10⁻⁹)	1.20 × (0.10⁻⁹)	—	1.72 × (0.10⁻⁹)	1.53 × (0.10⁻⁹)	1.30 × (0.10⁻⁹)	1.09 × (0.002)	—	1.31 × (0.10⁻⁹)	1.23 × (0.009)	—	—	—

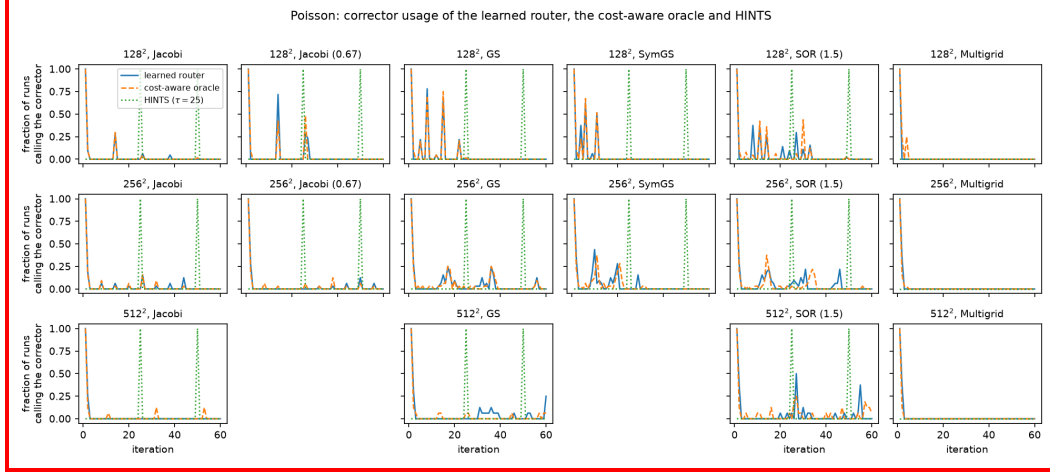


Figure 4: Poisson: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau=25$), for every pairing (columns) and grid (rows; all test instances of each grid).

Table 7: Poisson: median iterations (and corrector calls) to $\varepsilon = h^2$ for HINTS ($\tau=25$), the cost-aware oracle and the learned router on every grid.

Pairing	128 × 128			256 × 256			512 × 512		
	HINTS-25	oracle	router	HINTS-25	oracle	router	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	2 (1)	2 (1)	25 (1)	6 (1)	6 (1)	65 (2)	64 (1)	64 (1)
{NO, Jacobi (0.67)}	25 (1)	2 (1)	2 (1)	25 (1)	8 (1)	8 (1)	98 (4)	95 (1)	95 (1)
{NO, GS}	26 (1)	2 (1)	2 (1)	27 (1)	4 (1)	4 (1)	38 (1)	35 (1)	35 (1)
{NO, SymGS}	26 (1)	2 (1)	2 (1)	26 (1)	3 (1)	3 (1)	32 (1)	18 (1)	18 (2)
{NO, SOR (1.5)}	50 (2)	2 (1)	2 (1)	50 (2)	4 (1)	4 (1)	50 (2)	17 (1)	17 (2)
{NO, Multigrid}	3 (0)	2 (1)	2 (1)	3 (0)	2 (1)	2 (1)	4 (0)	2 (1)	3 (1)

Table 8: Poisson: measured cost of one classical iteration c_{solver} and of one corrector call c_{NO} (residual evaluation included), and the resulting macro-action size m_{solver} (sweeps per decision; the corrector is always applied once).

Pairing	128 × 128			256 × 256			512 × 512		
	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	52 μs	635 μs	12	167 μs	1.00 ms	6	1.07 ms	3.36 ms	3
{NO, Jacobi (0.67)}	52 μs	639 μs	12	150 μs	971 μs	6	914 μs	3.18 ms	3
{NO, GS}	200 μs	643 μs	3	780 μs	1.01 ms	1	3.50 ms	3.30 ms	1
{NO, SymGS}	366 μs	635 μs	2	1.40 ms	931 μs	1	5.82 ms	3.26 ms	1
{NO, SOR (1.5)}	197 μs	635 μs	3	771 μs	960 μs	1	3.41 ms	3.20 ms	1
{NO, Multigrid}	1.25 ms	698 μs	1	4.67 ms	950 μs	1	21.7 ms	2.86 ms	1

Table 9: Poisson, 128×128 : iteration-based metrics of the main text—relative error after $T = 300$ iterations and AUC over the T iterations (mean and standard error over 64 instances; p : one-sided paired t -test that the method’s AUC exceeds the router’s). Bold: router better than both solver-only and HINTS.

Method	$\ e_h^{(T)}\ /\ u_h\ $	AUC	p
{NO, Jacobi}			
Solver only	7.99e-01 (4.5e-03)	2.68e+02 (8.9e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	7.10e-06 (1.9e-06)	2.37e+01 (1.1e-02)	$< 10^{-10}$
Learned router (ours)	6.86e-06 (1.8e-06)	7.15e-03 (1.9e-03)	-
Cost-aware oracle	<i>6.86e-06 (1.8e-06)</i>	<i>7.15e-03 (1.9e-03)</i>	-
{NO, Jacobi (0.67)}			
Solver only	8.56e-01 (3.8e-03)	2.77e+02 (7.1e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	1.15e-12 (1.0e-13)	2.38e+01 (8.3e-03)	$< 10^{-10}$
Learned router (ours)	7.57e-10 (2.9e-11)	2.49e-03 (6.3e-04)	-
Cost-aware oracle	<i>8.19e-10 (1.2e-11)</i>	<i>2.49e-03 (6.3e-04)</i>	-
{NO, GS}			
Solver only	6.53e-01 (5.2e-03)	2.42e+02 (1.2e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.84e-11 (1.9e-12)	2.35e+01 (2.1e-02)	$< 10^{-10}$
Learned router (ours)	7.66e-10 (1.8e-11)	1.94e-03 (4.9e-04)	-
Cost-aware oracle	<i>7.08e-10 (2.5e-11)</i>	<i>1.94e-03 (4.9e-04)</i>	-
{NO, SymGS}			
Solver only	4.49e-01 (4.6e-03)	2.03e+02 (1.3e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	7.14e-10 (5.4e-11)	2.30e+01 (3.5e-02)	$< 10^{-10}$
Learned router (ours)	6.50e-10 (2.4e-11)	1.46e-03 (3.7e-04)	-
Cost-aware oracle	<i>6.32e-10 (2.5e-11)</i>	<i>1.46e-03 (3.7e-04)</i>	-
{NO, SOR (1.5)}			
Solver only	3.09e-01 (3.5e-03)	1.72e+02 (1.3e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	2.16e-10 (1.3e-11)	2.26e+01 (4.6e-02)	$< 10^{-10}$
Learned router (ours)	8.74e-10 (9.7e-12)	3.30e-03 (8.4e-04)	-
Cost-aware oracle	<i>8.36e-10 (1.5e-11)</i>	<i>3.29e-03 (8.4e-04)</i>	-

Table 10: Poisson, 128×128 : five independent router-training seeds per pairing—mean and standard deviation over seeds of the median work-unit time to h^2 , the share of test instances on which the seed router’s decision sequence to h^2 coincides with the main router’s, the range over seeds of the paired speedup over HINTS ($\tau=25$), and the number of seeds significantly faster ($p < 0.01$) than both HINTS ($\tau=25$) and the best fixed period.

Pairing	time to h^2 over 5 seeds	identical decisions	speedup vs. HINTS-25	seeds with $p < 0.01$
{NO, Jacobi}	0.74 $\pm$ 0.00 ms	100%	2.59–2.59 $\times$	5/5
{NO, Jacobi (0.67)}	0.75 $\pm$ 0.00 ms	100%	2.58–2.58 $\times$	5/5
{NO, GS}	0.90 $\pm$ 0.00 ms	100%	6.30–6.30 $\times$	5/5
{NO, SymGS}	1.06 $\pm$ 0.00 ms	100%	9.29–9.29 $\times$	5/5
{NO, SOR (1.5)}	0.89 $\pm$ 0.00 ms	100%	10.45–10.45 $\times$	5/5

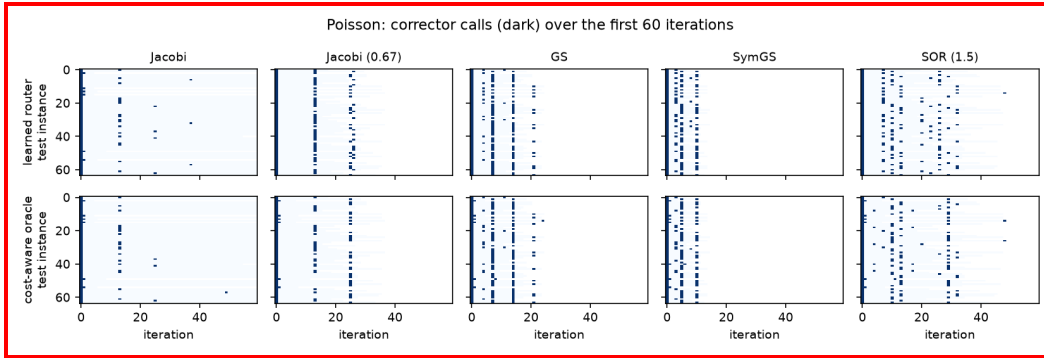


Figure 5: Poisson, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance, for every pairing.

972 E.3 Convection–diffusion

973 The convection–diffusion results (Tables 11 to 16 and fig. 6) mirror the Poisson ones: at 128^2 the
 974 router is the fastest deployable method in every pairing at h^2 and at 10^{-8} , reproduces the cost-aware
 975 oracle, and is $2\text{--}3\times$ faster than multigrid alone and than multigrid-preconditioned BiCGSTAB; on
 976 256^2 the same holds in every pairing; at 512^2 the router is faster than HINTS and than the best
 977 fixed period in the Jacobi, GS and SOR pairings and the {NO, multigrid} router is again the fastest
 978 iterative method, with the same mechanism as on Poisson (the Jacobi and GS pairings are limited by
 979 the modes outside the corrector’s band: the router, the oracle and HINTS all reach h^2 after 84–85
 980 Jacobi iterations). The nonsymmetric operator changes only the details: the best fixed period is $\tau=10$
 981 rather than 5 in several cells, the undamped-Jacobi pairing stalls at the near-Nyquist band below 10^{-6}
 982 for every policy, and GMRES(20) is two orders of magnitude slower than the router.

Table 11: Convection–diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 , 256^2 and 512^2 grids for every pairing and policy, and for the classical baselines without corrector (conventions as in Table 5).

Pairing	Method	128 × 128			256 × 256			512 × 512		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	1.07 s	1.51 s	2.90 s	>12.73 s ¹²⁶	>12.73 s ¹³²	>12.73 s ¹³²	>13.99 s ¹¹⁶	>13.99 s ¹¹⁶	>13.99 s ¹¹⁶
	HINTS ($\tau=25$)	3.41 ms	3.44 ms	384.4 ms	9.15 ms	9.37 ms	500.8 ms	49.2 ms	158.0 ms	1.48 s ¹⁷
	HINTS (best τ)	1.26 ms _{$\tau=5$}	1.28 ms _{$\tau=5$}	321.5 ms _{$\tau=50$}	2.80 ms _{$\tau=5$}	4.95 ms _{$\tau=10$}	475.6 ms _{$\tau=50$}	13.1 ms _{$\tau=5$}	147.4 ms _{$\tau=50$}	1.43 s ¹⁷ _{$\tau=50$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	874 μ s	992 μ s	281.0 ms	1.57 ms	4.43 ms	436.3 ms	6.23 ms	156.1 ms	1.37 s ¹⁷
	Cost-aware oracle	979 μ s	1.15 ms	281.8 ms	1.70 ms	4.81 ms	437.5 ms	6.23 ms	154.3 ms	1.31 s ¹⁷
	Learned router (ours)	1.01 ms	1.14 ms	292.7 ms	1.77 ms	4.76 ms	435.1 ms	6.67 ms	144.3 ms	1.29 s¹⁷
{NO, Jacobi (0.67)}	Solver only	1.55 s	2.22 s	4.32 s	>12.55 s ¹³²	>12.55 s ¹³²	>12.55 s ¹³²	>12.96 s ¹¹⁶	>12.96 s ¹¹⁶	>12.96 s ¹¹⁶
	HINTS ($\tau=25$)	3.40 ms	3.40 ms	6.53 ms	9.03 ms	9.31 ms	30.4 ms	45.8 ms	203.4 ms	575.0 ms
	HINTS (best τ)	1.28 ms _{$\tau=5$}	1.31 ms _{$\tau=5$}	3.79 ms _{$\tau=10$}	2.73 ms _{$\tau=5$}	5.68 ms _{$\tau=10$}	30.4 ms _{$\tau=25$}	12.2 ms _{$\tau=5$}	203.4 ms _{$\tau=25$}	575.0 ms _{$\tau=25$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	885 μ s	1.10 ms	3.56 ms	1.54 ms	5.93 ms	26.8 ms	6.38 ms	220.3 ms	600.4 ms
	Cost-aware oracle	878 μ s	1.08 ms	3.61 ms	1.58 ms	5.94 ms	27.5 ms	6.15 ms	218.4 ms	594.8 ms
	Learned router (ours)	912 μs	1.10 ms	3.70 ms	1.61 ms	5.76 ms	26.9 ms	6.22 ms	222.2 ms	587.6 ms
{NO, GS}	Solver only	1.15 s	1.63 s	3.14 s	18.36 s	30.05 s	>36.70 s ¹³²	>32.99 s ¹¹⁶	>32.99 s ¹¹⁶	>32.99 s ¹¹⁶
	HINTS ($\tau=25$)	7.03 ms	8.27 ms	20.5 ms	23.5 ms	31.0 ms	59.3 ms	105.1 ms	214.1 ms	504.5 ms
	HINTS (best τ)	1.90 ms _{$\tau=5$}	3.60 ms _{$\tau=5$}	6.24 ms _{$\tau=5$}	5.29 ms _{$\tau=5$}	10.5 ms _{$\tau=5$}	36.7 ms _{$\tau=10$}	22.0 ms _{$\tau=5$}	192.7 ms _{$\tau=10$}	504.5 ms _{$\tau=25$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	887 μ s	1.23 ms	5.32 ms	1.57 ms	7.20 ms	31.1 ms	5.70 ms	184.8 ms	493.9 ms
	Cost-aware oracle	909 μ s	1.32 ms	5.46 ms	1.61 ms	7.15 ms	31.0 ms	5.97 ms	185.9 ms	489.9 ms
	Learned router (ours)	906 μs	1.35 ms	5.59 ms	1.57 ms	7.05 ms	31.4 ms	5.79 ms	179.9 ms	486.0 ms
{NO, SymGS}	Solver only	1.10 s	1.56 s	2.99 s	16.90 s	27.77 s	46.85 s	>58.57 s ¹¹⁶	>58.57 s ¹¹⁶	>58.57 s ¹¹⁶
	HINTS ($\tau=25$)	10.7 ms	11.1 ms	21.6 ms	39.2 ms	42.3 ms	78.1 ms	183.8 ms	282.8 ms	567.3 ms
	HINTS (best τ)	2.52 ms _{$\tau=5$}	2.96 ms _{$\tau=5$}	7.35 ms _{$\tau=5$}	7.80 ms _{$\tau=5$}	10.9 ms _{$\tau=5$}	34.8 ms _{$\tau=5$}	36.1 ms _{$\tau=5$}	187.9 ms _{$\tau=5$}	496.9 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	911 μ s	1.25 ms	4.91 ms	1.58 ms	6.93 ms	28.4 ms	6.75 ms	169.4 ms	448.9 ms
	Cost-aware oracle	905 μ s	1.27 ms	4.48 ms	1.56 ms	6.89 ms	28.4 ms	6.21 ms	169.4 ms	448.3 ms
	Learned router (ours)	924 μs	1.28 ms	4.54 ms	1.55 ms	6.96 ms	29.6 ms	6.23 ms	172.6 ms	453.2 ms
{NO, SOR (1.5)}	Solver only	270.0 ms	383.6 ms	737.6 ms	5.14 s	8.46 s	14.22 s	>32.91 s ¹¹⁶	>32.91 s ¹¹⁶	>32.91 s ¹¹⁶
	HINTS ($\tau=25$)	6.92 ms	13.6 ms	20.2 ms	23.2 ms	46.1 ms	69.0 ms	103.8 ms	206.6 ms	308.3 ms
	HINTS (best τ)	1.98 ms _{$\tau=5$}	3.64 ms _{$\tau=5$}	9.62 ms _{$\tau=5$}	5.22 ms _{$\tau=5$}	10.6 ms _{$\tau=5$}	31.2 ms _{$\tau=5$}	22.1 ms _{$\tau=5$}	89.3 ms _{$\tau=5$}	234.2 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	919 μ s	1.39 ms	7.66 ms	1.56 ms	5.90 ms	28.1 ms	6.82 ms	83.3 ms	212.8 ms
	Cost-aware oracle	922 μ s	1.63 ms	7.97 ms	1.57 ms	5.74 ms	27.5 ms	5.68 ms	85.0 ms	217.1 ms
	Learned router (ours)	934 μs	1.59 ms	8.18 ms	1.57 ms	5.80 ms	27.6 ms	5.64 ms	83.6 ms	216.7 ms
{NO, Multigrid}	Solver only	3.18 ms	4.71 ms	9.28 ms	10.9 ms	16.4 ms	31.8 ms	49.5 ms	90.8 ms	148.5 ms
	HINTS ($\tau=25$)	3.17 ms	4.68 ms	9.28 ms	10.9 ms	16.7 ms	31.6 ms	50.0 ms	97.6 ms	150.4 ms
	HINTS (best τ)	3.17 ms _{$\tau=25$}	4.66 ms _{$\tau=5$}	7.14 ms _{$\tau=5$}	10.9 ms _{$\tau=5$}	16.5 ms _{$\tau=5$}	23.0 ms _{$\tau=5$}	49.4 ms _{$\tau=10$}	92.5 ms _{$\tau=10$}	112.8 ms _{$\tau=5$}
	One-shot schedule	—	—	—	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	896 μ s	2.34 ms	5.57 ms	1.58 ms	7.30 ms	20.8 ms	6.18 ms	50.7 ms	103.5 ms
	Cost-aware oracle	851 μ s	2.33 ms	5.49 ms	1.55 ms	7.23 ms	19.9 ms	5.74 ms	50.9 ms	104.1 ms
	Learned router (ours)	873 μs	2.32 ms	5.50 ms	1.60 ms	7.26 ms	20.2 ms	5.63 ms	53.7 ms	102.6 ms
Classical baselines without corrector (same stencil, same accounting)										
{NO, Multigrid}	Multigrid V(2,2) alone	3.06 ms	4.52 ms	8.92 ms	12.3 ms	18.8 ms	36.1 ms	52.4 ms	99.7 ms	150.7 ms
	BiCGSTAB	56.0 ms	67.6 ms	93.1 ms	377.7 ms	486.6 ms	605.9 ms	3.79 s	5.56 s	6.72 s ¹¹
	BiCGSTAB (multigrid)	3.12 ms	6.17 ms	9.21 ms	12.5 ms	24.0 ms	36.1 ms	46.1 ms	93.5 ms	144.8 ms
	GMRES(20)	301.7 ms	434.1 ms	882.8 ms	1.85 s	3.00 s	5.14 s	—	—	—

Table 12: Convection–diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$), the best fixed period, multigrid alone and multigrid-preconditioned BiCGSTAB, with one-sided Wilcoxon p -values (conventions as in Table 6).

Pairing	ε	128 × 128				256 × 256				512 × 512			
		vs. HINTS-25	vs. best τ	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ	vs. one-shot	vs. multigrid	vs. MG-Krylov	vs. HINTS-25	vs. best τ
{NO, Jacobi}	h^2	2.95 × (c 10 ⁻⁹)	1.34 × (c 10 ⁻⁹)	—	3.54 × (c 10 ⁻⁹)	5.18 × (c 10 ⁻⁹)	1.92 × (c 10 ⁻⁹)	1.30 × (c 10 ⁻⁹)	—	4.60 × (c 10 ⁻⁹)	5.06 × (c 10 ⁻⁹)	1.17 × (c 10 ⁻⁹)	1.14 × (c 10 ⁻⁹)
	10^{-8}	1.35 × (c 10 ⁻⁹)	1.17 × (c 10 ⁻⁹)	—	0.01 × (c 10 ⁻⁹)	0.01 × (c 10 ⁻⁹)	3.12 × (c 10 ⁻⁹)	1.00 × (c 10 ⁻⁹)	—	0.03 × (c 10 ⁻⁹)	0.03 × (c 10 ⁻⁹)	0.09 × (c 10 ⁻⁹)	0.08 × (c 10 ⁻⁹)
{NO, Jacobi (0.67)}	h^2	3.14 × (c 10 ⁻⁹)	1.32 × (c 10 ⁻⁹)	—	3.71 × (c 10 ⁻⁹)	5.31 × (c 10 ⁻⁹)	1.88 × (c 10 ⁻⁹)	1.25 × (c 10 ⁻⁹)	—	3.85 × (c 10 ⁻⁹)	4.42 × (c 10 ⁻⁹)	1.08 × (c 10 ⁻⁹)	1.08 × (c 10 ⁻⁹)
	10^{-8}	1.77 × (c 10 ⁻⁹)	1.07 × (c 10 ⁻⁹)	—	2.37 × (c 10 ⁻⁹)	2.86 × (c 10 ⁻⁹)	1.06 × (c 10 ⁻⁹)	1.06 × (c 10 ⁻⁹)	—	1.27 × (c 10 ⁻⁹)	1.36 × (c 10 ⁻⁹)	1.08 × (c 10 ⁻⁹)	1.08 × (c 10 ⁻⁹)
{NO, GS}	h^2	6.76 × (c 10 ⁻⁹)	2.60 × (c 10 ⁻⁹)	—	3.26 × (c 10 ⁻⁹)	4.62 × (c 10 ⁻⁹)	8.25 × (c 10 ⁻⁹)	1.49 × (c 10 ⁻⁹)	—	3.06 × (c 10 ⁻⁹)	3.67 × (c 10 ⁻⁹)	1.05 × (c 10 ⁻⁹)	1.05 × (c 10 ⁻⁹)
	10^{-8}	3.76 × (c 10 ⁻⁹)	1.17 × (c 10 ⁻⁹)	—	1.60 × (c 10 ⁻⁹)	1.91 × (c 10 ⁻⁹)	1.86 × (c 10 ⁻⁹)	1.17 × (c 10 ⁻⁹)	—	1.16 × (c 10 ⁻⁹)	1.19 × (c 10 ⁻⁹)	1.01 × (c 10 ⁻⁹)	1.01 × (c 10 ⁻⁹)
{NO, SymGS}	h^2	6.76 × (c 10 ⁻⁹)	2.60 × (c 10 ⁻⁹)	—	3.26 × (c 10 ⁻⁹)	4.62 × (c 10 ⁻⁹)	8.25 × (c 10 ⁻⁹)	1.49 × (c 10 ⁻⁹)	—	3.06 × (c 10 ⁻⁹)	3.67 × (c 10 ⁻⁹)	1.05 × (c 10 ⁻⁹)	1.05 × (c 10 ⁻⁹)
	10^{-8}	4.76 × (c 10 ⁻⁹)	1.62 × (c 10 ⁻⁹)	—	1.91 × (c 10 ⁻⁹)	2.08 × (c 10 ⁻⁹)	2.63 × (c 10 ⁻⁹)	1.18 × (c 10 ⁻⁹)	—	1.24 × (c 10 ⁻⁹)	1.24 × (c 10 ⁻⁹)	1.10 × (c 10 ⁻⁹)	1.10 × (c 10 ⁻⁹)
{NO, SOR (1.5)}	h^2	8.86 × (c 10 ⁻⁹)	2.80 × (c 10 ⁻⁹)	—	3.07 × (c 10 ⁻⁹)	4.75 × (c 10 ⁻⁹)	7.99 × (c 10 ⁻⁹)	1.96 × (c 10 ⁻⁹)	—	3.72 × (c 10 ⁻⁹)	4.15 × (c 10 ⁻⁹)	2.40 × (c 10 ⁻⁹)	1.36 × (c 10 ⁻⁹)
	10^{-8}	2.98 × (c 10 ⁻⁹)	1.31 × (c 10 ⁻⁹)	—	1.16 × (c 10 ⁻⁹)	1.27 × (c 10 ⁻⁹)	2.90 × (c 10 ⁻⁹)	1.19 × (c 10 ⁻⁹)	—	1.32 × (c 10 ⁻⁹)	1.29 × (c 10 ⁻⁹)	1.41 × (c 10 ⁻⁹)	1.06 × (c 10 ⁻⁹)
{NO, Multigrid}	h^2	3.07 × (c 10 ⁻⁹)	2.81 × (c 10 ⁻⁹)	—	1.12 × (c 10 ⁻⁹)	2.44 × (c 10 ⁻⁹)	2.19 × (c 10 ⁻⁹)	2.28 × (c 10 ⁻⁹)	—	2.56 × (c 10 ⁻⁹)	3.27 × (c 10 ⁻⁹)	1.63 × (c 10 ⁻⁹)	1.75 × (c 10 ⁻⁹)
	10^{-8}	1.70 × (c 10 ⁻⁹)	1.28 × (c 10 ⁻⁹)	—	1.65 × (c 10 ⁻⁹)	1.74 × (c 10 ⁻⁹)	1.54 × (c 10 ⁻⁹)	1.14 × (c 10 ⁻⁹)	—	1.70 × (c 10 ⁻⁹)	1.81 × (c 10 ⁻⁹)	1.37 × (c 10 ⁻⁹)	1.07 × (c 10 ⁻⁹)

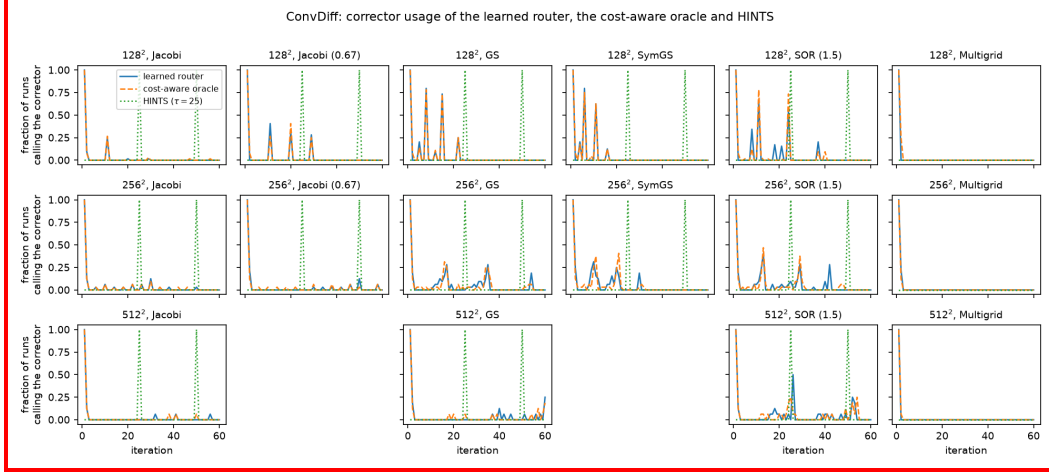


Figure 6: Convection–diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 13: Convection–diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$ on every grid.

Pairing	128 × 128			256 × 256			512 × 512		
	HINTS-25	oracle	router	HINTS-25	oracle	router	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	2 (1)	2 (1)	25 (1)	10 (1)	10 (1)	85 (2)	84 (1)	84 (1)
{NO, Jacobi (0.67)}	25 (1)	4 (1)	4 (1)	25 (1)	15 (1)	15 (1)	128 (4)	124 (1)	124 (1)
{NO, GS}	30 (1)	2 (1)	2 (1)	33 (1)	7 (1)	7 (1)	51 (2)	44 (1)	44 (2)
{NO, SymGS}	26 (1)	2 (1)	2 (1)	27 (1)	4 (1)	4 (1)	38 (1)	24 (1)	24 (2)
{NO, SOR (1.5)}	50 (2)	4 (1)	4 (1)	50 (2)	6 (1)	6 (1)	50 (2)	20 (1)	20 (2)
{NO, Multigrid}	3 (0)	2 (1)	2 (1)	3 (0)	2 (1)	2 (1)	4 (0)	3 (1)	3 (1)

Table 14: Convection–diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128			256 × 256			512 × 512		
	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	72 μs	658 μs	9	286 μs	1.13 ms	4	1.42 ms	3.61 ms	3
{NO, Jacobi (0.67)}	71 μs	659 μs	9	288 μs	1.15 ms	4	1.57 ms	3.87 ms	2
{NO, GS}	234 μs	673 μs	3	855 μs	1.08 ms	1	3.94 ms	3.67 ms	1
{NO, SymGS}	378 μs	665 μs	2	1.55 ms	1.11 ms	1	6.88 ms	3.76 ms	1
{NO, SOR (1.5)}	223 μs	660 μs	3	813 μs	1.03 ms	1	3.90 ms	3.62 ms	1
{NO, Multigrid}	1.47 ms	737 μs	1	5.68 ms	1.06 ms	1	24.7 ms	3.60 ms	1

Table 15: Convection–diffusion, 128×128 : iteration-based metrics (conventions as in Table 9).

Method	$\ e_h^{(T)}\ /\ u_h\ $	AUC	p
{NO, Jacobi}			
Solver only	7.43e-01 (9.1e-03)	2.57e+02 (1.9e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	1.55e-05 (4.2e-06)	2.36e+01 (2.7e-02)	$< 10^{-10}$
Learned router (ours)	1.50e-05 (4.1e-06)	1.55e-02 (4.1e-03)	-
Cost-aware oracle	1.50e-05 (4.1e-06)	1.55e-02 (4.1e-03)	-
{NO, Jacobi (0.67)}			
Solver only	8.11e-01 (7.9e-03)	2.69e+02 (1.5e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	1.46e-12 (2.7e-13)	2.37e+01 (2.1e-02)	$< 10^{-10}$
Learned router (ours)	7.96e-10 (1.6e-11)	5.39e-03 (1.4e-03)	-
Cost-aware oracle	8.18e-10 (1.3e-11)	5.39e-03 (1.4e-03)	-
{NO, GS}			
Solver only	5.46e-01 (9.9e-03)	2.20e+02 (2.4e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	8.78e-11 (4.8e-12)	2.32e+01 (5.4e-02)	$< 10^{-10}$
Learned router (ours)	6.84e-10 (2.8e-11)	4.03e-03 (1.0e-03)	-
Cost-aware oracle	6.73e-10 (2.8e-11)	4.03e-03 (1.0e-03)	-
{NO, SymGS}			
Solver only	3.77e-01 (9.1e-03)	1.84e+02 (2.7e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	3.64e-12 (2.3e-13)	2.26e+01 (8.1e-02)	$< 10^{-10}$
Learned router (ours)	5.19e-10 (3.4e-11)	3.16e-03 (8.0e-04)	-
Cost-aware oracle	5.00e-10 (3.2e-11)	3.16e-03 (8.0e-04)	-
{NO, SOR (1.5)}			
Solver only	1.23e-01 (3.4e-03)	1.17e+02 (1.9e+00)	$< 10^{-10}$
HINTS ($\tau=25$)	8.06e-10 (2.3e-11)	2.12e+01 (1.2e-01)	$< 10^{-10}$
Learned router (ours)	8.08e-10 (1.3e-11)	6.49e-03 (1.7e-03)	-
Cost-aware oracle	7.77e-10 (1.8e-11)	6.49e-03 (1.7e-03)	-

Table 16: Convection–diffusion, 128×128 : five-seed router-training trials (conventions as in Table 10).

Pairing	time to h^2 over 5 seeds	identical decisions	speedup vs. HINTS-25	seeds with $p < 0.01$
{NO, Jacobi}	0.84 ± 0.00 ms	100%	$2.91\text{--}2.91\times$	5/5
{NO, Jacobi (0.67)}	0.91 ± 0.00 ms	100%	$2.67\text{--}2.67\times$	5/5
{NO, GS}	1.11 ± 0.00 ms	100%	$7.22\text{--}7.22\times$	5/5
{NO, SymGS}	1.12 ± 0.00 ms	100%	$9.10\text{--}9.10\times$	5/5
{NO, SOR (1.5)}	1.30 ± 0.00 ms	100%	$9.41\text{--}9.41\times$	5/5

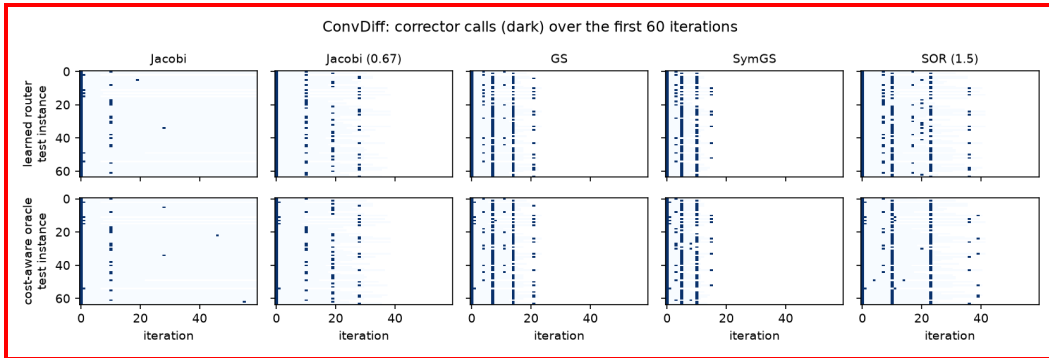


Figure 7: Convection–diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

983 E.4 Anisotropic diffusion

984 The anisotropic equation is where the two components of the hybrid are most complementary and
 985 where the classical solvers are weakest: the point smoothers cannot damp the x_1 -oscillatory modes

at all (Table 17: none of the five solvers alone reaches h^2 within the iteration cap on either grid), whereas the corrector, whose sensor grid keeps full resolution along x_1 , removes exactly those modes together with the smooth ones in a single call and leaves the smoother only the x_2 -oscillatory band, which it damps at the isotropic rate. Consequently the learned router reaches h^2 in under 1 ms at 128^2 for every pairing—the same absolute time as on the isotropic equations—while HINTS ($\tau=25$) needs 3–10 ms and the best fixed period 1.2–2.4 ms; at 256^2 (Jacobi, GS and SOR pairings, 16 instances) the router needs 9–10 ms against 15–30 ms for HINTS and 10–13 ms for the best period (Table 18). The router again matches the cost-aware oracle, its decisions are identical across the five training seeds (Table 22), and it spends the iterations after the single corrector call on the x_2 -oscillatory band (Figure 8 and table 19). The multigrid and Krylov baselines are not run for this equation (Section E.1), and neither is the 512^2 grid: the anisotropic corrector keeps full resolution along x_1 ($N \times N/4$ sensors), so at 512^2 its least-squares branch would be a dense $65,536 \times 65,536$ matrix (17 GB in single precision), which the current design cannot hold; a coarser x_1 resolution would leave the point smoothers modes they cannot damp (Section E.5 discusses exactly this regime).

Table 17: Anisotropic diffusion: median wall-clock time to relative error $\varepsilon \in \{10^{-3}, h^2, 10^{-8}\}$ on the 128^2 (64 instances, five pairings) and 256^2 (16 instances; Jacobi, GS and SOR pairings) grids for every policy (conventions as in Table 5; the solver-only runs are capped at 60,000 and 40,000 iterations and never reach h^2).

Pairing	Method	128 × 128			256 × 256		
		$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$	$\varepsilon=10^{-3}$	$\varepsilon=h^2$	$\varepsilon=10^{-8}$
{NO, Jacobi}	Solver only	>4.62 s ¹⁶⁴	>4.62 s ¹⁶⁴	>4.62 s ¹⁶⁴	>9.29 s ¹¹⁶	>9.29 s ¹¹⁶	>9.29 s ¹¹⁶
	HINTS ($\tau=25$)	2.90 ms	2.90 ms	67.6 ms	14.9 ms	14.9 ms	127.4 ms
	HINTS (best τ)	1.15 ms _{$\tau=5$}	1.15 ms _{$\tau=5$}	58.8 ms _{$\tau=50$}	10.2 ms _{$\tau=5$}	10.3 ms _{$\tau=5$}	94.6 ms _{$\tau=50$}
	One-shot schedule	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	801 μ s	826 μ s	49.6 ms	9.19 ms	9.29 ms	65.1 ms
	Cost-aware oracle	876 μ s	892 μ s	49.9 ms	9.26 ms	9.42 ms	65.3 ms
	Learned router (ours)	903 μs	911 μs	55.6 ms	9.35 ms	9.55 ms	65.6 ms
{NO, Jacobi (0.67)}	Solver only	>4.52 s ¹⁶⁴	>4.52 s ¹⁶⁴	>4.52 s ¹⁶⁴	>9.36 s ¹¹⁶	>9.36 s ¹¹⁶	>9.36 s ¹¹⁶
	HINTS ($\tau=25$)	2.85 ms	2.85 ms	5.46 ms	14.9 ms	14.9 ms	29.5 ms
	HINTS (best τ)	1.16 ms _{$\tau=5$}	1.18 ms _{$\tau=5$}	4.21 ms _{$\tau=10$}	10.1 ms _{$\tau=5$}	10.2 ms _{$\tau=5$}	23.2 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	787 μ s	810 μ s	3.70 ms	9.08 ms	9.25 ms	22.9 ms
	Cost-aware oracle	802 μ s	823 μ s	3.81 ms	9.10 ms	9.22 ms	29.3 ms
	Learned router (ours)	831 μs	850 μs	3.86 ms	9.16 ms	9.38 ms	25.7 ms
{NO, GS}	Solver only	>13.63 s ¹⁶⁴	>13.63 s ¹⁶⁴	>13.63 s ¹⁶⁴	>34.18 s ¹¹⁶	>34.18 s ¹¹⁶	>34.18 s ¹¹⁶
	HINTS ($\tau=25$)	6.48 ms	6.48 ms	23.7 ms	29.7 ms	29.7 ms	64.5 ms
	HINTS (best τ)	1.73 ms _{$\tau=5$}	1.74 ms _{$\tau=5$}	16.5 ms _{$\tau=5$}	12.7 ms _{$\tau=5$}	12.8 ms _{$\tau=5$}	37.8 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	812 μ s	831 μ s	5.87 ms	9.24 ms	9.40 ms	38.7 ms
	Cost-aware oracle	802 μ s	833 μ s	5.33 ms	9.20 ms	9.32 ms	35.1 ms
	Learned router (ours)	827 μs	852 μs	5.40 ms	9.20 ms	9.30 ms	35.9 ms
{NO, SymGS}	Solver only	>23.15 s ¹⁶⁴	>23.15 s ¹⁶⁴	>23.15 s ¹⁶⁴	—	—	—
	HINTS ($\tau=25$)	10.3 ms	10.3 ms	20.5 ms	—	—	—
	HINTS (best τ)	2.40 ms _{$\tau=5$}	2.40 ms _{$\tau=5$}	7.45 ms _{$\tau=5$}	—	—	—
	One-shot schedule	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	811 μ s	832 μ s	5.63 ms	—	—	—
	Cost-aware oracle	826 μ s	844 μ s	5.59 ms	—	—	—
	Learned router (ours)	856 μs	871 μs	5.47 ms	—	—	—
{NO, SOR (1.5)}	Solver only	>13.80 s ¹⁶⁴	>13.80 s ¹⁶⁴	>13.80 s ¹⁶⁴	>33.90 s ¹¹⁶	>33.90 s ¹¹⁶	>33.90 s ¹¹⁶
	HINTS ($\tau=25$)	6.62 ms	6.62 ms	17.9 ms	29.6 ms	29.7 ms	60.2 ms
	HINTS (best τ)	1.78 ms _{$\tau=5$}	1.80 ms _{$\tau=5$}	12.8 ms _{$\tau=10$}	12.7 ms _{$\tau=5$}	12.8 ms _{$\tau=5$}	51.7 ms _{$\tau=10$}
	One-shot schedule	—	—	—	—	—	—
	Greedy oracle (Alg. 1)	813 μ s	843 μ s	7.60 ms	9.21 ms	9.33 ms	43.3 ms
	Cost-aware oracle	823 μ s	862 μ s	6.71 ms	9.12 ms	9.27 ms	34.2 ms
	Learned router (ours)	845 μs	869 μs	7.68 ms	9.18 ms	9.93 ms	44.1 ms

Table 18: Anisotropic diffusion: paired median speedup of the learned router at $\varepsilon = h^2$ and 10^{-8} over HINTS ($\tau=25$) and the best fixed period, with one-sided Wilcoxon p -values (no classical baselines without corrector are run for this equation).

Pairing	ε	128 × 128			256 × 256		
		vs. HINTS-25	vs. best τ	vs. one-shot	vs. HINTS-25	vs. best τ	vs. one-shot
{NO, Jacobi}	h^2	3.16 × ($< 10^{-10}$)	1.25 × (10^{-9})	—	1.58 × (10^{-5})	1.11 × (10^{-4})	—
	10^{-8}	1.44 × ($< 10^{-10}$)	1.21 × ($< 10^{-10}$)	—	1.93 × (10^{-5})	1.80 × (10^{-5})	—
{NO, Jacobi (0.67)}	h^2	3.31 × ($< 10^{-10}$)	1.34 × ($< 10^{-10}$)	—	1.55 × (10^{-5})	1.11 × (10^{-4})	—
	10^{-8}	1.43 × ($< 10^{-10}$)	1.17 × (10^{-9})	—	1.13 × (0.042)	1.17 × (0.002)	—
{NO, GS}	h^2	7.61 × ($< 10^{-10}$)	1.99 × ($< 10^{-10}$)	—	3.18 × (10^{-5})	1.37 × (10^{-5})	—
	10^{-8}	4.05 × ($< 10^{-10}$)	2.74 × ($< 10^{-10}$)	—	2.26 × (10^{-5})	1.64 × (10^{-5})	—
{NO, SymGS}	h^2	11.8 × ($< 10^{-10}$)	2.72 × ($< 10^{-10}$)	—	—	—	—
	10^{-8}	3.71 × ($< 10^{-10}$)	1.36 × ($< 10^{-10}$)	—	—	—	—
{NO, SOR (1.5)}	h^2	7.36 × ($< 10^{-10}$)	2.04 × ($< 10^{-10}$)	—	2.98 × (10^{-5})	1.38 × (10^{-5})	—
	10^{-8}	2.40 × ($< 10^{-10}$)	1.82 × ($< 10^{-10}$)	—	1.54 × (10^{-5})	1.35 × (10^{-5})	—

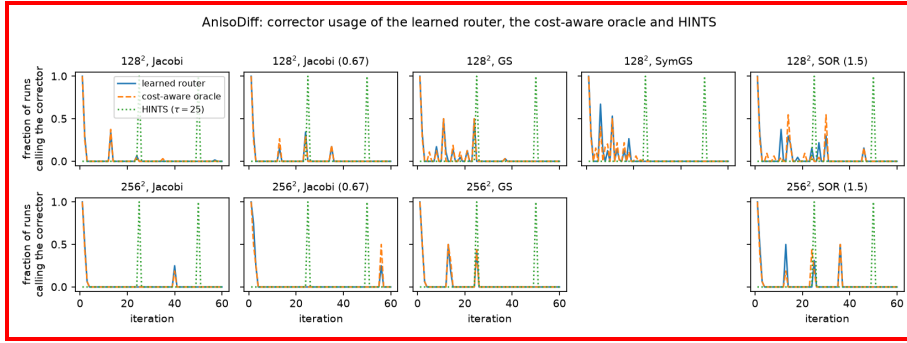


Figure 8: Anisotropic diffusion: corrector usage per iteration of the learned router, the cost-aware oracle and HINTS ($\tau=25$) for every pairing (columns) and grid (rows).

Table 19: Anisotropic diffusion: median iterations (and corrector calls) to $\varepsilon = h^2$.

Pairing	128 × 128			256 × 256		
	HINTS-25	oracle	router	HINTS-25	oracle	router
{NO, Jacobi}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)
{NO, Jacobi (0.67)}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)
{NO, GS}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)
{NO, SymGS}	25 (1)	1 (1)	1 (1)	—	—	—
{NO, SOR (1.5)}	25 (1)	1 (1)	1 (1)	25 (1)	1 (1)	1 (1)

Table 20: Anisotropic diffusion: measured per-iteration costs and macro-action sizes.

Pairing	128 × 128			256 × 256		
	c_{solver}	c_{NO}	m_{solver}	c_{solver}	c_{NO}	m_{solver}
{NO, Jacobi}	63 μs	695 μs	11	235 μs	8.89 ms	38
{NO, Jacobi (0.67)}	64 μs	692 μs	11	179 μs	9.61 ms	54
{NO, GS}	215 μs	711 μs	3	826 μs	8.91 ms	11
{NO, SymGS}	382 μs	711 μs	2	—	—	—
{NO, SOR (1.5)}	219 μs	730 μs	3	777 μs	8.80 ms	11

Table 21: Anisotropic diffusion, 128×128 : iteration-based metrics (conventions as in Table 9).

Method	$\ e_h^{(T)}\ /\ u_h\ $	AUC	p
{NO, Jacobi}			
Solver only	9.96e-01 (1.5e-04)	2.99e+02 (2.3e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	7.65e-07 (2.1e-07)	2.40e+01 (1.7e-04)	$< 10^{-10}$
Learned router (ours)	7.45e-07 (2.0e-07)	6.80e-04 (1.8e-04)	-
Cost-aware oracle	7.45e-07 (2.0e-07)	6.80e-04 (1.8e-04)	-
{NO, Jacobi (0.67)}			
Solver only	9.97e-01 (1.0e-04)	3.00e+02 (1.6e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	3.34e-10 (4.6e-11)	2.40e+01 (1.2e-04)	$< 10^{-10}$
Learned router (ours)	8.97e-10 (7.4e-12)	2.82e-04 (6.6e-05)	-
Cost-aware oracle	8.96e-10 (7.4e-12)	2.82e-04 (6.6e-05)	-
{NO, GS}			
Solver only	9.92e-01 (2.7e-04)	2.99e+02 (4.3e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	9.79e-10 (1.4e-12)	2.40e+01 (3.4e-04)	$< 10^{-10}$
Learned router (ours)	8.15e-10 (1.3e-11)	1.95e-04 (4.6e-05)	-
Cost-aware oracle	8.02e-10 (1.5e-11)	1.95e-04 (4.6e-05)	-
{NO, SymGS}			
Solver only	9.84e-01 (4.8e-04)	2.98e+02 (7.7e-02)	$< 10^{-10}$
HINTS ($\tau=25$)	7.48e-10 (3.2e-11)	2.40e+01 (6.5e-04)	$< 10^{-10}$
Learned router (ours)	7.53e-10 (2.2e-11)	1.23e-04 (2.9e-05)	-
Cost-aware oracle	7.54e-10 (1.8e-11)	1.23e-04 (2.9e-05)	-
{NO, SOR (1.5)}			
Solver only	9.76e-01 (6.8e-04)	2.96e+02 (1.1e-01)	$< 10^{-10}$
HINTS ($\tau=25$)	9.09e-10 (1.6e-11)	2.40e+01 (9.5e-04)	$< 10^{-10}$
Learned router (ours)	8.67e-10 (9.3e-12)	2.87e-04 (6.9e-05)	-
Cost-aware oracle	8.63e-10 (9.4e-12)	2.87e-04 (6.9e-05)	-

Table 22: Anisotropic diffusion, 128×128 : five-seed router-training trials (conventions as in Table 10).

Pairing	time to h^2 over 5 seeds	identical decisions	speedup vs. HINTS-25	seeds with $p < 0.01$
{NO, Jacobi}	0.76 $\pm$ 0.00 ms	100%	2.97–2.97 $\times$	5/5
{NO, Jacobi (0.67)}	0.76 $\pm$ 0.00 ms	100%	3.01–3.01 $\times$	5/5
{NO, GS}	0.78 $\pm$ 0.00 ms	100%	7.63–7.63 $\times$	5/5
{NO, SymGS}	0.78 $\pm$ 0.00 ms	100%	12.77–12.77 $\times$	5/5
{NO, SOR (1.5)}	0.80 $\pm$ 0.00 ms	100%	7.58–7.58 $\times$	5/5

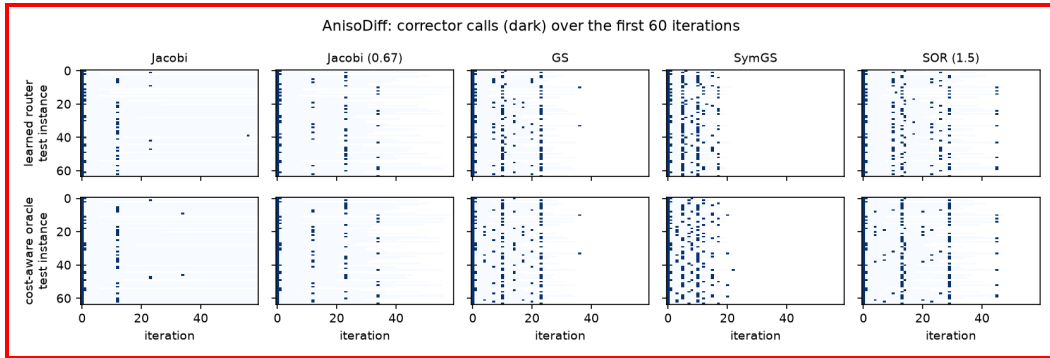


Figure 9: Anisotropic diffusion, 128×128 : routing decisions (dark: corrector call) of the learned router (top row) and the cost-aware oracle (bottom row) over the first 60 iterations of every test instance.

1000 E.5 Solver ensembles

1001 **When can a second classical solver help?** The cost-aware oracle is a per-decision comparison of
1002 macro-actions, so a router over $\text{NO} \cup \mathcal{W}$ can only beat the best pairwise router of its members if,

1003 along the way to the tolerance, the error passes through regimes in which *different* members give the
 1004 largest reduction per unit cost. We screened this ceiling exhaustively before training any router: for
 1005 every subset $\mathcal{W}$ of up to three members of {Jacobi, Jacobi (0.67), GS, SymGS, SOR (1.5), multigrid}
 1006 (multigrid on the isotropic equations only), every equation and grid, we ran the cost-aware oracle over
 1007 $\text{NO} \cup \mathcal{W}$ on the test instances and compared its work-unit time to that of the oracle over each member
 1008 alone (Table 23). Two facts emerge. At $\varepsilon = h^2$ no ensemble beats the best single solver anywhere:
 1009 one corrector call followed by one or two macro-actions of the cheapest smoother already reaches
 1010 truncation-level accuracy, so there is no second regime to exploit. At $\varepsilon = 10^{-8}$, after the corrector
 1011 has removed the smooth band, the smoother has to damp the whole band $|k| > 31$, whose sub-bands
 1012 favour different solvers: undamped Jacobi is the cheapest per unit cost on the modes near the band
 1013 edge (it contracts them by $\cos(2\pi \cdot 32/N)$ -like factors that damped Jacobi cannot match), but it does
 1014 not damp the near-Nyquist modes at all, which damped Jacobi and GS remove in a few sweeps. The
 1015 oracle over {Jacobi, Jacobi (0.67)}—a *nonstationary* Jacobi iteration whose damping parameter is
 1016 chosen per macro-action—is therefore faster than every single solver, by 13–28% on four of the six
 1017 (equation, grid) settings at 128^2 and 256^2 , and adding a third member (GS) changes nothing, i.e., the
 1018 gain saturates. At 512^2 multigrid dominates every subset (its cycle removes all sub-bands at once), so
 1019 the oracle over any ensemble containing it equals the oracle over {NO, multigrid}: adding members
 1020 neither helps nor hurts. The “difficulty” that matters for ensembles is thus not the equation but the
 1021 width of the spectrum left to the classical members after the corrector, which grows with the grid
 1022 (fixed band) and with the tolerance.

1023 **Nested ensembles.** Table 24 evaluates the learned router over the nested sets {Jacobi},
 1024 {Jacobi (0.67)}, {Jacobi, Jacobi (0.67)} and {Jacobi, Jacobi (0.67), GS} on every equation at 128^2
 1025 and 256^2 ; the single-member routers are the pairwise routers of Sections E.2 to E.4, evaluated in the
 1026 same session on the same instances, and all members keep the macro-action sizes of their pairwise
 1027 runs. The table reports live and work-unit times to h^2 , 10^{-6} and 10^{-8} for the router and its oracle,
 1028 and the paired speedup at 10^{-8} over the best single-member router (and of the ensemble oracle over
 1029 the best single-member oracle) with its p -value; work units are the primary measure, since they are
 1030 determined by the decision sequence alone. Three things are visible. *Strict separation:* at $\varepsilon = 10^{-8}$
 1031 the ensemble router is significantly faster than every single-member router in 7 of the 12 ensemble
 1032 cells (six settings times two nested sets; $1.08\times$ – $1.42\times$ over the best single solver, paired Wilcoxon
 1033 $p < 0.01$), with the largest gains on the 256^2 grids, where the band left to the smoothers is widest; it
 1034 reproduces its oracle to within 1–2% in every cell, so the gain is the oracle’s nonstationary damp-
 1035 ing schedule—undamped Jacobi macro-actions while the band-edge modes dominate the residual,
 1036 damped Jacobi once the near-Nyquist modes do—which the router detects from the decay rate of
 1037 the residual over the last macro-action. *Saturation:* adding GS changes neither the oracle’s nor
 1038 the router’s schedule beyond a few percent (the three-member cells match the two-member ones),
 1039 because GS never wins a cost-aware comparison against the Jacobi pair. *Monotonicity, and its one*
 1040 *caveat:* at h^2 every ensemble router is within a few percent of the best single-member router (paired
 1041 ratios $0.96\times$ – $1.02\times$ at h^2 and $0.92\times$ and above at 10^{-8}), and at 10^{-8} it is never significantly slower
 1042 except on Poisson at 128^2 , where the two-member set gains nothing and the three-member router is
 1043 8% slower than the damped-Jacobi router. That exception is a property of the greedy rule, not of the
 1044 learned router: the cost-aware oracle over {Jacobi, Jacobi (0.67)} is itself 10% slower to 10^{-8} than
 1045 the oracle over {Jacobi (0.67)} alone (2.56 against 2.31 work-unit ms), because the larger immediate
 1046 reduction of an undamped-Jacobi macro-action leaves a near-Nyquist remainder that costs more
 1047 later—the myopia that Theorem 4.1 bounds—and the router imitates that oracle faithfully. A greedy
 1048 rule is therefore not monotone in its action set in general; in our settings the losses stay within a few
 1049 percent, whereas the gains reach $1.42\times$.

Table 23: Oracle-level screening of ensembles: for every equation and grid, the fastest single solver and the fastest ensemble of up to three members under the cost-aware oracle (median work-unit time over 16 test instances), at $\varepsilon = h^2$ and 10^{-8} ; the ratio is the speedup of the ensemble over the best single solver, and the last column is the ratio for $\{\text{Jacobi}, \text{Jacobi} (0.67)\}$.

Equation	N	$\varepsilon = h^2$			$\varepsilon = 10^{-8}$		
		best single	best ensemble	ratio	best single	best ensemble	ratio
Poisson	128 ²	Jacobi (772 μ s)	$\{\text{Jacobi}, \text{GS}\}$ (772 μ s)	1.00 $\times$	Jacobi (0.67) (2.51 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (2.45 ms)	1.02 $\times$
	256 ²	Jacobi (2.86 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (2.86 ms)	1.00 $\times$	Jacobi (0.67) (14.2 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (11.1 ms)	1.28 $\times$
	512 ²	Multigrid (36.5 ms)	$\{\text{Jacobi}, \text{Multigrid}\}$ (36.5 ms)	1.00 $\times$	Multigrid (90.8 ms)	$\{\text{Jacobi}, \text{Multigrid}\}$ (90.8 ms)	1.00 $\times$
Convection-diffusion	128 ²	Jacobi (923 μ s)	$\{\text{Jacobi}, \text{GS}\}$ (923 μ s)	1.00 $\times$	Jacobi (0.67) (3.06 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (2.71 ms)	1.13 $\times$
	256 ²	Jacobi (5.49 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (5.49 ms)	1.00 $\times$	Multigrid (24.1 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (19.3 ms)	1.25 $\times$
	512 ²	Multigrid (54.3 ms)	$\{\text{Jacobi}, \text{Multigrid}\}$ (54.3 ms)	1.00 $\times$	Multigrid (103.7 ms)	$\{\text{Jacobi}, \text{Multigrid}\}$ (103.7 ms)	1.00 $\times$
Anisotropic diffusion	128 ²	Jacobi (765 μ s)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (765 μ s)	1.00 $\times$	Jacobi (0.67) (3.58 ms)	$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$ (3.06 ms)	1.17 $\times$

Table 24: Nested ensembles: median time to $\varepsilon \in \{h^2, 10^{-6}, 10^{-8}\}$ of the learned router (live and work units) and of the cost-aware oracle (work units) over $\text{NO} \cup \mathcal{W}$, for the single members and the nested two- and three-member sets, all evaluated in one session on the same instances (64 at 128², 32 at 256², 16 for anisotropic diffusion at 256²). The last columns give the paired speedup at 10^{-8} over the best single-member router (resp. oracle) with the one-sided Wilcoxon p -value; bold: $\geq 1.1 \times$ with $p < 0.01$.

Equation	N	$\mathcal{W}$	$\varepsilon = h^2$			$\varepsilon = 10^{-6}$			$\varepsilon = 10^{-8}$			$\varepsilon = 10^{-8}$: vs. best single solver	
			router	router (WU)	oracle (WU)	router	router (WU)	oracle (WU)	router	router (WU)	oracle (WU)	router	oracle
Poisson	128 ²	$\{\text{Jacobi}\}$	841 μ s	742 μ s	732 μ s	2.72 ms	2.21 ms	2.19 ms	141.2 ms ¹⁸	116.1 ms ¹⁸	115.2 ms	—	—
		$\{\text{Jacobi}(0.67)\}$	955 μ s	747 μ s	737 μ s	1.45 ms	1.16 ms	1.15 ms	2.82 ms	2.33 ms	2.31 ms	(best single)	—
		$\{\text{GS}\}$	1.02 ms	901 μ s	891 μ s	2.68 ms	2.56 ms	2.53 ms	4.56 ms	4.41 ms	4.37 ms	—	—
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	929 μ s	746 μ s	735 μ s	1.40 ms	1.16 ms	1.15 ms	2.96 ms	2.58 ms	2.56 ms	1.00 $\times$ (0.907)	1.00 $\times$ (10^{-6})
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	888 μ s	748 μ s	738 μ s	1.44 ms	1.21 ms	1.15 ms	2.97 ms	2.59 ms	2.57 ms	0.92 $\times$ (slower, 10^{-7})	1.00 $\times$ (10^{-6})
Poisson	256 ²	$\{\text{Jacobi}\}$	2.35 ms	2.00 ms	1.98 ms	5.42 ms ¹⁵	4.51 ms ¹⁵	4.49 ms	72.1 ms ¹⁴	61.3 ms ¹⁴	61.0 ms	—	—
		$\{\text{Jacobi}(0.67)\}$	2.95 ms	2.24 ms	2.23 ms	7.51 ms	5.72 ms	5.69 ms	16.8 ms	12.8 ms	12.7 ms	(best single)	—
		$\{\text{GS}\}$	4.00 ms	3.52 ms	3.50 ms	11.9 ms	11.5 ms	10.9 ms	26.3 ms	24.9 ms	24.8 ms	—	—
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	2.52 ms	1.97 ms	1.96 ms	5.77 ms	4.49 ms	4.46 ms	12.7 ms	10.1 ms	10.1 ms	1.26 $\times$ (10^{-7})	1.26 $\times$ (10^{-7})
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	2.43 ms	1.98 ms	1.97 ms	5.06 ms	4.50 ms	4.47 ms	11.7 ms	10.2 ms	10.1 ms	1.26 $\times$ (10^{-7})	1.26 $\times$ (10^{-7})
Convection-diffusion	128 ²	$\{\text{Jacobi}\}$	1.01 ms	842 μ s	832 μ s	7.25 ms	6.00 ms	5.95 ms	268.3 ms ¹⁵	213.8 ms ¹⁵	212.2 ms	—	—
		$\{\text{Jacobi}(0.67)\}$	1.22 ms	913 μ s	902 μ s	2.04 ms	1.52 ms	1.51 ms	3.67 ms	2.94 ms	2.92 ms	(best single)	—
		$\{\text{GS}\}$	1.27 ms	1.11 ms	1.10 ms	3.22 ms	2.85 ms	2.82 ms	5.39 ms	4.94 ms	4.90 ms	—	—
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	1.01 ms	879 μ s	903 μ s	1.87 ms	1.49 ms	1.47 ms	3.24 ms	2.70 ms	2.67 ms	1.08 $\times$ ($< 10^{-10}$)	1.11 $\times$ ($< 10^{-10}$)
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	1.07 ms	886 μ s	911 μ s	1.99 ms	1.60 ms	1.48 ms	3.52 ms	2.85 ms	2.68 ms	0.99 $\times$ (slower, 0.155)	1.10 $\times$ (10^{-10})
Convection-diffusion	256 ²	$\{\text{Jacobi}\}$	5.03 ms	3.99 ms	3.97 ms	10.4 ms ¹⁶	8.58 ms ¹⁶	8.54 ms	518.9 ms ¹⁵	410.7 ms ¹⁵	407.7 ms	—	—
		$\{\text{Jacobi}(0.67)\}$	6.63 ms	5.48 ms	5.45 ms	14.8 ms	12.3 ms	12.2 ms	29.4 ms	25.3 ms	25.2 ms	(best single)	—
		$\{\text{GS}\}$	8.43 ms	6.51 ms	6.47 ms	18.8 ms	14.9 ms	14.7 ms	35.6 ms	29.3 ms	28.5 ms	—	—
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	4.39 ms	4.00 ms	3.98 ms	9.47 ms	8.59 ms	8.55 ms	19.9 ms	17.8 ms	17.7 ms	1.42 $\times$ (10^{-7})	1.42 $\times$ (10^{-7})
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	5.00 ms	3.98 ms	3.96 ms	10.7 ms	8.57 ms	8.53 ms	21.6 ms	17.7 ms	17.7 ms	1.42 $\times$ (10^{-7})	1.42 $\times$ (10^{-7})
Anisotropic diffusion	128 ²	$\{\text{Jacobi}\}$	849 μ s	762 μ s	752 μ s	1.51 ms	1.36 ms	1.19 ms	48.8 ms ¹⁰	41.2 ms ¹⁰	40.9 ms	—	—
		$\{\text{Jacobi}(0.67)\}$	906 μ s	760 μ s	750 μ s	1.65 ms	1.46 ms	1.42 ms	3.85 ms	3.26 ms	3.23 ms	(best single)	—
		$\{\text{GS}\}$	836 μ s	779 μ s	769 μ s	2.15 ms	1.97 ms	1.95 ms	5.52 ms	5.08 ms	5.04 ms	—	—
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	855 μ s	762 μ s	751 μ s	1.49 ms	1.40 ms	1.19 ms	3.32 ms	2.87 ms	2.84 ms	1.14 $\times$ ($< 10^{-10}$)	1.18 $\times$ ($< 10^{-10}$)
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	841 μ s	768 μ s	757 μ s	1.47 ms	1.21 ms	1.20 ms	3.29 ms	2.88 ms	2.85 ms	1.15 $\times$ ($< 10^{-10}$)	1.16 $\times$ ($< 10^{-10}$)
Anisotropic diffusion	256 ²	$\{\text{Jacobi}\}$	9.30 ms	9.11 ms	9.10 ms	21.9 ms ¹¹	22.6 ms ¹¹	22.2 ms	62.8 ms ¹⁷	66.3 ms ¹⁷	66.2 ms	—	—
		$\{\text{Jacobi}(0.67)\}$	9.31 ms	9.78 ms	9.77 ms	18.3 ms	19.4 ms	19.0 ms	26.1 ms	25.9 ms	29.0 ms	(best single)	—
		$\{\text{GS}\}$	9.20 ms	9.13 ms	9.12 ms	18.5 ms	18.5 ms	18.4 ms	34.8 ms	34.6 ms	34.5 ms	—	—
		$\{\text{Jacobi}, \text{Jacobi}(0.67)\}$	9.99 ms	9.45 ms	9.44 ms	19.6 ms	18.8 ms	18.7 ms	27.2 ms	25.0 ms	28.0 ms	1.03 $\times$ (0.302)	1.04 $\times$ (10^{-4})
		$\{\text{Jacobi}, \text{Jacobi}(0.67), \text{GS}\}$	9.20 ms	9.34 ms	9.33 ms	18.5 ms	18.5 ms	18.5 ms	29.5 ms	27.6 ms	27.6 ms	0.97 $\times$ (slower, 0.117)	1.05 $\times$ (10^{-4})

Larger ensembles at truncation level. Table 25 repeats the ensembles of Section 7 ($\{\text{Jacobi}, \text{GS}\}$ up to all five solvers) at 128² and $\varepsilon = h^2$, where the screening predicts no gain: the ensemble router matches the best pairwise router of its members to within about $\pm 20\%$ in live time (ratio of medians $0.82 \times$ – $1.14 \times$ over the 12 cells) and within 1.5% in work units, without knowing in advance which member is fastest, and is $380 \times$ – $> 10^3 \times$ faster than the fastest member alone; Table 26 lists how its iterations are distributed over the members. Its oracle is up to 25% faster in a few cells because, at h^2 , several cost-equalised smoothers are near-ties and the oracle’s choice among them depends on the spectral content of the current error, which the router’s residual-norm features reflect only indirectly (agreement with the oracle 55–83% on its own rollouts against 88–99% for pairwise routers). More capacity or imitation data does not change this: routers with twice the hidden width, three times the DAgger rounds and twice the oracle rollouts (Table 28) take the same number of iterations and corrector calls to h^2 as the default routers on 53%–91% of the test instances and are significantly faster in 2 and significantly slower in 3 of the 8 cells ($p < 0.05$); the residual disagreements are near-ties whose resolution does not affect the time to h^2 . Deciding *when* to call the corrector—the decision that dominates the wall-clock gains—is learned equally well in the ensembles.

Table 25: Ensembles of Section 7 at 128×128 and $\varepsilon = h^2$: median live time of the best member solver alone, of the best pairwise router among the members (paired speedup of the ensemble router over it in parentheses), and of the router and the cost-aware oracle over $\text{NO} \cup \mathcal{W}$ (64 instances; pairwise baselines from the pairwise runs on the same instances).

Equation	$\mathcal{W}$	Best solver only	Best pairwise router	Router($\text{NO} \cup \mathcal{W}$)	Oracle($\text{NO} \cup \mathcal{W}$)
Poisson	$\{Jacobi, GS\}$	1.15 s (Jacobi)	987 μs (Jacobi)	964 μs (1.09 $\times$ vs pairwise)	834 μs
	$\{Jacobi, GS, \text{SymGS}\}$	1.15 s (Jacobi)	987 μs (Jacobi)	951 μs (1.07 $\times$ vs pairwise)	854 μs
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	1.15 s (Jacobi)	912 μs (Jacobi (0.67))	868 μs (0.98 $\times$ vs pairwise)	842 μs
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	600.9 ms (SOR (1.5))	912 μs (Jacobi (0.67))	1.12 ms (0.86 $\times$ vs pairwise)	900 μs
ConvDiff	$\{Jacobi, GS\}$	1.51 s (Jacobi)	1.14 ms (Jacobi)	1.25 ms (1.09 $\times$ vs pairwise)	974 μs
	$\{Jacobi, GS, \text{SymGS}\}$	1.51 s (Jacobi)	1.14 ms (Jacobi)	1.01 ms (1.16 $\times$ vs pairwise)	996 μs
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	1.51 s (Jacobi)	1.10 ms (Jacobi (0.67))	996 μs (1.00 $\times$ vs pairwise)	1.07 ms
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	383.6 ms (SOR (1.5))	1.10 ms (Jacobi (0.67))	1.01 ms (1.06 $\times$ vs pairwise)	1.08 ms
AnisoDiff	$\{Jacobi, GS\}$	>4.62 s [†] (none)	852 μs (GS)	898 μs (1.01 $\times$ vs pairwise)	870 μs
	$\{Jacobi, GS, \text{SymGS}\}$	>4.62 s [†] (none)	852 μs (GS)	880 μs (1.03 $\times$ vs pairwise)	857 μs
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	>4.52 s [†] (none)	850 μs (Jacobi (0.67))	880 μs (1.00 $\times$ vs pairwise)	858 μs
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	>4.52 s [†] (none)	850 μs (Jacobi (0.67))	890 μs (1.00 $\times$ vs pairwise)	863 μs

Table 26: Ensembles at 128×128 : mean (standard deviation over instances) fraction of the iterations up to the h^2 crossing that the ensemble router spends in each member and in the corrector.

Equation	$\mathcal{W}$	Jacobi	GS	SymGS	Jacobi (0.67)	SOR (1.5)	DeepONet
Poisson	$\{Jacobi, GS\}$	0.228 (0.365)	0.144 (0.216)	-	-	-	0.628 (0.386)
	$\{Jacobi, GS, \text{SymGS}\}$	0.237 (0.337)	0.043 (0.134)	0.086 (0.165)	-	-	0.634 (0.378)
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	0.235 (0.297)	0.008 (0.062)	0.101 (0.207)	0.000 (0.000)	-	0.656 (0.352)
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	0.094 (0.195)	0.020 (0.051)	0.182 (0.265)	0.046 (0.125)	0.007 (0.023)	0.650 (0.366)
ConvDiff	$\{Jacobi, GS\}$	0.089 (0.255)	0.310 (0.337)	-	-	-	0.600 (0.373)
	$\{Jacobi, GS, \text{SymGS}\}$	0.279 (0.354)	0.067 (0.156)	0.055 (0.103)	-	-	0.599 (0.374)
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	0.305 (0.340)	0.000 (0.000)	0.022 (0.038)	0.110 (0.214)	-	0.563 (0.410)
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	0.166 (0.302)	0.028 (0.127)	0.000 (0.000)	0.233 (0.375)	0.000 (0.000)	0.573 (0.399)
AnisoDiff	$\{Jacobi, GS\}$	0.173 (0.320)	0.000 (0.000)	-	-	-	0.827 (0.320)
	$\{Jacobi, GS, \text{SymGS}\}$	0.052 (0.153)	0.000 (0.000)	0.097 (0.229)	-	-	0.851 (0.273)
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.185 (0.339)	-	0.815 (0.339)
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.185 (0.339)	0.000 (0.000)	0.815 (0.339)

Table 27: Ensembles at 128×128 : one-sided Wilcoxon p -values at $\varepsilon = h^2$ for the ensemble router against the fastest member solver alone (censored runs at their lower bound), against the best pairwise router in both directions, and against the cost-aware oracle over the same ensemble (alternative: the ensemble router is slower).

Equation	$\mathcal{W}$	vs. best solver only	vs. best pairwise router (faster)	vs. best pairwise router (slower)	vs. oracle($\text{NO} \cup \mathcal{W}$) (slower)
Poisson	$\{Jacobi, GS\}$	$< 10^{-10}$	10^{-7}	1.000	0.040
	$\{Jacobi, GS, \text{SymGS}\}$	$< 10^{-10}$	0.001	0.999	0.001
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	$< 10^{-10}$	0.856	0.144	10^{-4}
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	$< 10^{-10}$	1.000	10^{-8}	10^{-8}
ConvDiff	$\{Jacobi, GS\}$	$< 10^{-10}$	10^{-4}	1.000	10^{-5}
	$\{Jacobi, GS, \text{SymGS}\}$	$< 10^{-10}$	10^{-10}	1.000	0.492
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	$< 10^{-10}$	0.783	0.217	0.001
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	$< 10^{-10}$	0.047	0.953	0.566
AnisoDiff	$\{Jacobi, GS\}$	$< 10^{-10}$	0.315	0.685	0.014
	$\{Jacobi, GS, \text{SymGS}\}$	$< 10^{-10}$	0.055	0.945	10^{-4}
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	$< 10^{-10}$	0.551	0.449	0.064
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	$< 10^{-10}$	0.633	0.367	0.012

Table 28: Control on router capacity for the ensembles (128×128 , 64 instances): the default ensemble router (hidden width 64, 2 DAgger rounds, 128 oracle rollouts) against one with hidden width 128, 6 DAgger rounds and 256 rollouts. Times to h^2 are median work units net of the per-decision cost; “same decisions” is the share of instances on which both routers take the same number of iterations and corrector calls to h^2 ; p is the one-sided Wilcoxon p -value on the paired net times; “agreement” is each router’s agreement with the oracle on its own rollouts during training.

Equation	$\mathcal{W}$	default router	larger router	oracle	larger / default (medians)	same decisions	p (larger faster / slower)	agreement (default / larger)
Poisson	$\{Jacobi, GS\}$	858 μs	758 μs	758 μs	0.884 $\times$	84%	0.933 / 0.067	80% / 89%
	$\{Jacobi, GS, \text{SymGS}\}$	742 μs	742 μs	742 μs	1.000 $\times$	89%	0.525 / 0.494	73% / 73%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	740 μs	740 μs	740 μs	1.000 $\times$	61%	0.958 / 0.042	55% / 76%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	732 μs	732 μs	732 μs	1.000 $\times$	62%	10^{-4} / 0.999	65% / 64%
ConvDiff	$\{Jacobi, GS\}$	1.07 ms	887 μs	854 μs	0.829 $\times$	69%	0.005 / 0.995	83% / 86%
	$\{Jacobi, GS, \text{SymGS}\}$	854 μs	890 μs	856 μs	1.042 $\times$	91%	0.995 / 0.005	58% / 65%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67)\}$	888 μs	888 μs	932 μs	1.000 $\times$	56%	0.459 / 0.541	57% / 70%
	$\{Jacobi, GS, \text{SymGS}, Jacobi(0.67), \text{SOR}(1.5)\}$	892 μs	892 μs	936 μs	1.000 $\times$	53%	$1.000 / 10^{-8}$	65% / 74%

1065 E.6 Overheads and amortisation

1066 Table 29 lists the measured cost of every operation as a function of N , including one decision of
1067 the LSTM router of Section 7 (full-state input of size $4N^2$, hidden size 256, 4 layers): at 128^2 one

1068 LSTM decision costs 2.4 ms, about 46 Jacobi sweeps, which is why the wall-clock study uses the
 1069 scalar-feature router ($\approx 5 \mu\text{s}$, independent of N). The learned router’s decisions therefore account
 1070 for well under 1% of its solve time, and its corrector calls for 50–70% of the time to h^2 (one or two
 1071 calls). Table 30 accounts for training: the corrector (shared with HINTS) is fit in minutes, the router
 1072 in seconds to a few minutes per pairing, and the break-even number of solves after which the router’s
 1073 own training cost is recovered relative to HINTS ($\tau=25$) with the same corrector is in the thousands
 1074 at 128^2 and in the hundreds on the larger grids.

Table 29: Measured single-thread cost of one operation as a function of the grid size (Poisson; median of repeated calls). Multigrid uses a direct solve below 64^2 and is not listed there.

Operation	$N = 32$	$N = 64$	$N = 128$	$N = 256$	$N = 512$
Jacobi iteration	$14 \mu\text{s}$	$25 \mu\text{s}$	$53 \mu\text{s}$	$191 \mu\text{s}$	1.07 ms
GS iteration	$25 \mu\text{s}$	$64 \mu\text{s}$	$209 \mu\text{s}$	$820 \mu\text{s}$	3.74 ms
SymGS iteration	$38 \mu\text{s}$	$109 \mu\text{s}$	$378 \mu\text{s}$	1.48 ms	6.85 ms
Multigrid V(2,2) cycle	–	$390 \mu\text{s}$	1.35 ms	5.27 ms	25.35 ms
DeepONet corrector call	–	–	$663 \mu\text{s}$	1.13 ms	3.62 ms
Router decision (ours)	$5 \mu\text{s}$	$5 \mu\text{s}$	$5 \mu\text{s}$	$5 \mu\text{s}$	$5 \mu\text{s}$
LSTM router decision (Section 7)	$236 \mu\text{s}$	$696 \mu\text{s}$	2.43 ms	9.55 ms	37.46 ms

Table 30: Training-cost amortisation for the GS pairing: corrector training (data generation and least-squares fit, CPU), router training (oracle rollouts and DAGger rounds), median wall-clock saved per solve at $\varepsilon = h^2$ relative to HINTS ($\tau=25$) and to the solver alone, and the number of solves after which the router’s training cost is recovered relative to HINTS.

Equation	N	Corrector data + fit	Router training (GS pairing)	Saving per solve vs HINTS ($\tau=25$)	Break-even solves (router)	Saving per solve vs solver only
Poisson	128^2	82 s	8 s	5.64 ms	1419	1.83 s
	256^2	98 s	34 s	18.1 ms	1883	28.82 s
	512^2	504 s	229 s	11.4 ms	20139	–
Convection–diffusion	128^2	79 s	8 s	6.92 ms	1157	1.63 s
	256^2	106 s	33 s	24.0 ms	1376	30.05 s
	512^2	493 s	254 s	34.2 ms	7423	–

1075 E.7 Held-out confirmation and one-shot ablation

1076 Two configuration choices of this study were made after looking at results on the test instances: the
 1077 router of the convection–diffusion Jacobi pairing at 512^2 was retrained with more rollouts after the
 1078 first router missed h^2 on 4 of 16 instances (Section E.1), and the imitation budget of the ensemble
 1079 routers was raised after the pairwise recipe imitated its oracle imperfectly on one cell (Section E.5).
 1080 The optimistic HINTS baseline is tuned on the test set by design and labelled as such. To make
 1081 the reported numbers confirmatory, Table 31 re-evaluates every pairwise cell at 128^2 and 256^2 , the
 1082 retrained 512^2 cell and all nested ensembles—with the configurations frozen—on a fresh set of test
 1083 instances (GRF seed 73, never used for any decision) with three timed replays per instance, and
 1084 reports the main-test and held-out speedups side by side. Tables 5, 6, 11, 12, 17 and 18 additionally
 1085 include the one-shot schedule (corrector once at iteration 0, then the solver) as the static ablation of
 1086 state-dependent routing: at h^2 it is as fast as the router wherever a single corrector call suffices, and
 1087 the router’s advantage over it grows at tighter tolerances and on the pairings that need a second call.

Table 31: Held-out confirmation: paired median speedups of the learned router on the main test set (seed 72, one timed replay) and on a fresh test set (seed 73, three timed replays) with frozen configurations; pairwise cells at $\varepsilon = h^2$ against HINTS ($\tau=25$) and the best fixed period, nested ensembles at 10^{-8} in work units against the best single-member router and against their oracle; one-sided Wilcoxon p -values in parentheses.

(results pending)

E.8 Verification of the theoretical assumptions

Theorem 4.1 and Propositions 4.2–4.3 assume that every error propagation map $G_j = I - C_j \circ \mathcal{L}_h^a$ is Lipschitz with constant $\rho_j < 1$ (respectively ≤ 1 with a shared eigenbasis), zero-preserving, and—for postfix monotonicity—invertible; Theorem 5.1 assumes that the per-step errors $\tilde{c}_j$ are bounded above and that at least one solver cannot annihilate the error in one step. Table 32 verifies these conditions numerically for every operation of the study on the mean-free subspace (the constant mode is the null space of the periodic operator and is excluded from the data and from every iteration): the Euclidean Lipschitz constant $\|G_j\|_2$ (power iteration on $G_j^T G_j$; exact from the Fourier symbol for Jacobi), the Lipschitz constant $\|G_j\|_A = \|A_s^{1/2} G_j A_s^{-1/2}\|_2$ in the energy norm of the symmetric part A_s of the discrete operator, the same for the macro-action $G_j^{m_j}$, the spectral radius, the smallest observed contraction $\sigma_{\min} = \min \|G_j x\|/\|x\|$, $\|C_j(0)\|$, and the commutator $\|G_j G_{\text{NO}} - G_{\text{NO}} G_j\|$ on random vectors. Table 33 evaluates the quantities of Theorem 4.1 and Theorem 5.1 along the cost-aware oracle’s own path to $\varepsilon = 10^{-8}$ (a horizon of tens of macro-actions; to h^2 the path has one or two steps).

What holds, and in which norm. (i) Every classical solver is zero-preserving exactly and contracts in the sense that matters for convergence: the spectral radius of G_j is below 1 in every setting (at most 0.999997), and so is the energy-norm Lipschitz constant of the solver and of its macro-action (at most 0.999912). The *Euclidean* Lipschitz constant, however, exceeds 1 for Gauss–Seidel, SOR and symmetric GS (up to 1.89 for SOR), whose iteration matrices are non-normal; the proof of Proposition 4.2 uses only the triangle inequality and the composition rule for Lipschitz constants, so it holds verbatim with $\|\cdot\|_2$ replaced by the energy norm, in which the condition is satisfied by all classical solvers in all settings—but that statement concerns the greedy rule and objective defined with the energy norm, whereas the oracle and the router of this study minimise the Euclidean error. For the evaluated Euclidean rule, the strict-contraction premise of Proposition 4.2 therefore holds for the Jacobi-type solvers and fails for the non-normal sweeps, whose Euclidean expansion is confined to a single sweep (their spectral radii and macro-action contraction show the iteration is convergent). For (damped) Jacobi both constants agree (the maps are Fourier-diagonal); the values are close to 1 because the slowest modes (the smoothest ones, and the near-Nyquist mode for undamped Jacobi) are barely damped by a point smoother—which is exactly why the corrector is needed. (ii) The corrector’s map is zero-preserving, contracts every band mode to 1.7×10^{-3} or better (on average to 10^{-4} – 10^{-6}) and leaves the modes outside the band untouched, so its Lipschitz constant is exactly 1 (not < 1): Theorem 4.2 in its strict form does not apply to the corrector, but Theorem 4.3 does, and its second condition—a shared orthogonal eigenbasis—holds exactly for the exact band-limited inverse together with (damped) Jacobi, all of which are diagonal in the Fourier basis, and holds for the learned corrector up to its fit error (measured commutators $\approx 10^{-6}$, at the corrector’s single-precision accuracy; replacing the learned corrector by the exact inverse does not change the oracle’s decisions, Section E.1), so for the ensembles $\{\text{NO}, \text{Jacobi}\}$, $\{\text{NO}, \text{Jacobi} (0.67)\}$ and $\{\text{NO}, \text{Jacobi}, \text{Jacobi} (0.67)\}$ the objective is supermodular and postfix monotone up to that error and Theorem 4.1 applies with $\alpha = 1$; for GS, SOR, symmetric GS and multigrid the commutators are small but nonzero (10^{-3} – 10^{-1}) and no guarantee is claimed. (iii) Invertibility fails for most maps in exact arithmetic: a Gauss–Seidel, SOR or symmetric-GS sweep annihilates an error concentrated at the first grid point ($\sigma_{\min} = 0$), undamped Jacobi annihilates the mode $(N/4, N/4)$, and the corrector annihilates the band; postfix monotonicity is therefore not guaranteed, which Section 4 anticipates (“our experiments show that greedy routers remain effective even when invertibility is not met”). (iv) The worst-case ratio $\alpha(O)$ of Theorem 4.2, which only uses the Lipschitz constants, is loose here: along the oracle’s path to 10^{-8} (Table 33) $\sum_i \rho_{O_i}^2$ is close to T because each ρ is dominated by the slowest mode, so the bound ranges from 1 (the multigrid pairing, whose cycle contracts everything) to 10^2 – 10^4 for the point smoothers, and is infinite on the few instances where $\sum_i \rho_{O_i}^2 \geq T$. The empirically measured supermodularity ratio of Equation (7) along the same path—with S the oracle’s prefix and S' its suffix—is at most 1.000 (median 0.44) in every setting, and at most 1.000 on the short paths to h^2 . This is a diagnostic, not a verification of the theorem’s premise: the definition quantifies over all prefixes and over the unknown optimal sequence O , whereas we evaluate it on the greedy path itself; it shows that delayed inclusions do not pay off more than immediate ones along the states the greedy rule actually visits. (v) For Theorem 5.1, the per-step errors are bounded by the current error ($\max_j \tilde{c}_j / \|e^{(t)}\|^2 \leq 1$ at every visited state) and at least one solver leaves a positive fraction of it ($\min_t \max_j \tilde{c}_j / \|e^{(t)}\|^2 > 0$), so the surrogate’s Bayes consistency applies.

Table 32: Numerical verification of the assumptions of Propositions 4.2–4.3 for every operation of the study (mean-free subspace). $\|\cdot\|_2$: Euclidean Lipschitz constant of the error propagation map; $\|\cdot\|_A$: Lipschitz constant in the energy norm of the symmetric part of $\mathcal{L}_h$, for one iteration and for the macro-action of m_j iterations; ρ : spectral radius; $\sigma_{\min}$: smallest observed contraction (a unit error at the first grid point for the triangular sweeps, the minimum over Fourier modes otherwise; for the corrector, the largest remaining fraction of a band mode); $\|C_j(0)\|$: zero preservation; last column: commutator with the corrector’s map on random unit vectors (Proposition 4.3). For (damped) Jacobi the constants are exact from the Fourier symbol (the value with the Nyquist modes included; without them undamped Jacobi has $\|G\|_2 = \rho = 0.9994$ at 128^2).

Equation	Operation	m_j	$\ I - C_j \mathcal{L}_h\ _2$	$\ I - C_j \mathcal{L}_h\ _A$	$\ (I - C_j \mathcal{L}_h)^{m_j}\ _A$	$\rho(I - C_j \mathcal{L}_h)$	$\sigma_{\min}$	$\ C_j(0)\ $	$\ [G_j, G_{\text{No}}]\ $
Poisson, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-05 (band)	0	–
	Jacobi	12	0.999398	0.999398	0.992797	0.999398	0	0	5.7e-06
	Jacobi (0.67)	12	0.999596	0.999596	0.995168	0.999596	1.4e-04	0	4.1e-06
	GS	3	1.021515	0.998595	0.996368	0.998797	0	0	3.1e-02
	SymGS	2	1.006607	0.997396	0.995187	0.997623	0	0	2.2e-02
	SOR (1.5)	3	1.449417	0.996467	0.989336	0.996411	7.2e-01	0	8.5e-02
	Multigrid	1	0.098203	0.097860	0.097860	0.096877	1.9e-03	0	8.8e-03
Poisson, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-05 (band)	0	–
	Jacobi	6	0.999849	0.999849	0.999097	0.999849	0	0	1.3e-06
	Jacobi (0.67)	6	0.999899	0.999899	0.999395	0.999899	5.6e-05	0	1.2e-06
	GS	1	1.022053	0.998925	0.998925	0.999699	0	0	1.1e-02
	SymGS	1	1.005470	0.998839	0.998839	0.999401	0	0	9.4e-03
	SOR (1.5)	1	1.451367	0.998352	0.998352	0.999098	7.2e-01	0	3.8e-02
	Multigrid	1	0.102361	0.102263	0.102263	0.101344	1.4e-03	0	4.2e-03
Poisson, 512^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	2.1e-05 (band)	0	–
	Jacobi	3	0.999962	0.999962	0.999887	0.999962	0	0	6.1e-07
	GS	1	1.022202	0.998983	0.998983	0.999925	0	0	4.5e-03
	SOR (1.5)	1	1.451920	0.999043	0.999043	0.999774	7.2e-01	0	1.7e-02
	Multigrid	1	0.103470	0.103434	0.103434	0.102511	1.0e-03	0	1.3e-03
Convection–diffusion, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.5e-05 (band)	0	–
	Jacobi	9	0.999398	0.999398	0.994593	0.999398	0	0	3.1e-06
	Jacobi (0.67)	9	0.999596	0.999596	0.996374	0.999596	1.4e-04	0	2.2e-06
	GS	3	1.025726	0.998460	0.995816	0.998589	0	0	3.1e-02
	SymGS	2	1.010112	0.997353	0.995168	0.997571	0	0	2.3e-02
	SOR (1.5)	3	1.554544	0.996981	0.983568	0.993973	7.5e-01	0	9.1e-02
	Multigrid	1	0.076967	0.074905	0.074905	0.074609	1.5e-03	0	6.9e-03
Convection–diffusion, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.1e-05 (band)	0	–
	Jacobi	4	0.999849	0.999849	0.999398	0.999849	0	0	7.6e-07
	Jacobi (0.67)	4	0.999899	0.999899	0.999596	0.999899	5.6e-05	0	6.8e-07
	GS	1	1.024025	0.998934	0.998934	0.999674	0	0	1.2e-02
	SymGS	1	1.008004	0.998846	0.998846	0.999398	0	0	9.8e-03
	SOR (1.5)	1	1.497088	0.998192	0.998192	0.998846	7.3e-01	0	4.1e-02
	Multigrid	1	0.084356	0.083809	0.083809	0.083459	1.2e-03	0	3.7e-03
Convection–diffusion, 512^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	9.3e-06 (band)	0	–
	Jacobi	3	0.999962	0.999962	0.999887	0.999962	0	0	3.4e-07
	GS	1	1.023154	0.998983	0.998983	0.999922	0	0	4.6e-03
	SOR (1.5)	1	1.473282	0.999075	0.999075	0.999745	7.3e-01	0	1.8e-02
	Multigrid	1	0.091267	0.091201	0.091201	0.090840	9.4e-04	0	1.2e-03
Anisotropic diffusion, 128^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	4.4e-04 (band)	0	–
	Jacobi	11	0.999988	0.999988	0.999869	0.999988	1.1e-16	0	2.4e-05
	Jacobi (0.67)	11	0.999992	0.999992	0.999912	0.999992	4.4e-03	0	1.6e-05
	GS	3	1.131668	0.999528	0.999537	0.999976	0	0	5.7e-02
	SymGS	2	1.102789	0.999489	0.999255	0.999953	0	0	5.5e-02
	SOR (1.5)	3	1.889774	0.999393	0.999471	0.999928	8.3e-01	0	1.5e-01
Anisotropic diffusion, 256^2	DeepONet corrector	1	1.000000	1.000000	1.000000	1.000000	1.7e-03 (band)	0	–
	Jacobi	38	0.999997	0.999997	0.999887	0.999997	1.1e-16	0	9.7e-05
	GS	11	1.131845	0.999245	0.999485	0.999994	0	0	4.0e-02
	SOR (1.5)	11	1.890225	0.999352	0.999473	0.999982	8.3e-01	0	1.1e-01

Table 33: Quantities of Theorem 4.1, Proposition 4.2 and Theorem 5.1 along the cost-aware oracle’s path to $\varepsilon = 10^{-8}$ (8 test instances; medians unless stated): number of macro-actions T , $\sum_i \rho_{O_i}^2$ with the energy-norm constants of the chosen macro-actions, the resulting $\alpha(O)$ of Proposition 4.2 (median over the instances on which $T - \sum_i \rho_{O_i}^2 > 0$; ∞ otherwise), the empirical supermodularity ratio $\hat{\alpha}$ of Equation (7) evaluated with S the oracle’s prefix and S' its suffix (maximum and median over prefixes and instances), and the normalised bounds of Theorem 5.1: the largest ratio $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ over visited states ($\bar{E}$) and the smallest value of $\max_j \tilde{c}_j / \|e^{(t)}\|^2$ ($E_{\min}$).

Equation	Ensemble	T	$\sum_i \rho_{O_i}^2$	$\alpha(O)$ (Prop. 4.2)	$\hat{\alpha}$ max	$\hat{\alpha}$ median	$\max_t \max_j \tilde{c}_j / \ e^{(t)}\ ^2$	$\min_t \max_j \tilde{c}_j / \ e^{(t)}\ ^2$
Poisson, 128 ²	{Jacobi}	52	51.775	6	1.000	0.420	1.000	6.3e-02
	{Jacobi(0.67)}	4	3.981	207	1.000	0.598	1.000	9.0e-02
	{GS}	6	5.971	138	1.000	0.498	1.000	1.4e-01
	{SymGS}	5	4.971	139	1.000	0.501	1.000	1.1e-01
	{SOR(1.5)}	9	8.852	27	1.000	0.376	1.000	1.2e-01
	{Multigrid}	4	1.029	1	1.000	0.417	1.000	9.6e-06
	{Jacobi, Jacobi(0.67)}	4	3.976	167	1.000	0.612	1.000	9.0e-02
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	4	3.976	167	1.000	0.612	1.000	1.4e-01
Poisson, 256 ²	{Jacobi}	10	9.486	297	1.000	0.459	1.000	3.6e-01
	{Jacobi(0.67)}	12	11.988	331	1.000	0.302	1.000	4.7e-01
	{GS}	26	26.447	76	1.000	0.201	1.000	6.3e-01
	{SymGS}	14	14.471	138	1.000	0.255	1.000	5.1e-01
	{SOR(1.5)}	24	23.928	55	1.000	0.207	1.000	5.4e-01
	{Multigrid}	4	1.037	1	1.000	0.418	1.000	1.1e-05
	{Jacobi, Jacobi(0.67)}	9	8.988	322	1.000	0.389	1.000	4.3e-01
Poisson, 512 ²	{Jacobi}	55	54.988	322 (∞ on 1/8)	1.000	0.236	1.000	7.8e-01
	{GS}	86	86.328	22 (∞ on 1/8)	1.000	0.154	1.000	8.4e-01
	{SOR(1.5)}	41	40.925	52 (∞ on 1/8)	1.000	0.166	1.000	7.4e-01
	{Multigrid}	4	1.043	1 (∞ on 1/8)	1.000	0.419	1.000	1.0e-05
Convection-diffusion, 128 ²	{Jacobi}	156	154.834	2	1.000	0.391	1.000	7.9e-02
	{Jacobi(0.67)}	5	4.978	184	1.000	0.503	1.000	1.2e-01
	{GS}	6	5.967	120	1.000	0.494	1.000	1.2e-01
	{SymGS}	5	4.971	138	1.000	0.501	1.000	1.0e-01
	{SOR(1.5)}	9	8.772	18	1.000	0.359	1.000	7.5e-02
	{Multigrid}	4	1.017	1	1.000	0.417	1.000	8.2e-06
	{Jacobi, Jacobi(0.67)}	4	3.982	222	1.000	0.659	1.000	1.2e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	4	3.982	222	1.000	0.615	1.000	1.2e-01
Convection-diffusion, 256 ²	{Jacobi}	26	25.971	185	1.000	0.464	1.000	5.0e-01
	{Jacobi(0.67)}	20	19.486	284	1.000	0.226	1.000	5.2e-01
	{GS}	28	28.444	71	1.000	0.186	1.000	5.9e-01
	{SymGS}	16	16.467	120	1.000	0.221	1.000	5.1e-01
	{SOR(1.5)}	24	23.921	50	1.000	0.207	1.000	5.2e-01
	{Multigrid}	4	1.021	1	1.000	0.418	1.000	2.8e-06
	{Jacobi, Jacobi(0.67)}	14	13.986	285	1.000	0.276	1.000	5.2e-01
Convection-diffusion, 512 ²	{Jacobi}	62	62.486	293	1.000	0.196	1.000	7.6e-01
	{GS}	94	93.813	21	1.000	0.138	1.000	8.4e-01
	{SOR(1.5)}	42	41.926	54	1.000	0.158	1.000	6.8e-01
	{Multigrid}	4	1.033	1	1.000	0.377	1.000	2.0e-06
Anisotropic diffusion, 128 ²	{Jacobi}	10	9.998	1386 (∞ on 1/8)	1.000	0.481	1.000	3.5e-01
	{Jacobi(0.67)}	5	4.999	7585 (∞ on 1/8)	1.000	0.504	1.000	3.5e-01
	{GS}	7	6.995	865 (∞ on 1/8)	1.000	0.500	1.000	3.5e-01
	{SymGS}	6	6.494	672 (∞ on 1/8)	1.000	0.435	1.000	3.5e-01
	{SOR(1.5)}	9	8.993	540 (∞ on 1/8)	1.000	0.397	1.000	3.5e-01
	{Jacobi, Jacobi(0.67)}	4	4.000	9130 (∞ on 1/8)	1.000	0.652	1.000	3.5e-01
	{Jacobi, Jacobi(0.67), GS, SymGS, SOR(1.5)}	4	4.000	9130 (∞ on 1/8)	1.000	0.639	1.000	3.5e-01
Anisotropic diffusion, 256 ²	{Jacobi}	3	3.000	17652 (∞ on 2/8)	1.000	0.608	1.000	2.9e-01
	{GS}	3	2.999	3888 (∞ on 2/8)	1.000	0.658	1.000	3.9e-01
	{SOR(1.5)}	4	3.498	1897 (∞ on 2/8)	1.000	0.578	1.000	2.2e-01

F Limitations

A primary limitation of our approach is the higher on-time computational cost requires to learn the greedy routing strategy. In contrast, the simpler baselines such as HINTS incur no training overhead. While this cost is amortized during inference time and leads to improved solution quality, it may be prohibitive in settings with limited computational resources or when rapid deployment is required. On the deployment side, Section E shows that the gains survive cost accounting: with a corrector that covers the smooth half of the spectrum and a classical smoother that covers the rest, one or two corrector calls suffice and the router’s advantage over fixed schedules is uniform across solvers, equations and tolerances. The corrector we use is specific to the constant-coefficient periodic setting (a fixed Fourier trunk basis on a coarse sensor grid), its cost grows with the fourth power of the sensor-grid size, and undamped Jacobi remains limited far below truncation level by the near-Nyquist modes that neither component damps. Our wall-clock study is restricted to single-thread CPU execution; with a corrector band that does not grow with the grid, the gains over fixed schedules shrink at 512² for the point smoothers, whereas pairing the corrector with geometric multigrid restores them (Section E). Ensembles of classical solvers pay off only where the error passes through regimes that favour different members; at truncation level a single well-chosen smoother suffices. For the constant-coefficient periodic test problems used here the discrete operator is diagonalised by the FFT, and the resulting direct solve—which we use as the reference—is faster than every iterative method we measured, including ours (about $3\times$ at 128² and $5\times$ at 512² against the best router); the hybrid is

1163 meant for problems on which such a direct diagonalisation is unavailable and the corrector must be
1164 learned rather than computed, and we have not evaluated such a problem in wall-clock terms.

1165 Additionally, the effectiveness of the learned router depends on the neural operator exhibiting
1166 heterogeneous performance across samples. When the neural operators produces predictions of
1167 relatively uniform quality, the benefit of adaptive routing diminishes and it is more practical to set a
1168 fixed schedule based on the quality of the neural operator. In practice, achieving such heterogeneity
1169 can be sensitive to training choices, and neural operators themselves can be finicky to train.

1170 **G LLM Usage**

1171 LLMs, specifically ChatGPT and Gemini, supported the writing process in an iterative manner. We
1172 drafted paragraphs and asked the models for feedback on grammar and clarity. We then incorporated
1173 selected suggestions into the writing and repeated this process until we were satisfied with the writing.

1174 The code developed for the experiments was written by the authors with the help of occasional code
1175 completions. The central components (e.g., the hybrid solver implementation and the greedy-router
1176 training pipelines) were implemented exclusively by the authors.

1177 All substantive intellectual contributions, which include ideas, theorems, and analyses, are our own.
1178 LLMs were occasionally used to verify the correctness of proofs, but all proof strategies originated
1179 from the authors and relevant literature.

1180 NeurIPS Paper Checklist

1181 The checklist is designed to encourage best practices for responsible machine learning research,
1182 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
1183 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
1184 follow the references and follow the (optional) supplemental material. The checklist does NOT count
1185 towards the page limit.

1186 Please read the checklist guidelines carefully for information on how to answer these questions. For
1187 each question in the checklist:

- 1188 • You should answer [Yes], [No], or [N/A].
- 1189 • [N/A] means either that the question is Not Applicable for that particular paper or the
1190 relevant information is Not Available.
- 1191 • Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

1192 **The checklist answers are an integral part of your paper submission.** They are visible to the
1193 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
1194 (after eventual revisions) with the final version of your paper, and its final version will be published
1195 with the paper.

1196 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
1197 While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a
1198 proper justification is given (e.g., error bars are not reported because it would be too computationally
1199 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
1200 [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we
1201 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
1202 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
1203 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
1204 please point to the section(s) where related material for the question can be found.

1205 IMPORTANT, please:

- 1206 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”,**
- 1207 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 1208 • **Do not modify the questions and only use the provided macros for your answers.**

1209 1. Claims

1210 Question: Do the main claims made in the abstract and introduction accurately reflect the
1211 paper’s contributions and scope?

1212 Answer: [Yes]

1213 Justification: The abstract clearly delineates the paper’s main contributions and scope, and
1214 the introduction reiterates these claims in a structured, bullet-point form. All stated claims
1215 are directly supported by the theoretical and experimental results presented in the paper.

1216 Guidelines:

- 1217 • The answer [N/A] means that the abstract and introduction do not include the claims
1218 made in the paper.
- 1219 • The abstract and/or introduction should clearly state the claims made, including the
1220 contributions made in the paper and important assumptions and limitations. A [No] or
1221 [N/A] answer to this question will not be perceived well by the reviewers.
- 1222 • The claims made should match theoretical and experimental results, and reflect how
1223 much the results can be expected to generalize to other settings.
- 1224 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1225 are not attained by the paper.

1226 2. Limitations

1227 Question: Does the paper discuss the limitations of the work performed by the authors?

1228 Answer: [Yes]

1229 Justification: The paper discusses key limitations in Appendix E.

1230 Guidelines:

- 1231 • The answer [N/A] means that the paper has no limitation while the answer [No] means
- 1232 that the paper has limitations, but those are not discussed in the paper.
- 1233 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1234 • The paper should point out any strong assumptions and how robust the results are to
- 1235 violations of these assumptions (e.g., independence assumptions, noiseless settings,
- 1236 model well-specification, asymptotic approximations only holding locally). The authors
- 1237 should reflect on how these assumptions might be violated in practice and what the
- 1238 implications would be.
- 1239 • The authors should reflect on the scope of the claims made, e.g., if the approach was
- 1240 only tested on a few datasets or with a few runs. In general, empirical results often
- 1241 depend on implicit assumptions, which should be articulated.
- 1242 • The authors should reflect on the factors that influence the performance of the approach.
- 1243 For example, a facial recognition algorithm may perform poorly when image resolution
- 1244 is low or images are taken in low lighting. Or a speech-to-text system might not be
- 1245 used reliably to provide closed captions for online lectures because it fails to handle
- 1246 technical jargon.
- 1247 • The authors should discuss the computational efficiency of the proposed algorithms
- 1248 and how they scale with dataset size.
- 1249 • If applicable, the authors should discuss possible limitations of their approach to
- 1250 address problems of privacy and fairness.
- 1251 • While the authors might fear that complete honesty about limitations might be used by
- 1252 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
- 1253 limitations that aren’t acknowledged in the paper. The authors should use their best
- 1254 judgment and recognize that individual actions in favor of transparency play an impor-
- 1255 tant role in developing norms that preserve the integrity of the community. Reviewers
- 1256 will be specifically instructed to not penalize honesty concerning limitations.

1257 **3. Theory assumptions and proofs**

1258 Question: For each theoretical result, does the paper provide the full set of assumptions and

1259 a complete (and correct) proof?

1260 Answer: [Yes]

1261 Justification: All theoretical results include clearly stated assumptions in the statement

1262 of theorems. Complete proofs are provided in Appendices A and B with appropriate

1263 cross-referencing from the main text.

1264 Guidelines:

- 1265 • The answer [N/A] means that the paper does not include theoretical results.
- 1266 • All the theorems, formulas, and proofs in the paper should be numbered and cross-
- 1267 referenced.
- 1268 • All assumptions should be clearly stated or referenced in the statement of any theorems.
- 1269 • The proofs can either appear in the main paper or the supplemental material, but if
- 1270 they appear in the supplemental material, the authors are encouraged to provide a short
- 1271 proof sketch to provide intuition.
- 1272 • Inversely, any informal proof provided in the core of the paper should be complemented
- 1273 by formal proofs provided in appendix or supplemental material.
- 1274 • Theorems and Lemmas that the proof relies upon should be properly referenced.

1275 **4. Experimental result reproducibility**

1276 Question: Does the paper fully disclose all the information needed to reproduce the main ex-

1277 perimental results of the paper to the extent that it affects the main claims and/or conclusions

1278 of the paper (regardless of whether the code and data are provided or not)?

1279 Answer: [Yes]

Justification: The paper provides sufficient detail to reproduce all experiments, including training procedures and implementation details described in the main text and Appendix C. Additionally, we release code with a comprehensive README to support the reproducibility of our results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to code with detailed instructions in a comprehensive README, including environment setup and commands to train routers and reproduce the main experimental results. These files will automatically generate the data needed for training. Data generation has also been documented in Appendix C.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: The paper specifies all key training and evaluation details in the Section 6 and Appendix C.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: The paper reports standard deviations in the main results tables, and further provides statistical significance via t-test p values in Appendix D.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The authors should answer [\[Yes\]](#) if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

1388 Answer: [Yes]

1389 Justification: The paper provides details on compute resources in Appendix C.

1390 Guidelines:

- 1391 • The answer [N/A] means that the paper does not include experiments.
- 1392 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
- 1393 or cloud provider, including relevant memory and storage.
- 1394 • The paper should provide the amount of compute required for each of the individual
- 1395 experimental runs as well as estimate the total compute.
- 1396 • The paper should disclose whether the full research project required more compute
- 1397 than the experiments reported in the paper (e.g., preliminary or failed experiments that
- 1398 didn't make it into the paper).

1399 **9. Code of ethics**

1400 Question: Does the research conducted in the paper conform, in every respect, with the

1401 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

1402 Answer: [Yes]

1403 Justification: This paper does not involve any human subjects and all data has been simu-

1404 lated. This research focuses on numerical methods for physical systems and does not raise

1405 concerns related to safety, security, discrimination, surveillance, deception & harassment,

1406 environment, human rights, and bias & fairness.

1407 Guidelines:

- 1408 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
- 1409 Ethics.
- 1410 • If the authors answer [No], they should explain the special circumstances that require a
- 1411 deviation from the Code of Ethics.
- 1412 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
- 1413 eration due to laws or regulations in their jurisdiction).

1414 **10. Broader impacts**

1415 Question: Does the paper discuss both potential positive societal impacts and negative

1416 societal impacts of the work performed?

1417 Answer: [Yes]

1418 Justification: The paper discusses broader impacts in the introduction section. No direct

1419 negative societal impacts are anticipated.

1420 Guidelines:

- 1421 • The answer [N/A] means that there is no societal impact of the work performed.
- 1422 • If the authors answer [N/A] or [No], they should explain why their work has no societal
- 1423 impact or why the paper does not address societal impact.
- 1424 • Examples of negative societal impacts include potential malicious or unintended uses
- 1425 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
- 1426 (e.g., deployment of technologies that could make decisions that unfairly impact specific
- 1427 groups), privacy considerations, and security considerations.
- 1428 • The conference expects that many papers will be foundational research and not tied
- 1429 to particular applications, let alone deployments. However, if there is a direct path to
- 1430 any negative applications, the authors should point it out. For example, it is legitimate
- 1431 to point out that an improvement in the quality of generative models could be used to
- 1432 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
- 1433 that a generic algorithm for optimizing neural networks could enable people to train
- 1434 models that generate Deepfakes faster.
- 1435 • The authors should consider possible harms that could arise when the technology is
- 1436 being used as intended and functioning correctly, harms that could arise when the
- 1437 technology is being used as intended but gives incorrect results, and harms following
- 1438 from (intentional or unintentional) misuse of the technology.

- 1439 • If there are negative societal impacts, the authors could also discuss possible mitigation
1440 strategies (e.g., gated release of models, providing defenses in addition to attacks,
1441 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
1442 feedback over time, improving the efficiency and accessibility of ML).

1443 11. Safeguards

1444 Question: Does the paper describe safeguards that have been put in place for responsible
1445 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
1446 image generators, or scraped datasets)?

1447 Answer: [N/A]

1448 Justification: The paper does not release any models or datasets at all. It focuses on method-
1449 ological contributions for numerical solvers using simulated data. Therefore, safeguards are
1450 not applicable.

1451 Guidelines:

- 1452 • The answer [N/A] means that the paper poses no such risks.
- 1453 • Released models that have a high risk for misuse or dual-use should be released with
1454 necessary safeguards to allow for controlled use of the model, for example by requiring
1455 that users adhere to usage guidelines or restrictions to access the model or implementing
1456 safety filters.
- 1457 • Datasets that have been scraped from the Internet could pose safety risks. The authors
1458 should describe how they avoided releasing unsafe images.
- 1459 • We recognize that providing effective safeguards is challenging, and many papers do
1460 not require this, but we encourage authors to take this into account and make a best
1461 faith effort.

1462 12. Licenses for existing assets

1463 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
1464 the paper, properly credited and are the license and terms of use explicitly mentioned and
1465 properly respected?

1466 Answer: [N/A]

1467 Justification: The paper does not use any external datasets or pretrained models. All
1468 experiments are conducted using simulated data and models trained in-house.

1469 Guidelines:

- 1470 • The answer [N/A] means that the paper does not use existing assets.
- 1471 • The authors should cite the original paper that produced the code package or dataset.
- 1472 • The authors should state which version of the asset is used and, if possible, include a
1473 URL.
- 1474 • The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- 1475 • For scraped data from a particular source (e.g., website), the copyright and terms of
1476 service of that source should be provided.
- 1477 • If assets are released, the license, copyright information, and terms of use in the
1478 package should be provided. For popular datasets, `paperswithcode.com/datasets`
1479 has curated licenses for some datasets. Their licensing guide can help determine the
1480 license of a dataset.
- 1481 • For existing datasets that are re-packaged, both the original license and the license of
1482 the derived asset (if it has changed) should be provided.
- 1483 • If this information is not available online, the authors are encouraged to reach out to
1484 the asset's creators.

1485 13. New assets

1486 Question: Are new assets introduced in the paper well documented and is the documentation
1487 provided alongside the assets?

1488 Answer: [Yes]

1489 Justification: We release code for our method which is documented with a detailed
1490 README.

1491 Guidelines:

1492 • The answer [N/A] means that the paper does not release new assets.

1493 • Researchers should communicate the details of the dataset/code/model as part of their

1494 submissions via structured templates. This includes details about training, license,

1495 limitations, etc.

1496 • The paper should discuss whether and how consent was obtained from people whose

1497 asset is used.

1498 • At submission time, remember to anonymize your assets (if applicable). You can either

1499 create an anonymized URL or include an anonymized zip file.

1500 **14. Crowdsourcing and research with human subjects**

1501 Question: For crowdsourcing experiments and research with human subjects, does the paper

1502 include the full text of instructions given to participants and screenshots, if applicable, as

1503 well as details about compensation (if any)?

1504 Answer: [N/A]

1505 Justification: The paper does not involve crowdsourcing or any research with human subjects.

1506 All experiments are conducted using simulated data.

1507 Guidelines:

1508 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1509 with human subjects.

1510 • Including this information in the supplemental material is fine, but if the main contribu-

1511 tion of the paper involves human subjects, then as much detail as possible should be

1512 included in the main paper.

1513 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

1514 or other labor should be paid at least the minimum wage in the country of the data

1515 collector.

1516 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1517 **subjects**

1518 Question: Does the paper describe potential risks incurred by study participants, whether

1519 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1520 approvals (or an equivalent approval/review based on the requirements of your country or

1521 institution) were obtained?

1522 Answer: [N/A]

1523 Justification: This paper does not involve human subjects. All experiments are conducted

1524 using simulated data. Therefore, IRB approval is not applicable.

1525 Guidelines:

1526 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1527 with human subjects.

1528 • Depending on the country in which research is conducted, IRB approval (or equivalent)

1529 may be required for any human subjects research. If you obtained IRB approval, you

1530 should clearly state this in the paper.

1531 • We recognize that the procedures for this may vary significantly between institutions

1532 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the

1533 guidelines for their institution.

1534 • For initial submissions, do not include any information that would break anonymity (if

1535 applicable), such as the institution conducting the review.

1536 **16. Declaration of LLM usage**

1537 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1538 non-standard component of the core methods in this research? Note that if the LLM is used

1539 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1540 scientific rigor, or originality of the research, declaration is not required.

1541 Answer: [N/A]

1542 Justification: LLMs are primarily used for writing and editing. Further details are provided
1543 in Appendix F.
1544 Guidelines:
1545 • The answer [N/A] means that the core method development in this research does not
1546 involve LLMs as any important, original, or non-standard components.
1547 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1548 be described.